



Bundesamt
für Sicherheit in der
Informationstechnik

Deutschland
Digital•Sicher•BSI

Detailspezifikationen zur TR-03109-1

Anforderungen an die Interoperabilität der Kommunikationseinheit eines
intelligenten Messsystems

Version 2.0
13.12.2024



Bundesamt für Sicherheit in der Informationstechnik
Postfach 20 03 63
53133 Bonn
E-Mail: smartmeter@bsi.bund.de
Internet: <https://www.bsi.bund.de>
© Bundesamt für Sicherheit in der Informationstechnik 2024

Inhalt

1	Einleitung	1
2	Wake-Up-Nachricht	2
2.1	Wake-Up-Nachricht Version 1	2
3	LMN-Zertifikatsprofil	4
3.1	Einleitung	4
3.2	Laufzeit	4
3.3	Zertifikatsstruktur	4
3.4	LMN-TLS-Zertifikate der Messeinrichtungen	7
3.5	LMN-TLS-Zertifikate des SMGW	7
4	HAN-Zertifikatsprofil	9
4.1	Einleitung	9
4.2	Laufzeit	10
4.3	Zertifikatsstruktur	10
4.4	Anforderungen an die HAN-TLS-Zertifikate des SMGW	13
4.5	Anforderungen an die TLS-Zertifikate der HAN-Teilnehmer	14
4.6	Anforderungen an das CA-Zertifikat zur Validierung der Zertifikate der Servicetechniker	14
5	CMS - Cryptographic Message Syntax	15
5.1	Einleitung	15
5.2	Anforderungen	15
5.3	Datenstrukturen	16
5.4	AES-CBC-CMAC Algorithm Identifier und Parameter	23
5.5	Nutzung des Sicherheitsmoduls bei der CMS Inhaltsdatensicherung	24
6	RESTful Webservice Protokoll	26
6.1	Einleitung	26
6.2	Anforderungen an HTTP-Server	26
6.3	Anforderungen an HTTP-Clients	28
6.4	Content-Types	30
6.5	Status Codes	30
6.6	Header-Felder	32
6.7	URI-Pfade für die HAN-API	32
6.8	URI Query-Parameter	33
6.9	Kanonisierung für JSON Signaturen	34
7	HTTP-Authentifizierungs-Protokoll	36
7.1	Einleitung	36
7.2	Anforderungen an den HTTP-Server	36
8	LMN HDLC Protokoll	37
8.1	Einleitung	37
8.2	Anforderungen	38
8.3	Technische Anwendungsfälle der HDLC Primary Station (Client)	38

8.4	Implementierungshinweise	39
9	SML - Smart Message Language	41
9.1	Einleitung	41
9.2	Datenstrukturen	41
9.3	TA des SML-Clients	41
10	Wireless M-Bus Protokoll	48
10.1	Einleitung	48
10.2	Datenstrukturen	49
10.3	Anforderungen an den Funkempfänger	51
10.4	TA.WMBUS.ProcessMeterData	52
11	SOCKS-TLS Protokoll	54
11.1	Einleitung	54
11.2	Technische Anwendungsfälle des SOCKS-TLS Clients	54
11.3	Technische Anwendungsfälle des SOCKS-TLS Servers	55
11.4	Technische Anwendungsfälle für SOCKS-TLS Client und Server	56
12	mDNS - Device/Service Discovery Protokoll	59
12.1	Einleitung	59
12.2	Technische Anwendungsfälle des mDNS-Responders	59
12.3	Technische Anwendungsfälle für das mDNS Querying	60
13	TLS - Transport Layer Security Protokoll	63
13.1	Einleitung	63
13.2	Datenstrukturen	63
13.3	Technische Anwendungsfälle	64
13.4	Nutzung des Sicherheitsmoduls beim TLS-Handshake des SMGW	78
14	NTP - Network Time Protokoll	81
14.1	Einleitung	81
14.2	Datenstrukturen	81
14.3	Technische Anwendungsfälle des NTP-Clients	82
15	Messwertstatus	87
15.1	Kombiniertes Statuswort - Codierung und Bedeutung	87
	Literaturverzeichnis	90
	Glossar	93
A	Abkürzungsverzeichnis	94
	Anforderungsverzeichnis	96
	ICS-Anforderungen	102

1 Einleitung

Die folgenden Kapitel enthalten die Detailspezifikationen zur Interoperabilität für die Technische Richtlinie BSI TR-03109-1.

Die Detailspezifikationen dieses Dokumentes konkretisieren die aus den Anwendungsfällen und Kommunikationsszenarien des SMGW und dessen Kommunikationsadapter referenzierten Standards.

Zielgruppe dieses Dokumentes sind daher Geräteentwickler und Prüfstellen für das SMGW und dessen Kommunikationsadapter.

Die in diesem Dokument enthaltenen Spezifikationen referenzieren in der Regel Universalspezifikationen oder Teile davon und können weitere anwendungsspezifische Anforderungen enthalten. Universalspezifikationen sind beispielsweise Normen, Standards, Richtlinien des BSI, der IETF und weiterer nationaler, europäischer oder internationaler Normungsorganisationen.

Die in diesem Dokument enthaltenen, einzelnen Kapitel widmen sich unterschiedlichsten Bereichen. Sie sind daher nicht zur zusammenhängenden Lektüre gedacht.

2 Wake-Up-Nachricht

2.1 Wake-Up-Nachricht Version 1

Die Wake-Up-Nachricht enthält insbesondere

- die Identifikation zum Aufbau der Datenstruktur und zur Bedeutung der Datenfelder
- die Geräteidentifikation des SMGW
- einen Zeitstempel des Absenders
- die Signatur des Absenders

Das SMGW **MUSS** Wake-Up-Nachrichten mit folgender Struktur und Bedeutung verarbeiten können: [REQ.WakeUp.Datenstruktur.10]

Feld	#Bytes	Beschreibung
Header	2	Header = „WU“ (ASCII “57h 55h” = “0101.0111b 0101.0101b”). Dient zur Kennzeichnung der Wake-Up-Nachricht und ermöglicht eine erste einfache (hardwarenahe) Überprüfung bzw. Klassifizierung der empfangenen Nachrichten.
VersionId	1	Wake-Up-Nachricht Version = 01h. Bezeichnet die verwendete Version des Wake-Up-Protokolles. Bei eventuellen zukünftigen Erweiterungen werden neue Versionsnummern vergeben. Die Version 01h impliziert die Anforderung eines Management-Kanalaufbaus zum GWA.
RecipientId	9	Eineindeutige Geräte-Identifikation des SMGW. Kodierung gemäß [DIN43849] Byte[1]: Sparte (01h..0Fh): 0Eh=Kommunikation Byte[2-4]: Herstellerkennzeichnung (3 ASCII Großbuchstaben) gemäß FLAG Registrierung. Zum Beispiel: „BSI“ 42 53 49h Die Kodierung erfolgt beginnend mit dem höchstwertigen Byte. Byte[5]: Fabrikationsblock (00h..FEh) Byte[6-9]: Fabrikationsnummer rechtsbündig mit führenden Nullen (8 Dezimalstellen 0000 0000 - 9999 9999). Die Kodierung erfolgt als 32 Bit Unsigned Integer und beginnend mit dem höchstwertigen Byte. Dient zur eindeutigen Identifizierung des SMGW. Die Vergabe und Bedeutung der RecipientId ist in [DIN43849] beschrieben.¹
Timestamp	8	UTC Unix Time als 64 Bit Signed Integer (Anzahl Sekunden seit dem 1. Januar 1970 00:00:00 UTC). Zum Beispiel: „13. Juli 2012 11:01:20 UTC“ „1.342.177.280d“ = „00 00 00 00 50 00 00 00h“ Die Kodierung erfolgt beginnend mit dem niederwertigen Byte. Enthält die aktuelle Zeit (in UTC) zum Zeitpunkt der Erstellung der Wake-Up-Nachricht. Geringfügige Unterschiede zwischen den jeweiligen Uhrzeiten auf den Servern und den SMGW sind üblich. Das SMGW prüft, ob der Timestamp im festgelegten Zeitfenster relativ zur Uhrzeit des SMGW liegt. Der Timestamp dient dazu, dass eine einzelne Wake-Up-Nachricht nicht mehrfach für den Aufbau von TLS-Verbindungen wiederverwendet werden kann (Replay-Attacke).
Padding / Reserviert	12	Zwölf mal "0Bh"
SignatureFormat	1	Dient zur Kennzeichnung welches Signaturformat in der Wake-Up-Nachricht verwendet wurde. Plain Format = 01h

¹ Nur das adressierte SMGW darf die Wake-Up-Nachricht verarbeiten. Hiermit soll verhindert werden, dass die Wake-Up-Nachricht von einem Angreifer missbraucht wird, um eine Vielzahl von SMGW in der Verantwortung eines GWA zu einem gleichzeitigen TLS-Callback zu verleiten (DoS-Attacke).

Feld	#Bytes	Beschreibung
SignatureAlgorithmOIDLength	1	Länge des folgenden SignatureAlgorithmObjectIdentifiers.
SignatureAlgorithmOID	1..14	OID des verwendeten Signaturalgorithmus gemäß [TR-03109-3] ohne Padding Zum Beispiel: ecdsa-plain-SHA256 ::= { itu-t(0) identified-organization(4) etsi(0) reserved(127) etsi-identified-organization(0) bsi-de(7) algorithms(1) id-ecc(1) signatures(4) ecdsa-plain-signatures(1) 3 }
Signature	2*L	Signatur R S über die Datenfelder "Header" bis "Padding" inklusive. Octet string R = I2OS(r; L) und S = I2OS(s; L) Die Länge L bestimmt sich aus der elliptischen Kurve des öffentlichen Signaturschlüssels zum GWA_WAN_SIG_CRT.

Tabelle 2.1 Datenstruktur der Wake-Up-Nachricht

Das SMGW **MUSS** den Hash über die Datenfelder "Header" bis einschließlich "Padding" nach [TR-03111] Kap 4.1.2.1 mit dem öffentlichen Schlüssel des GWA_WAN_SIG_CRT validieren. [REQ.WakeUp.Datenstruktur.20]

Das SMGW **MUSS** für die Validierung der Signatur das über **SignatureAlgorithmOID** bestimmte Hash- und Signaturverfahren gemäß [TR-03109-3] und die durch den öffentlichen Signaturschlüssel des GWA_WAN_SIG_CRT bestimmte elliptische Kurve verwenden. [REQ.WakeUp.Datenstruktur.30]

Die in ►Tabelle 2.1 abgebildete Struktur wird als Wake-Up-Nachricht an das SMGW versendet.

Header (2 Bytes)		Vers. (1 B)	RecipientId (9 Bytes)									Timestamp (8 Bytes)			
,W'	,U'	01h	0Eh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh
Timestamp (continued)			Padding / Reserved (11 Bytes + 1 Byte Padding-Length)												
xxh	xxh	xxh	xxh	0Bh	0Bh	0Bh	0Bh	0Bh	0Bh	0Bh	0Bh	0Bh	0Bh	0Bh	0Bh
Sig.	OID	SignatureAlgorithmOID und Padding													
Frm	Len	(OID-Length Bytes)													
01h	0Ah	04h	00h	7Fh	00h	07h	01h	01h	04h	01h	xxh	-	-	-	-
ECDSA-Signature (r) (L Bytes)															
xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh
xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh
xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh
ECDSA-Signature (s) (L Bytes)															
xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh
xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh
xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh	xxh

Tabelle 2.2 Struktur Wake-Up-Nachricht

3 LMN-Zertifikatsprofil

3.1 Einleitung

Die selbstsignierten LMN-Zertifikate, die das SMGW erstellt und verwendet, verwenden die Syntax und Semantik von X.509-Zertifikaten in der Version 3 nach [RFC5280] Kapitel 4 und **SOLLEN** konform zu dem in diesem Kapitel beschriebenen Zertifikatsprofil sein. [REQ.ZertifikateLMN.Allgemein.10]

- Typ A: Selbstsigniert, nicht aus der SM-PKI



ICS.ZertifikateLMN.Allgemein.10

Der Hersteller **MUSS** im ICS deklarieren, ob das SMGW die Anforderungen an die LMN-Zertifikatsprofile für MTR und SMGW für die vom SMGW erzeugten und verarbeiteten LMN-Zertifikate umsetzt.



ICS.ZertifikateLMN.Allgemein.11

Der Hersteller **MUSS** im ICS beschreiben, welche Abweichungen in der Implementierung bei Erzeugung oder Verarbeitung von Selbstsignierten Zertifikaten zu den Typ A-LMN-Zertifikatsprofilen bestehen.

Zertifikatstyp	Zertifikats-Profiltyp	Aussteller
GW_LMN_TLS_CRT	Typ A	SMGW
MTR_LMN_TLS_CRT	Typ A	SMGW

Tabelle 3.1 Zertifikatsprofiltypen der LMN-Zertifikate

Die folgenden Abschnitte enthalten Tabellen in denen Anforderungen an das Vorhandensein von Attributen und Datenfelder verschiedener Zertifikatsprofile beschrieben sind. Darin werden die folgenden Abkürzungen verwendet:

Zeichen	Langform	Bedeutung
m	Mandatory	Das Element muss vorhanden sein.
x	Not existent	Das Element darf nicht vorhanden sein.
o	Optional	Das Element kann vorhanden sein
c	Conditional	Das Element ist abhängig von einem anderen Element vorhanden
r	Recommended	Das Element soll vorhanden sein.

Tabelle 3.2 Beschreibung der Abkürzung zum Vorhandensein von Zertifikats-Elementen

3.2 Laufzeit

Die Laufzeit der selbstsignierten LMN-Zertifikate ist in [TR-03109-3] beschrieben.

3.3 Zertifikatsstruktur

Zertifikatsfeld	Referenz in RFC5280	Element vorhanden	Wert
TBSCertificate	4.1.1.1	m	Siehe ►Tabelle 3.4
SignatureAlgorithm	4.1.1.2	m	Siehe ►Abschnitt 3.3.1
SignatureValue	4.1.1.3	m	Abhängig vom gewählten Signatur-Algorithmus. Mit dem privaten Schlüssel zum "subjectPublicKeyInfo" selbstsigniert.

Zertifikatsfeld	Referenz in RFC5280	Element vorhanden	Wert
Größe von Certificate		r	<=600 Bytes.

Tabelle 3.3 Struktur des Elementes "Certificate"

Die folgende Tabelle gibt die Struktur des Feldes "TBSCertificate" verbindlich vor.

Zertifikatsfeld	Referenz in RFC5280	Element vorhanden	Wert
Version	4.1.2.1	m	'v3'
SerialNumber	4.1.2.2	m	Zufällig gewählte, positive Ganzzahl; bestimmt vom Aussteller (8-20 Octets).
Signature	4.1.2.3	m	Gleicher Wert wie im Feld "SignatureAlgorithm" (s. ▶Abschnitt 3.3.1).
Issuer	4.1.2.4	m	Identisch zu "Subject".
Validity	4.1.2.5	m	Zeitpunkte für Beginn und Ende der Gültigkeit des Zertifikates (s. ▶Abschnitt 3.3.4).
Subject	4.1.2.6	m	Eindeutiger Name (Distinguished Name, DN) des Zertifikatsinhabers siehe ▶Abschnitt 3.3.3.
SubjectPublicKeyInfo	4.1.2.7	m	Siehe ▶Abschnitt 3.3.2.
IssuerUniqueId	4.1.2.8	x	Entfällt
SubjectUniqueId	4.1.2.8	x	Entfällt
Extensions	4.1.2.9	m	Siehe ▶Abschnitt 3.3.5.

Tabelle 3.4 Struktur des Elementes "TBSCertificate"

3.3.1 SignatureAlgorithm

Durch die Datenstruktur SignatureAlgorithm wird nach [RFC5280] der Signaturalgorithmus des Zertifikats angegeben. Dieser besteht aus der folgenden Datenstruktur:

```
AlgorithmIdentifier ::= SEQUENCE {algorithm OBJECT IDENTIFIER, parameters ANY DEFINED BY algorithm OPTIONAL}
```

Der Wert von "algorithm" wird - angelehnt an die Vorgaben für Sub-CA nach [TR-03109-3] - mit ECDSA-with-SHA256 oder ECDSA-with-SHA384 angegeben. Das Datenfeld "parameters" bleibt leer.

3.3.2 SubjectPublicKeyInfo

Das Feld "SubjectPublicKeyInfo" besitzt folgende Struktur (s. [RFC5480]):

```
SubjectPublicKeyInfo ::= SEQUENCE {algorithm AlgorithmIdentifier, subjectPublicKey BIT STRING}
```

Die OID im Datenfeld "algorithm" enthält den Wert 1.2.840.10045.2.1 (id-ecPublicKey). Im Feld "ECParameters" ist gemäß [RFC5480] die Variante namedCurve zu verwenden. Die für Kurvenparameter bei der TLS-Kommunikation im LMN vorzusehenden Werte sind in [TR-03109-3] beschrieben.

Der PublicKey wird mit unkomprimierten Punktkoordinaten angegeben.

3.3.3 Issuer und Subject

- Für selbstsignierte LMN-Zertifikate sind "subject" und "issuer" identisch.
- Einzelne Attribute von Issuer und Subject dürfen nicht länger als 64 Zeichen sein.
- Es wird zwischen den Funktionsrollen SMGW und MTR unterschieden.
- Die LMN-Zertifikate (SMGW und MTR) stammen nicht aus der SM-PKI.

- Vorgaben an das Namensschema von Issuer und Subject sind in ▶Abschnitt 3.5 und ▶Abschnitt 3.4 beschrieben.

3.3.4 Validity

- Das Feld "notBefore" enthält den Erzeugungszeitpunkt des Schlüsselpaars und des Zertifikates.
- Das Feld "notAfter" enthält den Zeitpunkt, nach dem das Zertifikat und das dazugehörige Schlüsselpaar nicht mehr verwendet werden darf. Der Zeitpunkt muss nach dem Erzeugungszeitpunkt liegen.

3.3.5 Extensions

Die nun folgende Tabelle enthält eine Übersicht der Extensions und den Anforderungen an deren Vorhandensein. Weitere Zertifikats-Extensions sind nicht erlaubt.

Nr.	Bezeichnung	MTR_LMN_TLS_CRT Element vorhanden	GW_LMN_TLS_CRT Element vorhanden
1	AuthorityKeyIdentifier	x	x
2	SubjectKeyIdentifier	o	o
3	KeyUsage	r	r
4	PrivateKeyUsagePeriod	x	x
5	CertificatePolicies	x	x
6	SubjectAltNames	c (siehe ▶Abschnitt 3.3.5.3)	c (siehe ▶Abschnitt 3.3.5.3)
7	IssuerAltName	x	x
8	BasicConstraints	r	r
9	ExtendedKeyUsage	r ¹	r ¹
10	CRLDistributionPoints	x	x

Tabelle 3.5 Extensions

3.3.5.1 KeyUsage

- Extension-ID (OID): 2.5.29.15
- Kritisch: Ja
- Beschreibung: Die Extension KeyUsage (spezifiziert in [RFC5280], 4.2.1.1) definiert, für welche Zwecke der zertifizierte öffentliche Schlüssel verwendet werden darf.
- Gesetzte Bits: digitalSignature(0)

3.3.5.2 BasicConstraints

- Extension-ID (OID): 2.5.29.19
- Kritisch: Nein
- Beschreibung: Diese Extension (spezifiziert in [RFC5280], 4.2.1.9) gibt an, ob es sich bei dem gegebenen Zertifikat um eine CA handelt und wie viele CAs ihr folgen können.
- "cA": FALSE

3.3.5.3 SubjectAltName

- Extension-ID (OID): 2.5.29.27
- Kritisch: Ja

¹ Für die Verwendung mit id-kp-serverAuth und id-kp-clientAuth

- Beschreibung: Diese Extension (spezifiziert in [RFC5280], 4.2.1.6) ermöglicht die Bindung von Identitäten an das Subject des Zertifikates. Diese Angabe ist alternativ oder zusätzlich zur Identifikation im Subject möglich.
- "dNSName": Es wird empfohlen, die eindeutige Kennung von SMGW oder der Messeinrichtung basierend auf [DIN43849] als Domain Name Label mit angehängtem ".sm" zu formatieren.

3.4 LMN-TLS-Zertifikate der Messeinrichtungen

3.4.1 Einleitung

Zusätzlich zu den Anforderungen nach ▶Abschnitt 3.3 gelten für *MTR_LMN_TLS_CRT*-Zertifikate die Anforderungen in ▶Abschnitt 3.4.

3.4.2 Issuer und Subject

Für Zertifikate der Messeinrichtung gilt für die Bildung des *SubjectCN* die Bildungsregel <id>.<idscheme>

- Die Identifikation (<id>) enthält die herstellerübergreifende Identifikationsnummer für Messeinrichtungen nach [DIN43849] mit der Sparte "1" (Elektrizität), "5" (Kälte), "6" (Wärme), "7" (Gas), "8" (Kaltwasser), oder "9" (Warmwasser).
- Das Identifikationsschema (<idscheme>) enthält den Text "sm" ("Smart Meter")
- Der *SubjectCN* ist Case-insensitiv zu interpretieren.

3.5 LMN-TLS-Zertifikate des SMGW

3.5.1 Einleitung

Zusätzlich zu den Anforderungen nach ▶Abschnitt 3.3 gelten für *GW_LMN_TLS_CRT*-Zertifikate die Anforderungen in ▶Abschnitt 3.5.

3.5.2 Issuer und Subject

Für Gateway-Zertifikate gilt für die Bildung des *SubjectCN* die Bildungsregel <id>.<idscheme>:

- Die Identifikation (<id>) enthält die herstellerübergreifende Identifikationsnummer für Messeinrichtungen nach [DIN43849] mit der Sparte "E".
- Das Identifikationsschema (<idscheme>) enthält den Text "sm"
- Der *SubjectCN* ist Case-insensitiv zu interpretieren.

3.5.3 Beispiel

```
Certificate:
Data:
  Version: 3 (0x2)
  Serial Number: e8:a5:fe:ee:52:36:de:56
  Signature Algorithm: ecdsa-with-SHA256
  Issuer: CN = ebsi0012345678.sm
  Validity
    Not Before: Jul  1 00:00:00 2018 GMT
    Not After : Jul  1 00:00:00 2025 GMT
  Subject: CN = ebsi0012345678.sm
  Subject Public Key Info:
    Public Key Algorithm: id-ecPublicKey
    Public-Key: (256 bit)
      pub: 04:0c:fa:22:10:64:6d:05:c9:ff:ab:91:71:86:ca:
          94:3b:6a:08:8e:f7:78:ab:a0:3c:4f:fe:22:48:97:
          71:18:6e:13:dd:c2:bd:47:19:cf:2b:58:16:a3:d5:
          fc:7d:57:b8:02:63:71:47:f2:c7:36:f7:5d:51:b5:
          bf:ac:7f:b0:c3
```

```
ASN1 OID: brainpoolP256r1
X509v3 extensions:
  X509v3 Basic Constraints:
    CA:FALSE
  X509v3
    Key Usage: Digital Signature
    Signature
      Algorithm: ecdsa-with-SHA256
      30:45:02:20:2b:6d:80:e3:ea:a0:c1:19:5d:69:72:e1:e6:0c:
      76:77:0f:52:4d:55:6b:6e:44:31:fc:22:4a:6e:a7:2b:5d:3a:
      02:21:00:a3:7c:75:ed:3a:61:68:0e:ef:8b:4b:6b:47:36:05:
      b5:ec:26:45:68:37:6a:ed:2b:b0:a5:ba:3e:79:db:2b:78
```

Beispiel 3.1. Zertifikat nach den beschriebenen Anforderungen

4 HAN-Zertifikatsprofil

4.1 Einleitung

Dieses Kapitel beschreibt die Anforderungen an die Struktur, Datenfelder und Erweiterung von HAN-Zertifikaten für die Rollen SMGW, SRV und CLS. Das Kapitel wird sowohl aus der [TR-03109-1v2.0] (SMGW Anforderungen) als auch aus der [TR-03109-5] (CLS Anforderungen) referenziert. Die Herstellererklärungen (ICS) richten sich entsprechend an SMGW-Hersteller (GWH) oder CLS-Hersteller.

Die HAN-Zertifikate, die die Implementierung für sich erstellt, verwendet und akzeptiert, verwenden die Syntax und Semantik von X.509-Zertifikaten in der Version 3 nach [RFC5280] Kapitel 4 und **SOLLEN** je nach Zertifikatstyp in ►Tabelle 4.1 konform zu einem der in diesem Kapitel beschriebenen Zertifikatsprofiltypen sein: [REQ.ZertifikateHAN.Allgemein.10]

- CT_Selfsigned (Typ A): Selbstsigniert, nicht aus einer PKI
- CT_CA_Signed (Typ B): PKI-Zertifikat durch eine CA des GWA oder GWH ausgestellt
- CT_SMPKI_Signed (Typ C): C_{TLS}(SMGW)-Zertifikat der SM-PKI gemäß [TR-03109-4]



ICS.ZertifikateHAN.Allgemein.10

Der GWH **MUSS** im ICS deklarieren, ob die Implementierung die Anforderungen an die Verarbeitung von CT_Selfsigned HAN-Zertifikatsprofilen für die GW_HAN_TLS_CRT-Zertifikate umsetzt.



ICS.ZertifikateHAN.Allgemein.11

Der GWH **MUSS** im ICS beschreiben, welche Abweichungen in der Implementierung bei Verarbeitung von selbstsignierten GW_HAN_TLS_CRT-Zertifikaten zu den HAN-Zertifikatsprofilen vom Typ CT_Selfsigned bestehen.



ICS.ZertifikateHAN.Allgemein.12

Der Hersteller eines CLS **MUSS** im ICS deklarieren, ob die Implementierung die Anforderungen an die CT_Selfsigned HAN-Zertifikatsprofile für die CLS_HAN_TLS_CRT-Zertifikate umsetzt.



ICS.ZertifikateHAN.Allgemein.13

Der Hersteller eines CLS **MUSS** im ICS beschreiben, welche Abweichungen in der Implementierung von selbstsignierten CLS_HAN_TLS_CRT-Zertifikaten zu den HAN-Zertifikatsprofilen vom Typ CT_Selfsigned bestehen.



ICS.ZertifikateHAN.Allgemein.20

Der GWH **MUSS** im ICS deklarieren, ob das SMGW die Anforderungen an die CT_CA_Signed HAN-Zertifikatsprofile für SRV und SMGW für die vom SMGW erzeugten und verarbeiteten HAN-Zertifikate umsetzt.



ICS.ZertifikateHAN.Allgemein.21

Der Hersteller eines CLS **MUSS** im ICS beschreiben, welche Abweichungen in der Implementierung bei Erzeugung oder Verarbeitung von PKI-Zertifikaten zu den CT_CA_Signed-HAN-Zertifikatsprofilen bestehen.

Zertifikatstyp	Zertifikats-Profiltyp	Inhaber (Subject)	Aussteller (Issuer)
GW_HAN_TLS_CRT	CT_Selfsigned	SMGW	SMGW
GW_HAN_TLS_CRT (entspricht GW_WAN_TLS_CRT)	CT_SMPKI_Signed	SMGW	SUBCA SM-PKI

Zertifikatstyp	Zertifikats-Profiltyp	Inhaber (Subject)	Aussteller (Issuer)
SRV_HAN_TLS_CRT	CT_Selfsigned	SRV	SRV
SRV_HAN_TLS_CRT	CT_CA_Signed	SRV	GWACA, GWHCA
CON_HAN_TLS_CRT	CT_Selfsigned	CON	CON
CLS_HAN_TLS_CRT	CT_Selfsigned	CLS	CLS ¹

Tabelle 4.1 Zertifikatsprofiltypen der HAN-Zertifikate

Die folgenden Abschnitte enthalten Tabellen in denen Anforderungen an das Vorhandensein von Attributen und Datenfeldern verschiedener Zertifikatsprofile beschrieben sind. Darin werden die folgenden Abkürzungen verwendet:

Zeichen	Langform	Bedeutung
m	Mandatory	Das Element muss vorhanden sein.
x	Not existent	Das Element darf nicht vorhanden sein.
o	Optional	Das Element kann vorhanden sein
c	Conditional	Das Element ist abhängig von einem anderen Element vorhanden
r	Recommended	Das Element soll vorhanden sein.

Tabelle 4.2 Beschreibung der Abkürzung zum Vorhandensein von Zertifikats-Elementen

4.2 Laufzeit

Eine Prüfung der zeitlichen Gültigkeit von CT_Selfsigned Zertifikaten durch die Implementierung wird nicht vorgegeben, da es Aufgabe des Administrators ist, dass die Implementierung nur gültigen, selbst-signierten Zertifikaten vertraut und dies im Falle des SMGW über die HAN- und Proxy-Kommunikationsprofile parametrisiert.² HAN-Zertifikate vom Typ CT_SMPKI_Signed besitzen gemäß [TR-03109-4] eine Laufzeit von höchstens 2 Jahren.

Dennoch sollen die Verwender des GW_HAN_TLS_CRT im HAN Kenntnis über den geplanten Verwendungszeitraum der SMGW-Zertifikate erhalten können. Die Laufzeit richtet sich nach der erwarteten Sicherheit der verwendeten kryptografischen Verfahren unter Berücksichtigung einer möglichen kryptografischen Migration und entspricht der Laufzeit von Zertifikaten am LMN gemäß [TR-03109-3].



ICS.ZertifikateHAN.Allgemein.30

Der GWH **MUSS** im ICS deklarieren, welche Laufzeit (aufgerundet in Monaten) die von der Implementierung erstellten GW_HAN_TLS_CRT-Zertifikate besitzen (0: Wenn die Laufzeit nicht begrenzt ist).

4.3 Zertifikatsstruktur

Zertifikatsfeld	Referenz in RFC5280	Element vorhanden	Wert
TBSCertificate	4.1.1.1	m	Siehe ►Tabelle 4.4
SignatureAlgorithm	4.1.1.2	m	Siehe ►Abschnitt 4.3.1

¹ Erzeugt das SMGW das Schlüsselpaar und selbstsignierte Zertifikat im Auftrag des CLS, so enthalten Issuer und Subject die Funktionsrolle "cls"

² Geräte im HAN haben nach der Inbetriebnahme zunächst oft keine zuverlässige Kenntnis über die Zeit. Eine Prüfung der zeitlichen Gültigkeit ist hier nur eingeschränkt möglich.

Zertifikatsfeld	Referenz in RFC5280	Element vorhanden	Wert
SignatureValue	4.1.1.3	m	Abhängig vom gewählten Signatur-Algorithmus nach [TR-03109-3]: - CT_Selfsigned: Mit dem privaten Schlüssel zum "subjectPublicKeyInfo" selbstsigniert - CT_CA_Signed, CT_SMPKI_Signed: Mit dem privaten Schlüssel des Ausstellers (Zertifizierungsstelle) signiert.

Tabelle 4.3 Struktur des Elementes "Certificate"

Die folgende Tabelle gibt die Struktur des Feldes "TBSCertificate" verbindlich vor.

Zertifikatsfeld	Referenz in RFC5280	Element vorhanden	Wert
Version	4.1.2.1	m	'v3'
SerialNumber	4.1.2.2	m	Zufällig gewählte, positive Ganzzahl, bestimmt vom Aussteller (8-20 Octets).
Signature	4.1.2.3	m	Gleicher Wert wie im Feld "SignatureAlgorithm" (s. ▶Abschnitt 4.3.1).
Issuer	4.1.2.4	m	Eindeutiger Name (Distinguished Name, DN) des Zertifikatsinhabers siehe ▶Abschnitt 4.3.3: - Für CT_Selfsigned HAN-Zertifikate identisch zu "Subject". - Für CT_CA_Signed, CT_SMPKI_Signed: <i>SubjectCN</i> der ausstellenden CA
Validity	4.1.2.5	m	Zeitpunkte für Beginn und Ende der Gültigkeit des Zertifikates (s. ▶Abschnitt 4.3.4).
Subject	4.1.2.6	m	Eindeutiger Name (Distinguished Name, DN) des Zertifikatsinhabers siehe ▶Abschnitt 4.3.3.
SubjectPublicKeyInfo	4.1.2.7	m	Siehe ▶Abschnitt 4.3.2.
IssuerUniqueId	4.1.2.8	x	Entfällt
SubjectUniqueId	4.1.2.8	x	Entfällt
Extensions	4.1.2.9	m	Siehe ▶Abschnitt 4.3.5.

Tabelle 4.4 Struktur des Elementes "TBSCertificate"

4.3.1 SignatureAlgorithm

Durch die Datenstruktur SignatureAlgorithm wird nach [RFC5280] der Signaturalgorithmus des Zertifikats angegeben. Dieser besteht aus der folgenden Datenstruktur:

```
AlgorithmIdentifier ::= SEQUENCE {algorithm OBJECT IDENTIFIER, parameters ANY DEFINED BY algorithm OPTIONAL}
```

Der Wert von "algorithm" muss nach den Vorgaben zur TLS-Kommunikation im HAN nach [TR-03109-3] gewählt werden. Das Datenfeld "parameters" muss leer bleiben.

4.3.2 SubjectPublicKeyInfo

Das Feld "SubjectPublicKeyInfo" besitzt folgende Struktur (s. [RFC5480]):

```
SubjectPublicKeyInfo ::= SEQUENCE {algorithm AlgorithmIdentifier, subjectPublicKey BIT STRING}
```

Die OID im Datenfeld "algorithm" enthält den Wert 1.2.840.10045.2.1 (id-ecPublicKey). Im Feld "ECParameters" ist gemäß [RFC5480] die Variante namedCurve zu verwenden. Für die aktuell zu verwendenden Werte siehe [TR-03109-3] TLS-Kommunikation im HAN.

Der PublicKey wird mit unkomprimierten Punktkoordinaten angegeben.

4.3.3 Issuer und Subject

- Das Namensschema für den "commonName" der HAN-Teilnehmer lehnt sich an [SM-PKI-CP] Anhang A an.
- Für CT_Selfsigned HAN-Zertifikate müssen "subject" und "issuer" gemäß [RFC5280] identisch sein.
- Einzelne Attribute von Issuer und Subject dürfen nicht länger als 64 Zeichen sein.
- Vorgaben an das Namensschema von Issuer und Subject sind in ▶Abschnitt 4.4 und ▶Abschnitt 4.5 beschrieben.

4.3.4 Validity

- Das Feld "notBefore" enthält den Erzeugungszeitpunkt des Schlüsselpaares und des Zertifikates.
- Das Feld "notAfter" enthält den Zeitpunkt, nach dem das Zertifikat und das dazugehörige Schlüsselpaar nicht mehr verwendet werden darf. Der Zeitpunkt muss nach dem Erzeugungszeitpunkt liegen.

4.3.5 Extensions

Die nun folgende Tabelle enthält eine Übersicht der Extensions und den Anforderungen an deren Vorhandensein.

Bezeichnung	CT_Selfsigned	CT_CA_Signed	CT_SMPKI_Signed: CT-LS(SMGW)
AuthorityKeyIdentifier	o	m	m
SubjectKeyIdentifier	o	m	m
KeyUsage	r	r	m
PrivateKeyUsagePeriod	x	x	x
CertificatePolicies	x	o	o
SubjectAltNames	c (Bisher m)	o	x
BasicConstraints	r cA=FALSE	m cA=FALSE	m
ExtendedKeyUsage	r	r	r
CRLDistributionPoints	x	o	o
Weitere Extensions	o	x	o

Tabelle 4.5 Extensions

4.3.5.1 KeyUsage

- Extension-ID (OID): 2.5.29.15
- Kritisch: Nein für Typ CT_Selfsigned, Ja für Typ CT_CA_Signed, CT_SMPKI_Signed
- Beschreibung: Die Extension KeyUsage (spezifiziert in [RFC5280], 4.2.1.1) definiert, für welche Zwecke der zertifizierte öffentliche Schlüssel verwendet werden darf.
- Gesetzte Bits: digitalSignature(0)³.

4.3.5.2 BasicConstraints

- Extension-ID (OID): 2.5.29.19
- Kritisch: Ja für CT_CA_Signed und CT_SMPKI_Signed Zertifikate, Nein für Typ CT_Selfsigned Zertifikate
- Beschreibung: Diese Extension (spezifiziert in [RFC5280], 4.2.1.9) gibt an, ob es sich bei dem gegebenen Zertifikat um eine CA handelt und wie viele CAs ihr folgen können.

³ Für CLS-TLS-Zertifikate sind weitere Key Usages zulässig

4.3.5.3 SubjectAltName

- Extension-ID (OID): 2.5.29.27
- Kritisch: Nein
- Beschreibung: Diese Extension (spezifiziert in [RFC5280], 4.2.1.6) ermöglicht die Bindung von Identitäten an das Subject des Zertifikates. Diese Angabe ist alternativ oder zusätzlich zur Identifikation im Subject möglich.
- "dNSName": Es wird empfohlen, die eindeutige Kennung von SMGW oder HAN-Teilnehmer basierend auf [DIN43849] mit angehängtem ".sm" oder eine auf der Komponente des HAN-Teilnehmers lesbare, praktisch eindeutige, nicht-wechselnde, herstellerübergreifende Netzwerkschnittstellenadresse (MAC/EUI) mit angehängtem ".eui" als Domain Name Label zu verwenden.

4.3.5.4 ExtendedKeyUsage

- Extension-ID (OID): 2.5.29.37
- Kritisch: Nein
- Beschreibung: Die Extension ExtendedKeyUsage (spezifiziert in [RFC5280], 4.2.1.12) gibt an, ob das Zertifikat als TLS-Client und/oder als TLS-Server-Zertifikat genutzt werden kann.
- Die HAN-Teilnehmer Zertifikate sollen sowohl für TLS-Web-ServerAuthentifikation (1.3.6.1.5.5.7.3.1) als auch für TLS-Web-ClientAuthentifikation (1.3.6.1.5.5.7.3.2) verwendet werden.
- "TLS-Web-ClientAuthentication": Sofern GW_HAN_TLS_CRT Zertifikate des SMGW aus der SM-PKI stammen und die ExtendedKeyUsage "TLS-Web-ClientAuthentication" enthalten, dürfen diese vom Kommunikationsadapter auch zur Authentifizierung des SMGW als TLS-Server verwendet werden, sofern die im TLS Handshake für die TLS-Session abgeleiteten PSK- und Ephemeralen-Schlüssel für Client und Server Rolle des Kommunikationsadapters nur für ihre jeweilige Verbindung verwendet werden.

4.4 Anforderungen an die HAN-TLS-Zertifikate des SMGW

4.4.1 Einleitung

Zusätzlich zu den Anforderungen an CT_Selfsigned HAN-Zertifikate nach ▶Abschnitt 4.3 gelten für GW_HAN_TLS_CRT-Zertifikate die Anforderungen in ▶Abschnitt 4.4.

Die Anforderungen an die Zertifikatsstruktur der CT_SMPKI_Signed HAN-Zertifikate des SMGW sind in [TR-03109-4] C_{TLS}(SMGW) beschrieben.

SMGW-HAN-Zertifikate vom Typ B werden nicht verwendet.

4.4.2 Namensschema der SMGW-HAN-Zertifikate CT_Selfsigned

Attribut Typ	Kürzel	Vorgabe	Wert	Erläuterung
common name	CN	r	"<id>.smgw"	<id> Herstellerübergreifende Identifikationsnummer nach [DIN43849] mit der Sparte "E".
organisation	O	o	"<O>"	Falls verwendet darf das Feld "SM-*PKI" nicht enthalten.
organisational unit	OU	o	"<GWA-ID>"	Name des für das SMGW zuständigen GWA (aus "common name" des GWA_WAN_SIG_CRT).
country	C	r	"<LC>"	Zwei Zeichen Ländercode gemäß [ISO 3116 ALPHA-2]
serial number	SERIAL NUMBER	r	"<SN>"	Sequenznummer der Zertifikats im Bereich von 1 bis 2 ³¹ -1 und startet bei 1. Diese wird bei jedem neuen Zertifikat um den Wert 1 hochgezählt.

Tabelle 4.6 Namensschema der SMGW-HAN-Zertifikate

4.4.3 Namensschema der SMGW-HAN-Zertifikate Typ C

Die Anforderungen an das Namensschema sind in [SM-PKI-CP] Anhang A beschrieben.

4.5 Anforderungen an die TLS-Zertifikate der HAN-Teilnehmer

4.5.1 Einleitung

Zusätzlich zu den Anforderungen an CT_Selfsigned bzw. CT_CA_Signed SRV_HAN_TLS_CERT-Zertifikate nach ▶Abschnitt 4.3 gelten für CON_HAN_TLS_CERT, SRV_HAN_TLS_CERT und CLS_HAN_TLS_CERT-Zertifikate die Anforderungen in ▶Abschnitt 4.5.

4.5.2 Namensschema der HAN-Teilnehmer-Zertifikate

HAN-Teilnehmerzertifikate (außer GW_HAN_TLS_CERT) enthalten folgende Distinguished Name Attribute:

Attribut Typ	Kürzel	Vorgabe	Wert	Erläuterung
common name	CN	r	"<id>.<func>"	• Sofern <func> gleich "cls" ist, enthält <id> die auf der HAN-Komponente ablesbare, kanonisierte Geräteidentifikation gemäß [DIN43849]. • Die <id> und das <idschema> für Service-Techniker und Letztverbraucher-Zertifikate wird gemäß organisatorischer Anforderungen des Ausstellers vergeben und vom GWA im SMGW konfiguriert. • Aus Gründen der Lesbarkeit des CommonName wird eine Schreibweise einheitlich mit Groß- oder Kleinbuchstaben empfohlen. Eine semantische Interpretation des CN findet nicht statt.
country	C	o	"<LC>"	Zwei Zeichen Ländercode gemäß [ISO 3116 ALPHA-2]
serial number	SERIAL NUMBER	o	"<SN>"	Falls vorhanden, enthält das Feld die Sequenznummer des Zertifikats im Bereich von 1 bis $2^{31}-1$. und startet bei 1. Diese wird bei jedem neuen Zertifikat um den Wert 1 hochgezählt.

Tabelle 4.7 Namensschema der HAN-Teilnehmer-Zertifikate

Der Distinguished Name darf weitere Attribute enthalten (nicht empfohlen). Eine Prüfung durch das SMGW erfolgt aktuell nicht.

4.6 Anforderungen an das CA-Zertifikat zur Validierung der Zertifikate der Servicetechniker

4.6.1 Einleitung

Zusätzlich zu den Anforderungen an CT_Selfsigned HAN-Zertifikate nach ▶Abschnitt 4.3 gelten für GWA-CA_SIG_CERT- und GWHCA_SIG_CERT-Zertifikate zur Ausstellung von CT_CA_Signed Zertifikaten (signiert durch CA), die Anforderungen in ▶Abschnitt 4.6.2

4.6.2 Namensschema des CA-Zertifikates des GWA/GWH zur Validierung der Zertifikate der Servicetechniker

Es wird empfohlen, die Attribute des Distinguished Name (Subject/Issuer) in Anlehnung an [SM-PKI-CP] Anhang A zu wählen.

5 CMS - Cryptographic Message Syntax

5.1 Einleitung

Dieses Kapitel beschreibt Anforderungen an die Inhaltsdatenverschlüsselung, -signatur und -kompression unter Verwendung von CMS (Cryptographic Message Syntax). Es enthält Anforderungen an die Kommunikation zwischen einem Sender (Originator) und dem zugehörigen Empfänger (Recipient) und konkretisiert die Anforderungen allgemeiner CMS-Spezifikationen zur Verwendung in intelligenten Messsystemen.

Die Beschreibung der Anforderungen an die Konfiguration der Parameter, Zertifikate und die Zertifikatsvalidierung für CMS sind nicht Bestandteil dieses Kapitels.

5.2 Anforderungen

5.2.1 Zertifikate

Die Implementierung **MUSS** die in empfangenen CMS-Nachrichten referenzierten Zertifikate gemäß [RFC5280] Kapitel 6 validieren, indem sie auf Übereinstimmung mit einem für den Verwendungszweck geeigneten administrativ konfigurierten SM-PKI-Endnutzer-Zertifikat geprüft werden. [REQ.CMS.Zertifikate.10]

Die Implementierung **MUSS** die in CMS-Nachrichten empfangenen Zertifikate gemäß [RFC5280] Kapitel 4.1 verarbeiten können. [REQ.CMS.Zertifikate.20]

5.2.2 Algorithmenauswahl für CMS

Dieser Abschnitt benennt die zur Mindestinteroperabilität verwendeten Kombinationenmöglichkeiten ("CipherSuites") von CMS-Kompressions-, Verschlüsselungs- und Signaturverfahren. Die Verwendung wird in WAN-Kommunikationsprofilen festgelegt.

Die Implementierung **MUSS** die Kombination der kryptografischen Verfahren für die Verschlüsselung von Nachrichten gemäß ▶Tabelle 5.1 verarbeiten und erzeugen können. [REQ.CMS.MessageCipherSuite.10]

Die Implementierung **MUSS** die Kombination der kryptografischen Verfahren für die Signatur von Nachrichten gemäß ▶Tabelle 5.2 verarbeiten und erzeugen können. [REQ.CMS.MessageCipherSuite.20]

Suite	Key Agreement Algorithm, KDF und KDF-Hash	Key Encryption Algorithm	Authentication und Content Encryption Algorithm	eContent
CMS_ECKAEG_A-ES_128_GCM	ecka-eg-X693K-DF-SHA256 (Definiert in [TR-03111])	id-aes128-wrap (Definiert in [RFC3565])	id-aes128-GCM (Definiert in [RFC5084])	Encapsulated (id-compressedData oder id-Data)
CMS_ECKAEG_A-ES_128_CBCCMAC	ecka-eg-X693K-DF-SHA256 (Definiert in [TR-03111])	id-aes128-wrap (Definiert in [RFC3565])	id-aes-CBC-CMAC-128 (Definiert in ▶Abschnitt 5.4)	Encapsulated (id-CompressedData oder id-Data)

Tabelle 5.1 CipherSuites zur Verschlüsselung von Inhaltsdaten

Suite	Signature Algorithm	Digest Algorithm	eContent
CMS_ECDSA_SHA256	ecdsa-with-Sha256 (Definiert in [TR-03111])	id-sha-256 (Definiert in [RFC5754])	Encapsulated (id-authEnvelopedData)
CMS_ECDSA_SHA256_Detached	ecdsa-with-Sha256 (Definiert in [TR-03111])	id-sha-256 (Definiert in [RFC5754])	Detached (id-data)

Tabelle 5.2 CipherSuites zur Signatur von Inhaltsdaten

Suite	Compression Algorithm	eContent
CMS_ZLIB	id-alg-zlibCompression (Definiert in [RFC3274])	Encapsulated (id-Data)

Tabelle 5.3 CipherSuites zur Kompression von Inhaltsdaten

5.2.3 EC-Domain-Parameter

Die NIST- und Brainpool-Kurven über Primkörpern mit folgenden OIDs werden von diesem Dokument unterstützt:

```
secp256r1 OBJECT IDENTIFIER ::= { iso(1) member-body(2) us(840) ansi-X9-62(10045) curves(3) prime(1) 7 }
```

```
secp384r1 OBJECT IDENTIFIER ::= { iso(1) identified-organization(3) certicom(132) curve(0) 34 }
```

bzw.

```
brainpoolP256r1 OBJECT IDENTIFIER ::= { ellipticCurve versionOne(1) 7 }
```

```
brainpoolP384r1 OBJECT IDENTIFIER ::= { ellipticCurve versionOne(1) 11 }
```

```
brainpoolP512r1 OBJECT IDENTIFIER ::= { ellipticCurve versionOne(1) 13 }
```

wobei

```
ellipticCurve OBJECT IDENTIFIER ::= { iso(1) identified-organization(3) teletrust(36) algorithm(3) signature-algorithm(3) ecSign(2) ecStdCurvesAndGeneration(8) 1 }
```

Verbindliche Vorgaben über die jeweils aktuell zu verwendenden EC-Domain-Parameter sind in [TR-03109-3] zu finden.

5.3 Datenstrukturen

5.3.1 CMS-Datenformat der Nachrichten

Die Datenstrukturen nach ▶Abschnitt 5.3.2, ▶Abschnitt 5.3.3 und ▶Abschnitt 5.3.4 **MÜSSEN** für den Transport in die Datenstruktur "ContentInfo" nach [RFC5652] Kapitel 5.2 eingebettet werden. [REQ.CMS.ContentInfo.10] Als "ContentType" in der Datenstruktur "ContentInfo" **MUSS** für eine innere Verschlüsselung und äußere Signatur die OID id-SignedData verwendet werden. [REQ.CMS.ContentInfo.20]

Die folgenden Abschnitte enthalten Tabellen, in denen Anforderungen an das Vorhandensein von Attributen und Datenfeldern verschiedener Zertifikatsprofile beschrieben sind. Darin werden die folgenden Abkürzungen verwendet:

Zeichen	Langform	Bedeutung
m	Mandatory	Das Element muss vorhanden sein.
x	Not existent	Das Element darf nicht vorhanden sein.
o	Optional	Das Element kann vorhanden sein
c	Conditional	Das Element ist abhängig von einem anderen Element vorhanden
r	Recommended	Das Vorhandensein des Elements wird empfohlen.

Tabelle 5.4 Beschreibung der Abkürzungen zum Vorhandensein von CMS-Nachrichten-Elementen

5.3.2 CMS-Datenformat der Inhaltsdatenverschlüsselung

Die Datenstruktur "AuthEnvelopedData" **MUSS** gemäß [RFC5083] in authentisch verschlüsselten CMS-Nachrichten verwendet werden. [REQ.CMS.AuthEnvelopedData.10]

5.3.2.1 Authenticated-Enveloped-Data-Content-Type mit ECKA-EG

Der Content-Type für die Datenstruktur AuthEnvelopedData hat gemäß [RFC5083] die folgende OID:

```
id-ct-authEnvelopedData OBJECT IDENTIFIER ::= { iso(1) member-body(2) us(840) rsadsi(113549) pkcs(1) pkcs-9(9) smime(16) ct(1) 23 }
```

Es **MUSS** das folgende Profil der AuthEnvelopedData-Datenstruktur verwendet werden. Die ersten drei Spalten beziehen sich dabei auf die Definition der Datenstruktur gemäß [RFC5083]. Die rechte Spalte gibt verbindlich vor, wie das entsprechende Feld zu füllen ist. [REQ.CMS.AuthEnvelopedData.20]

Feld	Vorhanden	Tag	Typ	Wert
AuthEnvelopedData	m		SEQUENCE	
version	m		CMSVersion	'0'
originatorInfo	x		SEQUENCE	Entfällt, da der PublicKey des Senders ephemeral ist.
certs	x	[0] IMPLICIT	CertificateSet	Entfällt
crls	x	[1] IMPLICIT	RevocationInfoChoices	Entfällt
RecipientInfos	m		SET SIZE (1..MAX) OF	Enthält Infos über den Empfänger
	m	[1] IMPLICIT	RecipientInfo (CHOICE: ktri KeyTransRecipientInfo, kari [1] IMPLICIT KeyAgreeRecipientInfo, kekri [2] IMPLICIT KEKRecipientInfo, pwri [3] IMPLICIT PasswordRecipientInfo, ori [4] IMPLICIT OtherRecipientInfo)	Es MUSS die Variante kari (Typ: KeyAgreementRecipientInfo SEQUENCE) ausgewählt werden.
version	m		CMSVersion	'3'
originator	m	[0] EXPLICIT	OriginatorKeyOrIdentifier	
		[1] EXPLICIT	(CHOICE: issuerAndSerNo IssuerAndSerNo, subjectKeyId [0] SubjectKeyId, originatorKey [1] OriginatorPublicKey)	Es MUSS die Variante originatorKey (Typ: OriginatorPublicKey) verwendet werden.
algorithm	m		AlgorithmIdentifier	Daten des ephemeren Key Agreement Public Key des Senders
algorithm	m		OBJECT IDENTIFIER	id-ecPublicKey
parameters	c		ANY DEFINED BY algorithm	Falls vorhanden: namedCurve (OID) gemäß ▶Abschnitt 5.2.3. Die aktuell zu verwenden Werte sind in [TR-03109-3] zu finden. Sofern namedCurve vorhanden ist, MUSS implizit die namedCurve-OID des derid zugeordneten PublicKeys verwendet werden.
publicKey	m		BIT STRING	Wert des ephemeren Public Keys des Absenders
ukm	x	[1] EXPLICIT	UserKeyingMaterial (OCTET STRING)	Entfällt
keyEncryptionAlgorithm	m		KeyEncryptionAlgorithmIdentifier (AlgorithmIdentifier)	Informationen zum Schlüsselableitungsalgorithmus für die Verschlüsselung des Content-Encryption-Keys

Feld		Vorhanden	Tag	Typ	Wert
	algorithm	m		OBJECT IDENTIFIER	Algorithmus aus ▶Abschnitt 5.3.2.2.1. Die dabei aktuell zu verwendenden Werte sind in [TR-03109-3] zu finden.
	parameters	m		ANY DEFINED BY algorithm	Enthält den symmetrischen Key Encryption-Algorithmus für die Verschlüsselung des Content-Encryption-Keys.
		m		AlgorithmIdentifier (SEQUENCE)	Algorithmus aus ▶Abschnitt 5.3.2.2.2. Die dabei aktuell zu verwendenden Werte sind in [TR-03109-3] zu finden.
	recipientEncryptedKeys	m		RecipientEncryptedKeys (SEQUENCE OF RecipientEncryptedKey (SEQUENCE))	Enthält die Infos über den Content-Encryption-Key
	rid	m	[0] IMPLICIT	KeyAgreeRecipientIdentifier (CHOICE: issuerAndSerNo IssuerAndSerNo, rKeyID [0] IMPLICIT RecipientKeyIdentifier)(SEQUENCE)	Es wird die Variante rKeyID (Typ: RecipientKeyIdentifier) gewählt. Die optionalen Felder date und other in der SEQUENCE RecipientKeyIdentifier entfallen.
	subjectKeyIdentifier	m		SubjectKeyIdentifier (OCTET STRING)	Enthält den SKI des Zertifikates des Empfängers für den verschlüsselt wurde.
	date	x		GeneralizedTime	Entfällt
	other	o		OtherKeyAttribute	Kann Hinweise für den Empfänger enthalten, den passenden PublicKey auszuwählen
	encryptedKey	m		EncryptedKey (OCTET STRING)	Struktur abhängig vom gewählten contentEncryptionAlgorithm (Siehe ▶Abschnitt 5.3.2.2.3 und ▶Abschnitt 5.3.2.2.4). Die dabei aktuell zu verwendenden Werte sind in [TR-03109-3] zu finden.
	authEncryptedContentInfo	m		EncryptedContentInfo (SEQUENCE)	Enthält den verschlüsselten und authentisierten Inhalt der Nachricht
	contentType	m		ContentType (OBJECT IDENTIFIER)	Datentyp des Inhalts gemäß Kommunikationsszenario (z.B. id-Data, id-compressed-Data)

Feld	Vorhanden	Tag	Typ	Wert
contentEncryptionAlgorithm	m		ContentEncryptionAlgorithmIdentifier (AlgorithmIdentifier)	Algorithmus aus ▶Abschnitt 5.3.2.2.3 und ▶Abschnitt 5.3.2.2.4. Die dabei aktuell zu verwendenden Werte sind in [TR-03109-3] zu finden.
encryptedContent	o	[0] IMPLICIT	EncryptedContent (OCTET STRING)	Enthält den verschlüsselten Inhalt der Nachricht.
authAttrs	c	[1] IMPLICIT	AuthAttributes	Abhängig vom contentType der verschlüsselten Daten, vgl. [RFC5083]. MUSS vorhanden sein, falls contentType in EncryptedContentInfo nicht vom Typ id-data ist. In diesem Fall MUSS mindestens das contentType-Attribut enthalten sein
mac	m		MessageAuthenticationCode (OCTET STRING)	MAC-Wert
unauthAttrs	x	[2] IMPLICIT	UnauthAttributes	Entfällt

Tabelle 5.5 AuthEnvelopedData

5.3.2.2 Unterstützte Algorithmen

5.3.2.2.1 OIDs für den ECKA-EG-Algorithmus

Es **SOLLEN** folgende OIDs aus [TR-03111] unterstützt werden: [REQ.CMS.AuthEnvelopedData.25]

ecka-eg-X963KDF-SHA256 OBJECT IDENTIFIER ::= {ecka-eg ecka-eg-x963KDF(1) 3}

ecka-eg-X963KDF-SHA384 OBJECT IDENTIFIER ::= {ecka-eg ecka-eg-x963KDF(1) 4}

ecka-eg-X963KDF-SHA512 OBJECT IDENTIFIER ::= {ecka-eg ecka-eg-x963KDF(1) 5},

wobei

ecka-eg OBJECT IDENTIFIER ::= {id-ecc key-establishment(5) 1}

id-ecc OBJECT IDENTIFIER ::= { bsi-de algorithms(1) 1}

bsi-de OBJECT IDENTIFIER ::= { itu-t(0) identified-organization(4) etsi(0) reserved (127) etsi-identified-organization(0) 7 }



ICS.CMS.AuthEnvelopedData.10

Der Hersteller **MUSS** deklarieren, welche OIDs die Implementierung für die AuthEnvelopedData Schlüsselvereinbarung anbietet bzw. akzeptiert (▶REQ.CMS.AuthEnvelopedData.25)

5.3.2.2.2 Key-Encryption Algorithm

Für die Key Encryption **SOLLEN** die Algorithmen mit den OIDs id-aes128-wrap, id-aes192-wrap, id-aes256-wrap aus [RFC3565] unterstützt werden. Das "Parameter"-Feld bleibt bei diesen Algorithmen leer. Die Algorithmen sind in [RFC3394] Kapitel 2.2 beschrieben. [REQ.CMS.AuthEnvelopedData.30]

**ICS.CMS.AuthEnvelopedData.20**

Der Hersteller **MUSS** deklarieren, welche OIDs die Implementierung für AuthEnvelopedData Key-Wrap-Algorithmen anbietet bzw. akzeptiert.

5.3.2.2.3 Content-Authenticated-Encryption mit AES im CBC-CMAC-Mode

Außerdem **SOLL** die Verschlüsselung und MAC-Sicherung im CBC-CMAC-Mode via der Algorithmen id-aes-CBC-CMAC-128, id-aes-CBC-CMAC-192, id-aes-CBC-CMAC-256 unterstützt werden. Die OIDs werden in ▶Abschnitt 5.4 definiert. [REQ.CMS.AuthEnvelopedData.40]

Das "parameters"-Feld **MUSS** hier weggelassen werden, d.h. es gilt IV=0 und der MAC besteht aus 16 Oktetts. (vgl. ▶Abschnitt 5.4). [REQ.CMS.AuthEnvelopedData.50]

Bei Verwendung einer dieser OIDs ist im Feld "encryptedKey" die mit dem gewählten Key-Encryption-Algorithmus verschlüsselte Konkatenation $K_{enc} || K_{MAC}$ enthalten (AESxxx-Wrap($K_{enc} || K_{MAC}$) nach [RFC3394] Kapitel 2.2). Hierbei ist K_{enc} der (zufällig erzeugte) Schlüssel für die Verschlüsselung des Inhalts mit AES im CBC-Mode und K_{MAC} der (zufällig erzeugte) Schlüssel für die MAC-Sicherung mit AES im CMAC-Mode. Die Schlüssel K_{enc} und K_{MAC} für die Verschlüsselung und MAC-Sicherung der Daten müssen stets unmittelbar vor seiner Verwendung zufällig erzeugt werden und dürfen jeweils nur für die Versendung einer Nachricht verwendet werden.

5.3.2.2.4 Content-Authenticated-Encryption mit AES im GCM-Mode

Für den GCM-Mode **SOLLEN** die Algorithmen mit den OIDs id-aes128-gcm, id-aes192-gcm, id-aes256-gcm aus [RFC5084] unterstützt werden. [REQ.CMS.AuthEnvelopedData.60]

Das Parameter-Feld dieser Algorithmen ist ebenfalls in [RFC5084] zu finden.

Das "parameters"-Feld **MUSS** eine "aes-ICVlen" von 16 Oktetts enthalten. [REQ.CMS.AuthEnvelopedData.70]

Das "parameters"-Feld **MUSS** eine "nonce" mit einer Länge von 12 Oktetts enthalten. [REQ.CMS.AuthEnvelopedData.80]

Bei Verwendung einer dieser OIDs ist im Feld "encryptedKey" der mit dem gewählten Key-Encryption-Algorithmus verschlüsselte (zufällig erzeugte) Content-Encryption-Key K enthalten (AESxxx-Wrap(K)). Der Content-Encryption-Key K für die Verschlüsselung und MAC-Sicherung der Daten wird stets unmittelbar vor seiner Verwendung zufällig erzeugt und **MUSS** jeweils nur für die Versendung einer Nachricht verwendet werden. [REQ.CMS.AuthEnvelopedData.90]

**ICS.CMS.AuthEnvelopedData.30**

Der Hersteller **MUSS** deklarieren, welche OIDs die Implementierung für die authentische Inhaltsdatenverschlüsselung anbietet bzw. akzeptiert.

5.3.3 CMS-Datenformat der Inhaltsdatensignatur

Die Implementierung **MUSS** die Datenstruktur SignedData gemäß [RFC5652] in signierten CMS-Nachrichten verwenden. [REQ.CMS.SignedData.10]

5.3.3.1 Signed-Data Content-Type mit ECDSA

Als ContentType für die Datenstruktur SignedData **MUSS** die folgende OID nach [RFC5652] verwendet werden: [REQ.CMS.SignedData.20]

id-signedData OBJECT IDENTIFIER ::= { iso(1) member-body(2) us(840) rsadsi(113549) pkcs(1) pkcs-7(7) 2 }

Das Feld "content" der Datenstruktur ContentInfo **MUSS** dann vom Typ "SignedData" sein, vgl. [RFC5652]. [REQ.CMS.SignedData.30] Hierbei **MUSS** das im Folgenden definierte Profil der Datenstruktur verwendet werden. [REQ.CMS.SignedData.40] Die ersten drei Spalten beziehen sich dabei auf die Definition der Datenstruktur gemäß [RFC5652]. Die rechte Spalte gibt verbindlich vor, wie das entsprechende Feld zu füllen ist.

Feld	Vorhanden	Tag	Typ	Wert
SignedData	m			
version	m		CMSVersion	'3'
digestAlgorithms	m		DigestAlgorithmIdentifiers (SET OF DigestAlgorithmIdentifier (AlgorithmIdentifier))	Siehe ▶Abschnitt 5.3.3.2.1
encapContentInfo	m		EncapsulatedContentInfo (SEQUENCE)	Enthält die zu signierende Nachricht.
eContentType	m		ContentType (OBJECT IDENTIFIER)	Datentyp der Inhaltsdaten gemäß Kommunikationsszenario (z.B. id-ct-authEnvelopedData, siehe ▶Abschnitt 5.3.2 oder id-data)
eContent	m	[0] EXPLICIT	OCTET STRING	Datenstruktur aus ▶Abschnitt 5.3.2
certificates	o	[0] IMPLICIT (SEQUENCE Certificate)	CertificateSet (SET OF CertificateChoices CHOICE: certificate Certificate, extendedCertificate [0] IMPLICIT ExtendedCertificate, v1AttrCert [1] IMPLICIT AttributeCertificateV1, v2AttrCert [2] IMPLICIT AttributeCertificateV2, other [3] IMPLICIT OtherCertificateFormat)	Das Feld ist optional. Falls es vorhanden ist enthält es das X.509-Signaturzertifikat, mit dem die Nachricht signiert ist.
crls	x		RevocationInfoChoices (SET OF RevocationInfoChoices CHOICE: crl CertificateList, other [1] IMPLICIT OtherRevocationInfoFormat)	Entfällt
signerInfos	m		SignerInfos (SET OF SignerInfo (SEQUENCE))	Besteht aus einer SignerInfo und enthält die Informationen über den Ersteller der Signatur.
version	m		CMSVersion	'3'
sid	m	[0]	SignerIdentifier (CHOICE: issuerAndSerNo IssuerAndSerNo, subjectKeyId [0] SubjectKeyId)	Es wird die Variante subjectKeyId gewählt
digestAlgorithm	m		DigestAlgorithmIdentifier (AlgorithmIdentifier)	Enthält Infos über den Hash-Algorithmus. Es müssen die Algorithmen aus ▶Abschnitt 5.3.3.2.1 unterstützt werden. Die dabei aktuell zu verwendenden Werte sind in [TR-03109-3] zu finden.
signedAttrs	m	[0] IMPLICIT	SignedAttributes (SET SIZE (1..MAX) OF Attribute (SEQUENCE))	Enthält das Content-Type Attribut
attrType	m		OBJECT IDENTIFIER	id-contentType und id-messageDigest

Feld	Vorhanden	Tag	Typ	Wert
attrValues	m		SET OF AttributeValue (ANY)	Enthält jeweils den zugehörigen (genau einen) Attribut-Wert
contentType	m		OBJECT IDENTIFIER	Datentyp der Inhaltsdaten gemäß Kommunikationsszenario (z.B. id-ct-authEnvelopedData)
SignatureAlgorithm	m		SignatureAlgorithmIdentifier (AlgorithmIdentifier)	Enthält den Algorithmus, mit dem die Nachricht signiert wurde. Algorithmus aus ▶ Abschnitt 5.3.3.2.2 unterstützt werden. Die dabei aktuell zu verwendenden Werte sind in [TR-03109-3] zu finden
signature	m		SignatureValue (OCTET STRING)	Enthält das Ergebnis der Signaturerzeugung gemäß [RFC5083].
unsignedAttr	x	[1] IMPLICIT	UnsignedAttributes (SET SIZE (1..MAX) OF Attribute (SEQUENCE))	Entfällt

Tabelle 5.6 SignedData

5.3.3.2 Unterstützte Algorithmen

5.3.3.2.1 OIDs für Hashfunktionen

Es **SOLLEN** die OIDs id-sha-256, id-sha-384, id-sha-512 für die Berechnung des Message Digest unterstützt werden. [REQ.CMS.SignedData.50] Die OIDs sind in [RFC5754] spezifiziert. Das optionale "Parameter"-Feld entfällt.

5.3.3.2.2 Signaturalgorithmen

Die Signaturalgorithmen mit den folgenden OIDs aus [TR-03111] für die Signaturerzeugung **SOLLEN** unterstützt werden: [REQ.CMS.SignedData.60]

```
ecdsa-with-specified OBJECT IDENTIFIER ::= {id-ecSigType 3},
ecdsa-with-Sha256 OBJECT IDENTIFIER ::= {ecdsa-with-specified} 2},
ecdsa-with-Sha384 OBJECT IDENTIFIER ::= {ecdsa-with-specified 3},
ecdsa-with-Sha512 OBJECT IDENTIFIER ::= {ecdsa-with-specified 4},
```

wobei

```
id-ecSigType OBJECT IDENTIFIER ::= {iso(1) member-body(2) us(840) ansi-X9-62(10045) signatures(4)}
```

Als Signatur-Algorithmus ist hierbei jeweils die OID zu verwenden, deren Suffix mit der Hash-Funktion übereinstimmt, welche im DigestAlgorithm-Feld der entsprechenden SignerInfos enthalten ist.

Das Parameter-Feld entfällt.

Die Codierung der Signatur **MUSS** im X9.62 Format gemäß [TR-03111] Kap. 5.2.2 erfolgen. [REQ.CMS.SignedData.70]



ICS.CMS.SignedData.10

Der Hersteller **MUSS** deklarieren, welche OIDs die Implementierung für die Signaturalgorithmen bzw. Message-Digest Hash-Algorithmen anbietet bzw. akzeptiert.

5.3.4 CMS-Datenformat komprimierter Daten

Die Implementierung **MUSS** die Datenstrukturen CompressedData nach [RFC3274] und ZLIB mit deflate-Codierung nach [RFC1951] und [RFC1950] verarbeiten und erzeugen können. [REQ.CMS.CompressedData.10]

5.4 AES-CBC-CMAC Algorithm Identifier und Parameter

Dieser Abschnitt enthält die allgemeine Definition der Algorithm-Identifier für AES im CBC-CMAC-Mode sowie ihre Verwendung bei der Content-Authenticated-Encryption mit dem CMS Content Type Authenticated-Enveloped-Data.

Die konkrete Verwendung der OIDs im Falle der Inhaltsdatenverschlüsselung wird in ▶Abschnitt 5.3.2.2.3 festgelegt.

Die folgenden OIDs definieren die entsprechenden AES-Algorithmen im CBC-CMAC-Mode.

```
id-aes-cbc-cmac-128 OBJECT IDENTIFIER ::= { bsi-de algorithm(1) symmetricCiphers(3) AES(1) authEncr
(1) 2 }
```

```
id-aes-cbc-cmac-192 OBJECT IDENTIFIER ::= { bsi-de algorithm(1) symmetricCiphers(3) AES(1) authEncr
(1) 3 }
```

```
id-aes-cbc-cmac-256 OBJECT IDENTIFIER ::= { bsi-de algorithm(1) symmetricCiphers(3) AES(1) authEncr
(1) 4 }
```

Für diese Algorithmen bezeichne K_{enc} den (zufällig erzeugten) Schlüssel für die Verschlüsselung des Inhalts mit AES im CBC-Mode und K_{MAC} den (zufällig erzeugten) Schlüssel für die MAC-Sicherung mit AES im CMAC-Mode (jeweils in der entsprechenden Bitlänge).

Das Feld "encryptedContent" **MUSS** die mit AES im CBC-Mode und dem Schlüssel K_{enc} verschlüsselte Nachricht enthalten. [REQ.CMS.AesCbcCmac.10]

Die Berechnung des Feldes "mac" **MUSS** gemäß den Vorgaben von [RFC5083] mit dem "encryptedContent" sowie den DER-codierten "authAttrs"(als AAD) als Input via AES im CMAC-Mode mit den Schlüssel K_{MAC} erfolgen. [REQ.CMS.AesCbcCmac.20]

Der CMAC **MUSS** nach [RFC4493] über die Konkatenation aus $C \parallel \text{AAD}$ gebildet werden, wobei C der AES-CBC verschlüsselte Inhalt und AAD der Inhalt der "authAttrs" ist. [REQ.CMS.AesCbcCmac.30]

Als Parameter der Funktion AES-CBC **MUSS** der Input dabei jeweils wie in [RFC5652] beschrieben gepaddet werden. [REQ.CMS.AesCbcCmac.40]

In allen Fällen ist das "parameters"-Feld optional

Ist das "parameters"-Feld nicht vorhanden, **MÜSSEN** für den CBC-Mode $\text{IV}=0$ und als MAC-Länge 16 Oktetts verwendet werden. [REQ.CMS.AesCbcCmac.50] Die Content-Encryption-Keys K_{enc} und K_{MAC} werden unmittelbar vor der Verwendung zufällig erzeugt und **DÜRFEN NUR** nur für die Verschlüsselung und MAC-Sicherung einer Nachricht verwendet werden. [REQ.CMS.AesCbcCmac.60]

Ist das "parameters"-Feld vorhanden, so **MUSS** es folgende Struktur besitzen: [REQ.CMS.AesCbcCmac.70]

```
CBC-CMAC-Param ::= SEQUENCE { aes-iv OCTET STRING (SIZE(16)) aes-MacLen INTEGER (12 | 13 | 14 | 15
| 16) DEFAULT 16 }
```

Das Feld "aes-iv" enthält den Initialisierungsvektor für die CBC-Verschlüsselung und **MUSS** aus 16 Oktetts bestehen. [REQ.CMS.AesCbcCmac.80] Das Feld "aes-MacLen" gibt die Länge des Outputs des CMACs an. Es **MUSS** eine MAC-Länge von 16 Oktetts verwendet werden. [REQ.CMS.AesCbcCmac.90] Innerhalb der Verwendungszeit der zufälligen Content-Encryption-Keys K_{enc} und K_{MAC} muss ein IV für die Verschlüsselung einer Nachricht unvorhersagbar sein. Insbesondere **DARF** ein IV bei gleichen Schlüsseln praktisch **NICHT** zweimal verwendet werden. [REQ.CMS.AesCbcCmac.100]

Um im Falle des Vorhandenseins des "Parameter"-Feldes die Authentizität des Initialisierungsvektors für den CBC-Mode sicherzustellen, **MÜSSEN** die Parameter als authentifizierte Attribute im Feld "authAttr" in die MAC-Berechnung eingehen. [REQ.CMS.AesCbcCmac.110] Hierbei **MUSS** als Content-Type die folgende OID verwendet werden: [REQ.CMS.AesCbcCmac.120]

```
id-aes-CBC-CMAC-Param OBJECT IDENTIFIER ::= { bsi-de symmetricCiphers(3) AES(1) authEncr (1) 1 }
```

5.5 Nutzung des Sicherheitsmoduls bei der CMS Inhaltsdatensicherung

Das SMGW nutzt zur Verarbeitung der CMS-Nachrichten ein zertifiziertes Sicherheitsmodul gemäß [TR-03109-2]:

- zur Erzeugung der Zufallszahlen für symmetrische Verschlüsselung und Signaturerzeugung,
- zur Schlüsselaushandlung gemäß Elliptic Curve Diffie-Hellman,
- zur Signaturerzeugung und -prüfung gemäß ECDSA,
- zur Erzeugung der asymmetrischen Schlüsselpaare und Speicherung der statischen Schlüsselpaare.

Das SMGW führt die Verschlüsselung, Entschlüsselung, Schlüsselableitung, Hash-Erzeugung mit symmetrischen Verfahren ohne die Dienste des Sicherheitsmodules durch. Dies dient einer höheren Performance und der Verbesserung kryptografischer Agilität.

►Abbildung 5.1 zeigt exemplarisch für AES im CBC-CMAC-Mode die Interaktion zwischen SMGW und Sicherheitsmodul für die Inhaltsdatenverschlüsselung, Integritätssicherung und Signierung auf. Dabei wird angenommen, dass der öffentliche Schlüssel des Empfängers mit dem dazugehörigen privaten Schlüssel im Besitz des Empfängers auf der WAN-Seite ist.

Die Reihenfolge der Kommandos kann von der in den Abbildungen gezeigten Reihenfolge abweichen. Die Abbildungen sind diesbezüglich lediglich exemplarisch und informativ zu verstehen. Ausschlaggebend ist die Anforderung in [TR-03109-2], aus der sich Abhängigkeiten bezüglich der Reihenfolge der skizzierten Kommandos ergeben.

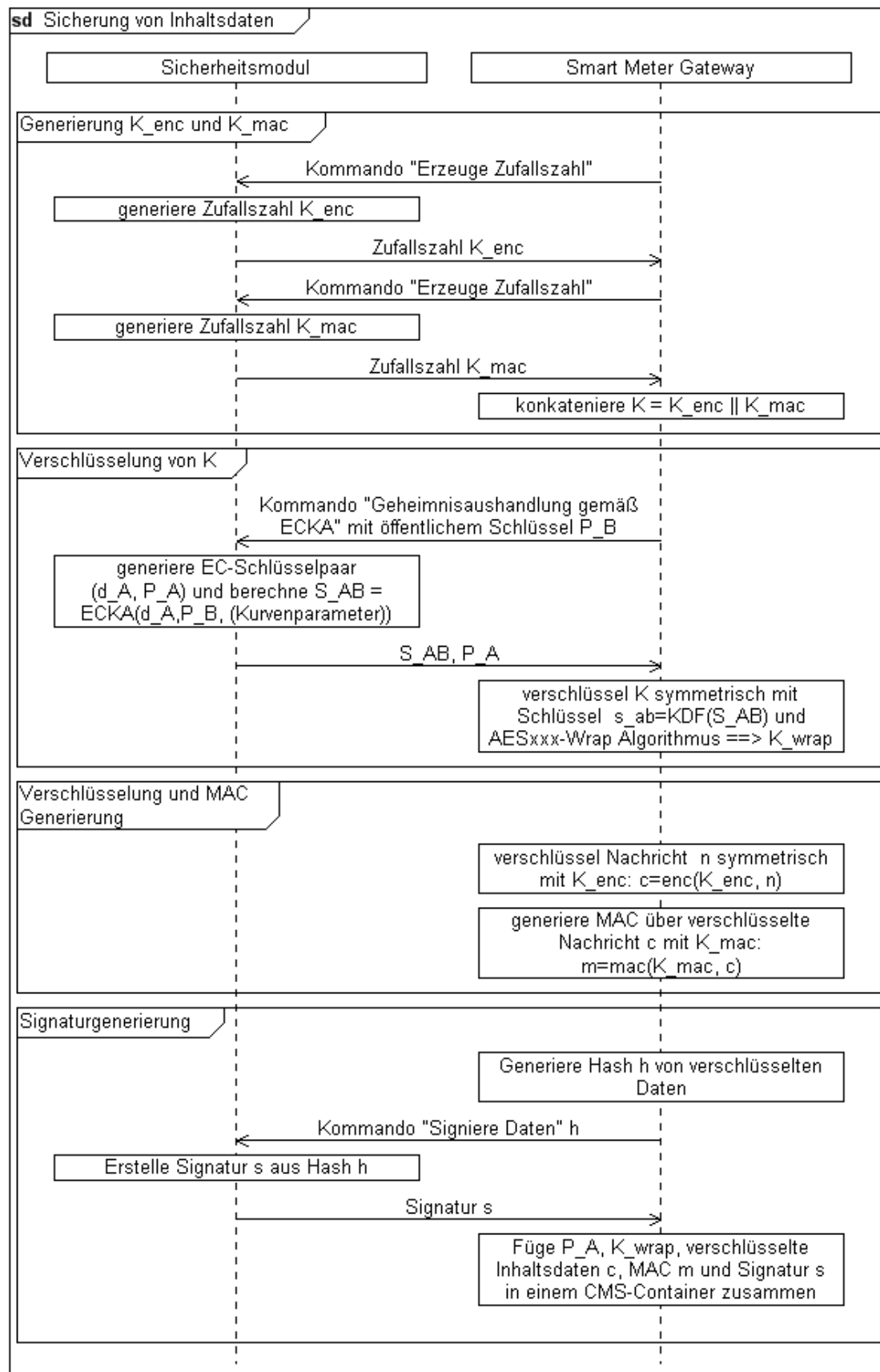


Abbildung 5.1. Interaktion zwischen SMGW und SM bei der Inhaltsdatensicherung mit AES-CBC-CMAC

6 RESTful Webservice Protokoll

6.1 Einleitung

Das Hypertext Transfer Protokoll (HTTP) ist ein OSI Layer 7 Protokoll, das zwischen Transaktionen zustandslos ist. Es ermöglicht die Zusammenarbeit von Informationssystemen über ausgetauschte Textnachrichten (HTTP-Request und HTTP-Response genannt). Das Protokoll definiert dabei das Interface, um mit den Ressourcen zu interagieren.

Die RFCs [RFC9110] und [RFC9112] bilden den Rahmen der Anforderungen an die Implementierung von HTTP/1.1. Sie beschreiben aber auch viele Protokoll-Optionen. Für den interoperablen Betrieb und da die Implementierung über begrenzte Ressourcen verfügt, sind die Optionen allerdings nicht vollumfänglich zu implementieren. Insbesondere Aspekte wie Datenvolumen und Migration zwischen Versionen begründen eine Einschränkung auf die notwendigen Protokoll-Optionen für einen leistungsfähigen und interoperablen Betrieb.

Die umgesetzten Optionen des HTTP-Servers und HTTP-Clients der Implementierung sollen den Vorgaben der [RFC9110] und [RFC9112] entsprechen

6.2 Anforderungen an HTTP-Server

Dieses Kapitel beschreibt die Anforderungen an das Verhalten der Implementierung eines HTTP-Servers



REQ.Webservice.HttpServer.10

Die Implementierung **MUSS** das Format, d.h. die Syntax, der HTTP-Nachrichten an den von außen zugänglichen Schnittstellen gemäß der Spezifikation in [RFC9112] verarbeiten und senden.



REQ.Webservice.HttpServer.20

Die Implementierung **MUSS** jede von außen zugängliche Ressource (Firmware-Dateien, Konfigurationsobjekte, Statusinformationen, NTP-Daten etc.) durch einen Uniform Resource Identifier (URI) nach [RFC3986] identifizieren.



REQ.Webservice.HttpServer.30

Die Implementierung **MUSS** die Bezeichner der HTTP-Header-Fields case-insensitiv verarbeiten.



REQ.Webservice.HttpServer.60

Die Implementierung **MUSS** Requests, die nicht der Syntax von [RFC9112] entsprechen, mit Response Status "Bad Request" zurückweisen.



REQ.Webservice.HttpServer.62

Die Implementierung **SOLL** Requests mit einem syntaktisch nicht gemäß Content-Type verarbeitbaren Content-Body mit Response Status "Unprocessable Entity (422)" beantworten.



REQ.Webservice.HttpServer.70

Die Implementierung **MUSS** einen zum Content passenden Content-Type Header gemäß ▶Tabelle 6.1 in der Response senden, sofern ein Content-Body vorhanden ist.



REQ.Webservice.HttpServer.80

Die Implementierung **MUSS** sicherstellen, dass die HTTP-Request-Response-Transaktion in der Implementierung beendet wird, wenn die TLS-Verbindung getrennt wird.

**REQ.Webservice.HttpServer.90**

Die Implementierung **MUSS** HTTP Status Codes 200-204 senden, wenn eine Anfrage an den Server ohne Fehler beendet wurde und Status Codes größer oder gleich 400 verwenden, wenn eine Anfrage mit Fehler beendet wird.

**REQ.Webservice.HttpServer.91**

Die Implementierung **MUSS** in der Response HTTP Status Codes mit der Semantik gemäß ▶Tabelle 6.2 verwenden.

**REQ.Webservice.HttpServer.110**

Die Implementierung **MUSS** die Methode GET mit der Semantik "Request Transfer" gemäß [RFC9110] Kapitel 9 verarbeiten und den Inhalt der adressierten, existierende Ressource in der Response exportieren.

**REQ.Webservice.HttpServer.120**

Die Implementierung **MUSS** die Methode PUT mit der Semantik "Replace" gemäß [RFC9110] Kapitel 9 verarbeiten, den Inhalt des Requests in die adressierte existierende Ressource übertragen.

**REQ.Webservice.HttpServer.130**

Die Implementierung **MUSS** die Methode DELETE mit der Semantik "Remove" gemäß [RFC9110] Kapitel 9 verarbeiten und die adressierte, existierende Ressource entfernen.

**REQ.Webservice.HttpServer.140**

(HAN-API) Die Implementierung **MUSS** die Methode POST für die adressierte Listen-Ressource (URI mit trailing "/") mit der Semantik "Create" bzw. "Append" gemäß [RFC9110] Kapitel 9 verarbeiten und den Inhalt des Requests in eine (neue) Ressource unterhalb der angegebenen Listen-Ressource übertragen.

**REQ.Webservice.HttpServer.150**

(HAN-API) Die Implementierung **MUSS** die Methode POST für die adressierte Entitäten-Ressource (URI ohne trailing "/") mit der Semantik "Provide data to data-handling process" gemäß [RFC9110] Kapitel 9 verarbeiten.

**REQ.Webservice.HttpServer.160**

Die Implementierung **SOLL** zur Minimierung der Datenmenge nur die für die Interoperabilität notwendigen Response-Header senden (siehe ▶Tabelle 6.3).

**REQ.Webservice.HttpServer.170**

Die Implementierung **MUSS** Request-Header-Fields mit einer Länge bis zu 250 Zeichen (inkl. CR, LF) verarbeiten können.

**REQ.Webservice.HttpServer.180**

Die Implementierung **MUSS** Request-Header mit einer Länge bis zu 8500 Zeichen (inkl. CR, LF) verarbeiten können.

**REQ.Webservice.HttpServer.190**

Sofern eine FA- oder KS-spezifische Vorgabe nichts anderes bestimmt, **MUSS** die Implementierung Requests innerhalb von 20 Sekunden mit einer Response beantworten.

**REQ.Webservice.HttpServer.201**

Die Implementierung **MUSS** Ressourcen-URIs gemäß ▶Tabelle 6.4, die über HKS.WS1 dem authentifizierten Service-Techniker (SRV) exponiert werden mit {role}/{version} = "srv/v1" bilden.

**REQ.Webservice.HttpServer.202**

Die Implementierung **MUSS** Ressourcen-URIs gemäß ▶Tabelle 6.4, die über HKS.WS1 dem authentifizierten CLS exponiert werden mit {role}/{version} = "cls/v1" bilden.

**REQ.Webservice.HttpServer.203**

Die Implementierung **MUSS** Ressourcen-URIs gemäß ▶Tabelle 6.4, die über HKS.WS1 oder HKS.WS2 dem authentifizierten Endkunden (CON) exponiert werden mit {role}/{version} = "con/v1" bilden.

**REQ.Webservice.HttpServer.210**

Die Implementierung **MUSS** die Filter-Aufrufparameter mit FA-spezifischen Query-Parametern gemäß ▶Abschnitt 6.8 implementieren.

**REQ.Webservice.HttpServer.220**

Die Implementierung **MUSS** für die Ressourcen-URI mit {role}/{version} = "con/v1", "cls/v1" und "srv/v1" die Transfersyntax JSON gemäß [RFC8259] implementieren.

**REQ.Webservice.HttpServer.230**

Das SMGW **MUSS** empfangene Nachrichten unter Berücksichtigung des Content-Type-Headers (siehe [DS] Kapitel "RESTful Webservice" Abschnitt "Content Type") verarbeiten.

**REQ.Webservice.HttpServer.240**

Das SMGW **MUSS** die an versendeten Responses unter Berücksichtigung des Accept-Headers (siehe [DS] Kapitel "RESTful Webservice" Abschnitt "Content Type") senden.

**REQ.Webservice.HttpServer.250**

Das SMGW **MUSS** für die Ressourcen-URI mit {role}/{version} = "con/v1", "cls/v1" und "srv/v1" den Content-Type "application/json" für die Response verwenden, wenn kein Accept-Header im Request vorhanden ist oder darüber nicht eindeutig ein Content-Type bestimmt werden kann.

Die Implementierung der Methode HEAD mit der Semantik gemäß [RFC9110] zur Abfrage von Metadaten ist nicht verpflichtend. Zur Verschlinkung der Schnittstelle verwendet der Client dazu die Methode GET und ignoriert die Inhaltsdaten.

6.3 Anforderungen an HTTP-Clients

Dieses Kapitel beschreibt die Anforderungen an das Verhalten der Implementierung eines HTTP-Clients.

**REQ.Webservice.HttpClient.10**

Die Implementierung **MUSS** das Format, d.h. die Syntax, der HTTP-Nachrichten an den von außen zugänglichen Schnittstellen gemäß der Spezifikation in [RFC9112] verarbeiten können.

**REQ.Webservice.HttpClient.50**

Die Implementierung **MUSS** HTTP-Statuscodes gemäß [RFC9110] Kapitel 15 mit der Semantik gemäß ▶Tabelle 6.2 verarbeiten können.

**REQ.Webservice.HttpClient.60**

Die Implementierung **MUSS** die Methode POST für die adressierte Entitäten-Ressource (URI ohne trailing "/") mit der Semantik "Post" gemäß [RFC9110] Kapitel 9 verwenden, um den Inhalt des Requests als Nachricht an den Empfänger zu übermitteln.

**REQ.Webservice.HttpClient.70**

Die Implementierung **MUSS** einen zum Content passenden Content-Type Header gemäß ▶Tabelle 6.1 im Request senden, sofern ein Content-Body vorhanden ist.

**REQ.Webservice.HttpClient.80**

Die Implementierung **MUSS** sicherstellen, dass die HTTP-Request-Response-Transaktion in der Implementierung beendet wird, wenn die zugeordnete TLS-Verbindung getrennt wird.

**REQ.Webservice.HttpClient.90**

Die Implementierung **SOLL** zur Minimierung der Datenmenge nur die für die Interoperabilität notwendigen Request-Header senden (siehe ▶Tabelle 6.3).

**REQ.Webservice.HttpClient.100**

Die Implementierung **MUSS** Response-Header-Fields mit einer Länge bis zu 250 Zeichen (inkl. CR, LF) verarbeiten können.

**REQ.Webservice.HttpClient.110**

Die Implementierung **MUSS** Response-Header mit einer Länge bis zu 1000 Zeichen (inkl. CR, LF) verarbeiten können.

**REQ.Webservice.HttpClient.120**

Die Implementierung **MUSS** die Transfersyntax JSON - sofern sie verwendet wird - gemäß [RFC8259] verarbeiten und senden können.

**REQ.Webservice.HttpClient.125**

Die Implementierung **MUSS** die Transfersyntax XML - sofern sie verwendet wird - gemäß [XML1.0] verarbeiten und senden können.

**REQ.Webservice.HttpClient.130**

Zur Reduktion des Datenvolumens **MUSS** die Implementierung den HTTP-Content mit dem Content-Encoding "gzip" gemäß [RFC1952] im POST-Request komprimiert übermitteln, sofern der Content-Type "application/json" oder "application/xml" ist.

6.4 Content-Types

► Tabelle 6.1 listet die in dieser Detailspezifikation referenzierten Content-Types auf. Kommunikationsszenarien beschreiben, welche Content-Types für die Mindestanforderungen an die Interoperabilität von der Implementierung unterstützt werden. Die Implementierung kann weitere Content-Types verwenden.

Sofern Inhaltsdaten durch CMS (siehe [DS] Kapitel "CMS - Cryptographic Message Syntax" gesichert werden, wird gemäß [RFC7193] der Content-Type "application/cms" verwendet. Über den in [RFC7193] definierten Parameter "encapsulatingContent" kann die Existenz und Reihenfolge von Kompression, Verschlüsselung und Signatur vereinbart werden, sofern dies nicht aus dem Kontext (Kommunikationsszenario) hervorgeht. Der "innerContent" Parameter entfällt, da er dem Default "data" entspricht. Sofern eine Ressource in verschiedenen Content-Types übermittelt werden kann, können HTTP-Server und HTTP-Client über den Accept-Header vereinbaren in welchem Format der Content-Type übermittelt wird. Die Transfersyntax-Angabe (+xml bzw. +json) bezieht sich auf den "innerContent".

Content-Type	Content-Body Schachtelung	Referenzen
application/cms+json; encapsulatingContent="signedData , authEnvelopedData , compressedData"	CMS SignedData (CMS AuthEnvelopedData (CMS CompressedData (json-Datenstruktur)))	CMS gemäß [DS] Kapitel "CMS - Cryptographic Message Syntax", Datenkompression gemäß [RFC3274], json Transfersyntax gemäß [RFC8259]. Datenstruktur gemäß Webservice-API zu dieser TR.
application/cms+json; encapsulatingContent="signedData , authEnvelopedData"	CMS SignedData (CMS AuthEnvelopedData (json-Datenstruktur))	CMS gemäß [DS] Kapitel "CMS - Cryptographic Message Syntax", json Transfersyntax gemäß [RFC8259]. Datenstruktur gemäß Webservice-API zu dieser TR.
application/cms+json; encapsulatingContent="signedData"	CMS SignedData (json-Datenstruktur)	CMS gemäß [DS] Kapitel "CMS - Cryptographic Message Syntax", json Transfersyntax gemäß [RFC8259]. Datenstruktur gemäß Webservice-API zu dieser TR.
application/cms+xml; encapsulatingContent="signedData , authEnvelopedData , compressedData"	CMS SignedData (CMS AuthEnvelopedData (CMS CompressedData (XML-Datenstruktur)))	Datenstruktur gemäß [DS] Kapitel "CMS - Cryptographic Message Syntax", Datenkompression gemäß [RFC3274] und Transfersyntax gemäß [XML1.0] mit SignedData(AuthEnvelopedData(CompressedData(XML-Datenstruktur)))
application/vnd.de-dke-k461-co-sem+xml; encaps=cms-tr03109		
application/json	json-Datenstruktur	Transfersyntax gemäß [RFC8259]
application/xml	XML-Datenstruktur	Transfersyntax gemäß [XML1.0]
application/octetstream	File-Download	
multipart/byteranges	Byterange von File-Download	[RFC9110] Partial Response großer File-Downloads

Tabelle 6.1 Bedeutung der Content-Types

6.5 Status Codes

HTTP-Statuscodes werden vom HTTP-Server in der Response an den HTTP-Client übermittelt, um das Ergebnis der Verarbeitung des Requests im HTTP-Server zusammenzufassen. ► Tabelle 6.2 konkretisiert die Semantik der Statuscodes aus [RFC9110] für die Verwendung mit den Kommunikationsszenarien. Client-Error Status Codes (4xx) Statuscodes, die nicht in [RFC9110] definiert sind und von der Implementierung nicht unterstützt werden, können auf "Bad request (400)" abgebildet werden.

Status	Beschreibung
Anfrage erfolgreich ausgeführt	
Ok (200)	Die GET, POST Anfrage wurde erfolgreich bearbeitet und das Ergebnis der Anfrage wird in der Antwort übertragen.
Created (201)	Die POST-Anfrage wurde erfolgreich bearbeitet. Die angeforderte Ressource wurde vor dem Senden der Antwort erstellt.

Status	Beschreibung
Accepted (202)	Die POST-Anfrage wurde akzeptiert, wird aber zu einem späteren Zeitpunkt (asynchron) ausgeführt. Das Gelingen der Anfrage kann nicht garantiert werden.
No Content (204)	Die DELETE, PUT, POST Anfrage wurde erfolgreich durchgeführt, die Antwort enthält keine Ressource-Inhaltsdaten.
Partial Content (206)	GET File: Der angeforderte Teil wurde erfolgreich übertragen (wird im Zusammenhang mit einem "Content-Range"-Header-Field oder dem Content-Type multipart/byteranges verwendet). Kann einen HTTP-Client über Teil-Downloads informieren (wird zum Beispiel von Wget genutzt, um den Downloadfortschritt zu überwachen oder einen Download in mehrere Streams aufzuteilen).
Client-seitig fehlerhafte Anfrage führt zum Abbruch	
Bad Request (400)	Die Anfrage-Nachricht war fehlerhaft aufgebaut. Der HTTP-Client soll die Anfrage nicht unverändert noch einmal stellen.
Unauthorized (401)	Die Anfrage kann nicht ohne gültige Authentifizierung durchgeführt werden. Wie die Authentifizierung durchgeführt werden soll, wird im "WWW-Authenticate"-Header-Field der Antwort übermittelt.
Forbidden (403)	Die Anfrage wurde mangels Berechtigung des HTTP-Clients nicht durchgeführt, bspw. weil der authentifizierte Benutzer nicht berechtigt ist.
Not Found (404)	Die angeforderte Ressource wurde nicht gefunden.
Method Not Allowed (405)	Die Anfrage darf nur mit anderen HTTP-Methoden (zum Beispiel GET statt POST) gestellt werden.
Not Acceptable (406)	Die angeforderte Ressource steht nicht in der gewünschten Form (gemäß Accept-Header) zur Verfügung.
Request Timed Out (408)	Innerhalb der vom HTTP-Server erlaubten Zeitspanne wurde keine vollständige Anfrage des HTTP-Clients empfangen.
Conflict (409)	Die Anfrage wurde unter falschen Annahmen gestellt. Eine existierende Ressource kann nicht erneut angelegt werden.
Length Required (411)	Die Anfrage kann ohne ein "Content-Length"-Header-Field nicht bearbeitet werden.
Content Too Large (413)	Die Content-Body der gestellten Anfrage war zu groß, um vom HTTP-Server bearbeitet werden zu können.
Request URL Too Large (414)	Die URL der Anfrage war zu lang.
Unsupported Media Type (415)	Der Inhalt der Anfrage wurde mit ungültigem oder nicht unterstützen Content-Type übermittelt.
Requested Range Not Satisfiable (416)	Der angeforderte Teil einer Ressource war ungültig oder steht auf dem HTTP-Server nicht zur Verfügung.
Unprocessable Entity (422)	Syntaktischer Fehler in den Inhaltsdaten der Anfrage. CMS- oder Transfersyntax bzw. XML/json Struktur des Content-Body kann nicht ausgewertet werden (RFC4918).
Request Header Fields Too Large (431)	Die Maximallänge eines HTTP-Header-Fields oder des Gesamtheaders wurde überschritten (RFC6585).
Server-seitiger Fehler führt zum Abbruch	
Internal Server Error (500)	Allgemeiner Statuscode für Fehler des Servers auf FA-Ebene, der im Content-Body konkretisiert werden kann.
Bad Gateway (502)	Der HTTP-Server konnte seine Funktion als Gateway oder Proxy nicht erfüllen, weil er seinerseits keine oder eine ungültige Antwort erhalten hat.
Service Unavailable (503)	Der Anwendungsdienst steht temporär nicht zur Verfügung.
HTTP Version Not Supported (505)	Die im Request verwendete HTTP-Version wird vom HTTP-Server nicht unterstützt oder abgelehnt.

Tabelle 6.2 HTTP-Statuscodes

6.6 Header-Felder

HTTP-Header werden in Request und Response des HTTP-Servers und des HTTP-Clients zur Vereinbarung der Verarbeitung der Inhaltsdaten (Content-Body) übermittelt. ▶Tabelle 6.3 konkretisiert die in [RFC9110] beschriebenen Header für die Verwendung mit den Kommunikationsszenarien.

Name	Verwendung	Kontext
Allgemeine Header-Felder		
Content-Length	Gemäß [RFC9110].	• GET-Response • POST-Request • POST-Response • PUT-Request
Content-Type	Gemäß ▶Tabelle 6.1.	• GET-Response • POST-Request • POST-Response • PUT-Request
Accept	Gemäß ▶Tabelle 6.1.	• GET-Request • POST-Request
Accept-Encoding	Gemäß [RFC9110]	• GET-Request • POST-Request
Content-Encoding	Gemäß [RFC9110]	• PUT-Request • GET-Response • POST-Request • POST-Response
File-Downloads		
Range	Gemäß [RFC9110].	GET-Request File-Download
Content-Range	Gemäß [RFC9110]	GET-Response File-Download
Digest-Authentifizierung		
Authorization	Gemäß [RFC9110].	Request (HAN-Webservice mit lokaler Nutzerauthentifizierung)
WWW-Authenticate	Gemäß [RFC9110].	Response (HAN-Webservice mit lokaler Nutzerauthentifizierung)

Tabelle 6.3 Header

6.7 URI-Pfade für die HAN-API

URI-Pfade werden gemäß [RFC3986] in der Form "path-absolute" gebildet. ▶Tabelle 6.4 listet die Umsetzung des FA-Bezeichners auf die URI-Pfade auf.

Dabei ist {role}/{version} ein Platzhalter, der die API-Variante beschreibt. Diese unterscheidet sich nach Interface- und Zugriffsrolle.

{ResourceType} identifiziert den Typ der Datenstruktur der Ressource (Beispielsweise Konfigurations- oder Statusprofile, die in den Fachanwendungsfälle verwendet werden). Die Unterscheidung von Lower/Uppercase des Ressource Typs ist zur Identifikation relevant.

{Action} identifiziert die Funktion des auslösenden FA.

{ResourceId} identifiziert eine individuelle Instanz (Ressource) eines {ResourceType}s. Für Konfigurationsprofile besteht eine eindeutige Beziehung zum im SMGW eindeutigen Bezeichner des Profiles. Die Unterscheidung von Lower/Uppercase des Ressource Typs ist zur Identifikation relevant. Entspricht die {ResourceId} einer Geräteidentifikation {DeviceId} nach [DIN43849], dann ist sie 14-stellig ohne Leerzeichen mit den Zeichen 0-9 und a-z anzugeben.

{ResourceContent} enthält den Inhalt einer identifizierten Ressource (eines {ResourceType}s).

{EnvelopedResourceContent} enthält ein oder mehrere Instanzen des {ResourceContent} in einer Envelope eingebettet. Die Datenstruktur des Enveloped ist RessourceType-spezifisch in [TR-03109-1v2.0] beschrieben.

[[ResultContent]] Optional den Response-Status ergänzende Informationen zum Ergebnis (siehe [TR-03109-1v2.0] Anhang der zu protokollierenden Ereignisse)

FA-Bezeichner	Aufrufparameter	URI	Rückgabeparameter
FA.Read{ResourceType}	{ResourceId}	GET /smgw/{role}/{version}/rts/{ResourceType}/{ResourceId}	{ResourceContent}
FA.Update{ResourceType}	{ResourceId} {ResourceContent}	PUT /smgw/{role}/{version}/rts/{ResourceType}/{ResourceId}	[[ResultContent]]
FA.Create{ResourceType}	{ResourceContent} mit {ResourceId}	POST /smgw/{role}/{version}/rts/{ResourceType}/	[[ResultContent]]
FA.Delete{ResourceType}	{ResourceId}	DELETE /smgw/{role}/{version}/rts/{ResourceType}/{ResourceId}	-
FA.Do{Action}	(Abhängig vom FA)	POST /smgw/{role}/{version}/fas/{Action}	[[ResultContent]] FA-spezifisch
FA.Export{ResourceType}	{ResourceId}	GET /smgw/{role}/{version}/exp/{ResourceType}/{ResourceId}[?{QueryParam}]	{EnvelopedResourceContent}
FA.ListSmgwResources	-	GET /smgw/	URI-Liste /smgw/{role}/{version}/
FA.ListSmgwResources	-	GET /smgw/{role}/{version}/	URI-Liste /smgw/{role}/{version}/rts/, URI-Liste /smgw/{role}/{version}/fas/, URI-Liste /smgw/{role}/{version}/exp/
FA.ListSmgwResources	-	GET /smgw/{role}/{version}/rts/	URI-Liste /smgw/{role}/{version}/rts/{ResourceType}/
FA.ListSmgwResources	-	GET /smgw/{role}/{version}/rts/{ResourceType}/	URI-Liste /smgw/{role}/{version}/rts/{ResourceType}/{ResourceId}
FA.ListSmgwResources	-	GET /smgw/{role}/{version}/exp/	URI-Liste /smgw/{role}/{version}/exp/{ResourceType}/
FA.ListSmgwResources	-	GET /smgw/{role}/{version}/exp/{ResourceType}/	URI-Liste /smgw/{role}/{version}/exp/{ResourceType}/{ResourceId}
FA.ListSmgwResources	-	GET /smgw/{role}/{version}/fas/	URI-Liste /smgw/{role}/{version}/fas/{Action}

Tabelle 6.4 URI-Pfade

6.8 URI Query-Parameter

QueryParameter können Bestandteil einer URI nach [RFC3986] sein. Diese werden zur Selektion von Ressourcen in RESTful APIs verwendet. Da die Syntax des Query Strings "query" nicht in [RFC3986] Kapitel 3.4 beschrieben ist, spezifiziert diese Detailspezifikation die Struktur und Syntax der Query Parameter.

Die möglichen Query-Parameter sind FA-spezifisch. Die Mindestens zu implementierenden Query-Parameter werden in den FA-Anforderungen konkretisiert.

Struktur und Syntax der URI QueryParameter

Die Grammatik der QueryParameter wird mit Augmented BNF nach [RFC5234] beschrieben:

Der Query String wird durch ein "?" eingeleitet. Auf dieses Zeichen folgen die QueryParameter. Die QueryParameter **SOLLEN** nach der Bildungsregel <query-component> validiert werden. [REQ.HTTP.QueryParameter.10]

query-component = parameter [*("&" parameter)]

parameter = name "=" value

name = *qchar mit einer Länge von 1..50 Zeichen

value = *qchar mit einer Länge von 0..255 Zeichen

qchar = unreserved / pct-encoded / qspecial

qspecial = "/" / "?" / ":" / "@" / "!" / "\$" / "'" / "(" / ")" / "*" / "+" / "," / ";"

Die folgenden Regeln sind in [RFC3986] (normativ) definiert. Sie werden hier wiederholt:

unreserved = ALPHA / DIGIT / "-" / "." / "_" / "~"

pct-encoded = "%" HEXDIG HEXDIG



Anmerkung

- Diese Grammatik erlaubt die Verwendung aller nach [RFC3986] zulässigen Zeichen.
- In der URI ist kein Leerzeichen erlaubt (s. [RFC3986] Appendix C).
- [RFC3986] erlaubt keine leeren QueryParameter. D.h. falls "?" in der URI vorkommt, dann muss mindestens ein Parameter folgen.
- Die oben definierte Regel <qchar> entspricht der Regel <pchar> aus [RFC3986], die die zulässigen Zeichen für die *QueryParameter* definiert, ohne die Zeichen "=" und "&".

Parameter	Wertebereich	Bedeutung
q.fromtime	String (date) oder String (date-time)	Startzeitpunkt. Es werden nur Einträge exportiert, die jünger oder gleich (inklusive) dem Startzeitpunkt sind. Fehlt dieser Parameter oder liegt er vor dem Inbetriebnahmezeitpunkt des Gerätes, wird der Inbetriebnahmezeitpunkt des Gerätes angenommen.
q.totime	String (date) oder String (date-time)	Endzeitpunkt. Es werden nur Einträge exportiert, die älter (exklusive) dem Endpunkt sind. Fehlt dieser Parameter oder liegt er in der Zukunft, wird der Default "aktueller Zeitpunkt" verwendet.
q.type	String (name)	Es werden nur Einträge exportiert, die dem Typ genau entsprechen. Fehlt dieser Parameter werden keine Beschränkungen auf den Typ festgelegt (Beispiel: Event-Type)
q.count	String (number)	Max. Anzahl zu exportierender Datensätze. Fehlt der Parameter oder ist er 0, wird die Anzahl nicht begrenzt.
q.obis	String (A-B:C.D.E[*F])	Es werden nur Einträge exportiert, die der OBIS-ID entsprechen. Fehlt die Value Group "F", wird 255 angenommen. Fehlt der Parameter, werden die Einträge beim Export nicht nach OBIS-ID begrenzt.
q.level	String (1). I, W, E, F	Es werden nur Einträge exportiert, die den Statuslevel Information, Warning, Error oder Fatal entsprechen. Fehlt dieser Parameter, werden die Einträge beim Export nicht nach Statuslevel begrenzt.
q.signed	String (name)	Bezeichner des Signaturverfahrens für den Export. Fehlt dieser Parameter werden die Exportierten Daten nicht signiert. Wird CMS_ECDSA_SHA256_Detached angegeben, enthält die Exportierte Datenstruktur eine Signatur gemäß DS CMS SignedData über die kanonisierten Datenfelder (Für JSON siehe ▶Abschnitt 6.9)

Tabelle 6.5 URI Query Parameter

6.9 Kanonisierung für JSON Signaturen

JSON ermöglicht zur Verbesserung der Lesbarkeit das Einfügen von CR/LF und sog. Whitespaces in der Transfersyntax. Ohne Veränderung des Inhaltes oder der Semantik würden so verschiedene Nachrichten erzeugt. Zur Verbesserung der Interoperabilität werden die möglichen Freiheitsgrade durch eine Kanonisierungsregel

eingeschränkt. Die kanonisierten Daten sind nicht an den Schnittstellen sichtbar, da sie nur als Grundlage für die Hash-Berechnung dienen, der der Signaturberechnung zugrundeliegt.

Zur Erzeugung eines Kanonisierten Strings werden alle CR (0x0D), LF (0x0A), TAB (0x09), SPACE (0x20) zwischen den serialisierten JSON-Komponenten entfernt.

7 HTTP-Authentifizierungs-Protokoll

7.1 Einleitung

Dieses Kapitel beschreibt die Anforderungen an die Authentifizierung von Nutzern über einen Webservice mittels Nutzerkennung (Username) und Passwort.

Die Authentifizierung erfolgt über das HTTP-Digest Authentifizierungsverfahren. Dabei sendet der Nutzer zunächst einen HTTP-Request an das SMGW, um auf eine Ressource über eine URI zuzugreifen. Das SMGW berechnet eine zufällige "Challenge" und übermittelt den "Realm" (Bereich), für den die Authentifizierung erforderlich ist. Der Nutzer wird aufgefordert seinen Username und sein Passwort im Web-Client des Nutzers einzugeben. Aus der Challenge, einer Nonce, der URI und dem Hash über Realm, Username und Passwort berechnen sowohl SMGW als auch Web-Client einen Hash ("Digest"). Der Web-Client sendet den Hash an das SMGW. Nur falls der vom SMGW berechnete Hash mit dem vom Web-Client empfangenen Hash übereinstimmt, wird die Autorisierung zum Zugriff auf die Ressource (URI) erteilt.

Als "Realm" wird für das SMGW die kanonisierte SMGW-ID (lowercase) verwendet.

7.2 Anforderungen an den HTTP-Server

Das SMGW **MUSS** die Authentifizierung über HTTP-Digest Authentication gemäß [RFC7616] durchführen. [REQ.BenutzerAuth.Allgemein.10]

Das SMGW **SOLL** die Digest-Berechnung basierend auf dem SHA-256 Verfahren anbieten. [REQ.BenutzerAuth.Allgemein.20]

Das SMGW **KANN** die Digest-Berechnung basierend auf dem MD5 Verfahren anbieten, sofern der anfragende HTTP-Client kein Digest-Verfahren mit SHA-256 unterstützt. [REQ.BenutzerAuth.Allgemein.30]



ICS.BenutzerAuth.Allgemein.10

Der GWH **MUSS** im ICS deklarieren, ob das SMGW für die Digest-Authentifizierung nach [RFC7616] das SHA-256-Verfahren unterstützt.



ICS.BenutzerAuth.Allgemein.20

Der GWH **MUSS** im ICS deklarieren, ob das SMGW das MD5-Verfahren unterstützt.

8 LMN HDLC Protokoll

8.1 Einleitung

Dieses Kapitel konkretisiert die Verwendung des High-Level Data Link Control (HDLC) Transportprotokolles zur bidirektionalen LMN-Kommunikation über eine physische RS-485 Schnittstelle. Dabei nimmt das SMGW als TLS-Client die Rolle der HDLC Primary Station ein.

Das HDLC-Protokoll stellt sowohl einen zuverlässigen verbindungsorientierten Transportkanal zwischen Primary- und Secondary Station bereit, als auch einen verbindungslosen Nachrichtenaustausch.

Das HDLC-Protokoll realisiert auf dem OSI Layer 2 und 3 (Data Link und Network) den zuverlässigen verbindungsorientierten oder verbindungslosen Transport über ein Bussystem (Netzwerk) mit mehreren Teilnehmern, aber genau einer Primary Station.

Dieses Kapitel beschreibt den Empfang und Versand von HDLC-Frames, zum Verbindungsauf- und abbau und zur HDLC-Adressvergabe und HDLC-Adressauflösung. Anforderungen an die Verwendung des HDLC-Protokolles sind in [ISO13239] beschrieben.

[ISO13239] bietet eine Vielzahl von Betriebsmodi für HDLC an. Diese Modi werden durch [DIN VDE 0418-63-7] auf eine Auswahl eingeschränkt, die implizit für die Kommunikation zwischen SMGW und Messeinrichtung als vereinbart gilt:

- Non-basic frame format transparency, d.h. ohne Bit-Stuffing, ohne Control-Escape nach [ISO13239]ISO13239 Kapitel 4.3.4 (möglich, da ein Längenfeld im Frame Control Field verwendet wird).
- Normal Response Mode („Master/Slave“ Betriebsart für Halb-Duplex Medien).
- Unbalanced operation Normal Response Mode Class (UNC) Frame Control Type Set.
- Unbalanced operation Connectionless mode Class (UCC) Frame Control Type Set.
- Control-Field Format für Modulo 8 Sequence Numbers (Window Size 1).

Die von der Implementierung umzusetzenden technischen Anwendungsfälle sind:

- Verbindungsaufbau
- Verbindungsabbau
- Erkennung neuer HDLC-Teilnehmer
- Erkennung verstummter HDLC-Teilnehmer
- Erzeugung von HDLC-Frames für TLS mit SML als Applikationsprotokoll (LKS1)
- Erzeugung von HDLC-Frames für das "SYM-Protokoll" zum Initialen Pairing (LKS1.PSK)
- Verarbeitung von HDLC-Frames mit TLS und SML als Applikationsprotokoll (LKS1)
- Verarbeitung von HDLC-Frames mit dem "SYM-Protokoll" zum Initialen Pairing (LKS1.PSK)

HDLC-Frame Format

Der HDLC-Frame wird nach [ISO13239]ISO13239 Annex H.4 (Frame Format Type 3) gebildet. ►Abbildung 8.1 stellt einen solchen Frame dar.

Hierbei gilt:

- Destination Address und Source Address haben jeweils eine Länge von zwei Bytes.
- Das Control-Feld hat eine Länge von einem Byte.
- Das Informationsfeld hat eine Länge von 0..2034 Bytes.

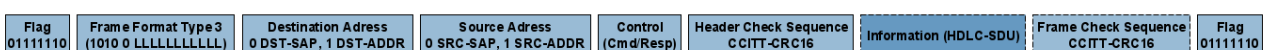


Abbildung 8.1. HDLC-Frame Struktur Type 3

Zuordnung HDLC-Adresse und Geräte-Identifikation

Um die Auflösung der Geräte-Identifikation zu HDLC-Adressen durchzuführen und eine zuverlässige HDLC-Kommunikation zu gewährleisten, verwaltet die Primary Station pro LMN-Teilnehmer die folgenden (nicht persistierten) Informationen:

- Zustand der Data Link Assoziation
- HDLC Link-Layer Adresse des Teilnehmers (2, 3..126)
- Zugeordnete Geräte-Identifikation der Messeinrichtung nach [DIN43849].
- HDLC-Send Sequence Number V(S) (Modulo 8) (nach Empfang der Bestätigung des SNRM-Kommandos auf 0 gesetzt).
- HDLC-Receive Sequence Number V(R) (Modulo 8) (nach Empfang der Bestätigung des SNRM-Kommandos auf 0 gesetzt).
- HDLC-Response Timeout Timer.
- Weitere, wie z.B. HDLC-Retry-Zähler, Empfangs- und Sendepuffer.

8.2 Anforderungen

Das SMGW **MUSS** die leitungsgebundene, asynchrone, serielle, Halb-Duplex-Kommunikation nach EIA/RS-485 zwischen Messeinrichtung und SMGW gemäß [DIN VDE 0418-63-7] umsetzen. [REQ.TLSHDLC.SMGW.10]

Das SMGW **MUSS** das HDLC Link-Layer- und Transportprotokoll nach den Anforderungen der Spezifikation [ISO13239] als primäre Kommunikationsstation gemäß der Anforderungen nach [DIN VDE 0418-63-7] umsetzen, um eine zuverlässige Übertragung zu gewährleisten. [REQ.TLSHDLC.SMGW.20] Das SMGW **MUSS** die automatische Zuordnung (Adressauflösung) von HDLC Adressen und Identifikation der Messeinrichtung nach [DIN43849] gemäß [DIN VDE 0418-63-7] umsetzen. [REQ.TLSHDLC.SMGW.30]

Das SMGW **MUSS** TLS nach den Anforderungen von [TR-03109-3] in der Rolle des TLS-Clients gemäß der Anforderungen nach [DIN VDE 0418-63-7] umsetzen, um vertrauliche und authentische Transportverbindungen zwischen SMGW und einem Kommunikationspartner zu gewährleisten. [REQ.TLSHDLC.SMGW.40]

8.3 Technische Anwendungsfälle der HDLC Primary Station (Client)

8.3.1 TA.SYMHDLC.SMGW.InitialPairing

Die Implementierung **MUSS** die initiale kommunikative Anbindung der Messeinrichtung an das SMGW mit dem Symmetrischen Verfahren nach [TR-03109-3] gemäß [DIN VDE 0418-63-7] Kapitel "Initialer TLS-Zertifikatstausch, SYM-Messages" mit dem vom Hersteller vergebenen, geräteindividuellem PSK aus dem LMN-Kommunikationsprofil umsetzen können, um authentisch Zertifikate auszutauschen und das vom SMGW erzeugte Schlüsselpaar der Messeinrichtung vertraulich an die Messeinrichtung zu übermitteln. [REQ.SYMHDLC.SMGW.InitialPairing.10]

Der Wechsel des symmetrischen Schlüssels (PSK) $MK_0 \rightarrow MK'$ ([DIN VDE 0418-63-7] Bild 8 Phase 2b) ist optional.

► Abbildung 8.2 zeigt die Schlüsselableitung zur Berechnung des Authentifizierungs- und Verschlüsselungsschlüssels für das PSK-basierte Pairing.

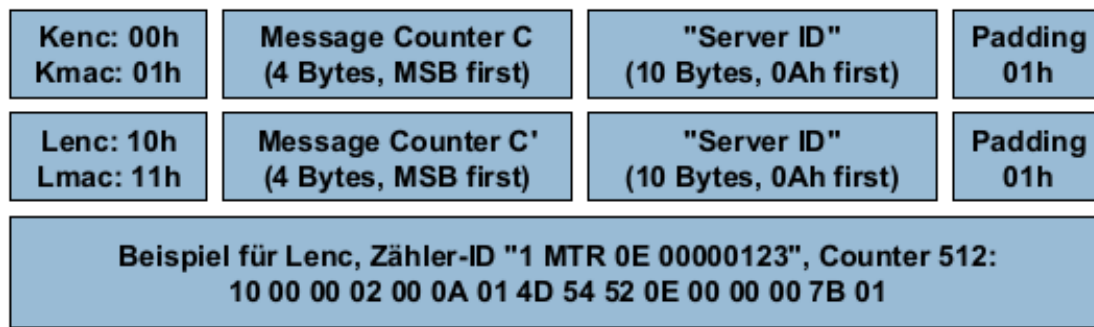


Abbildung 8.2. Übersicht über die Schlüsselableitung für das PSK-Verfahren

8.3.2 TA.SYMHDLC.SMGW.RemovePairingInMtr

Das SMGW **MUSS** das Aufheben der Assoziation der Messeinrichtung mit dem SMGW gemäß [DIN VDE 0418-63-7] Kapitel "Crypto-Reset SYM-Ablauf" auf Basis des geräteindividuellen, vom Hersteller vergebenen "Pairing-Key" (MK0) umsetzen. [REQ.SYMHDLC.SMGW.RemovePairingInMtr.10] Dieser Ablauf verwendet SYM-Message-Type 6 und SYM-Message-Type 7.

8.4 Implementierungshinweise

Aufgrund der Empfehlung zur künftigen Verwendung von TLS1.3 (siehe [RFC8446]) im iMSys (siehe [TR-03109-3]) sind für die Interoperabilität Implementierungshinweise zur Verwendung von [DIN VDE 0418-63-7] erforderlich.

1. [DIN VDE 0418-63-7][LMN_0153] fordert die Übermittlung der von der Messeinrichtung unterstützten Ciphersuite- und Named-Curve ("supported_group") IDs zum SMGW. Messeinrichtungen, die sowohl TLS1.2 als auch TLS1.3 unterstützen, können in den entsprechenden Datenfeldern die unterstützten und gemäß [TR-03109-3] zulässigen TLS1.2- als auch TLS1.3-Ciphersuite-IDs und (brainpool)-Supported Group IDs gemäß IANA-Registry "TLS-Parameters" angeben. Das SMGW kann die Information verwenden um den TLS-Handshake Client Hello zur Messeinrichtung entsprechend der Fähigkeiten der Messeinrichtung anzupassen. Bevorzugt sollte die Vereinbarung der TLS-Parameter allerdings über die Vereinbarung im TLS-Handshake erfolgen.
2. Entgegen [DIN VDE 0418-63-7] [LMN_0188] wird für Messeinrichtungen empfohlen die PSK-basierte TLS1.3-Session Resumption gemäß [RFC8446] Kap 2.2 zu unterstützen.
3. Die Verwendung von HDLC gemäß [DIN VDE 0418-63-7] als Transportlayer für TLS erfordert, dass die über HDLC transportierten TLS-Records nicht größer als 2034 Bytes werden.¹ Dies wird durch die Vereinbarung einer Max. Fragmentlänge (TLS-Extension max_fragment_length²) zwischen Zähler und SMGW auf 2048 Bytes vereinbart. Die Implementierung muss beim Erstellen von TLS-Records die Einhaltung des 2034 Bytes Limits für HDLC gewährleisten. Insbesondere wenn die TLS-Handshake-Records durch Zertifikate oder TLS-Extensions größer als 2034 Bytes werden, muss die Implementierung für eine TLS-Fragmentierung der TLS-Handshake-Records vorbereitet sein.

Da die Identifikation der Messeinrichtung in verschiedenen Standards unterschiedlich benannt ist und in Geräten auch unterschiedlich implementiert wurde, erfolgt folgende Klarstellung:

- Die „meterId“ identifiziert die Messeinrichtung auf Anwendungsebene mit Herstellerübergreifend eindeutiger ID (HÜID), die im kanonisierten Format gemäß [DIN43849]³ notiert wird. Diese ID entspricht der auf der Messeinrichtung gemäß [DIN43849] aufgedruckten ID.

¹ Der Wert für die SDU ergibt sich aus der HDLC-Framelänge PDU von 2048 Bytes abzüglich der Header- und CRC-Informationen.

² Die Extension record_size_limit gemäß [RFC8449] wäre geeigneter, da die Record-Länge byte-genau begrenzt wird und weil das Limit vom Zähler (TLS-Server) vorgegeben wird. RFC8449 wird allerdings kaum von TLS-Libraries unterstützt.

³ Bisher DIN43863-5: 2012-04

- Die „Sensor-ID“ in [DIN VDE 0418-63-7] wird aus der "meterId" des LMN-Kommunikationsprofils in das Format einer „Server-ID“ abgeleitet.
- Die „LMN-Teilnehmer-ID“ in [DIN VDE 0418-63-7] identifiziert den Kommunikationspartner auf Netzwerk- oder Linkebene und wird für die Schlüsselableitung und die Messdatenverarbeitung nicht verwendet. „LMN-Teilnehmer-ID“ und „Sensor-ID“ unterscheiden sich im Falle eines Kommunikationsadapters oder Repeaters.
- Für Kommunikationsadapter bzw. BAB entspricht die Geräte-ID der „Server-ID“ der Messeinrichtung und nicht der ID des Kommunikationsadapters (genannt „Sensor-ID“ in der betrachteten Kommunikation gemäß [DIN VDE 0418-63-7]).
- Das SMGW MUSS die Geräte-ID gemäß [DIN43849] als Server-ID im Format 0x0A für die Schlüsselableitung nach [TR-03109-3] als Zähler-ID verwenden (FNN LH Basiszähler 1.4), sofern kein herstellerspezifischer Parameter im Zählerprofil eine für die Schlüsselableitung zu verwendende Zähler-ID (gemäß [TR-03109-3]) vorgibt (siehe nächster Spiegelpunkt).
- Um Messeinrichtungen und Kommunikationsadapter zu unterstützen, deren Zähler-ID für die Schlüsselableitung gemäß [TR-03116-3] nicht gemäß Abbildungsregel (s.u.) in eine Server-ID des Formates 0x0A abgebildet werden kann, kann der GWA im Zählerprofil ein weiteres Datenfeld konfigurieren. Dieses Feld enthält die vom Zählerhersteller an den MSB übermittelte Server-Id des Zählers, die für die Schlüsselableitung und kommunikative Anbindung der Messeinrichtung verwendet wird. Sofern dieses Datenfeld einen Octet-String enthält, SOLL das SMGW dieses für die Schlüsselableitung gemäß [TR-03109-3] als Zähler-ID bzw. in der LMN-Kommunikation als Sensor-ID verwenden. Für die Verknüpfung mit Auswertungsprofilen und im Versand an EMT wird die Geräte-ID gemäß [DIN43849] verwendet.

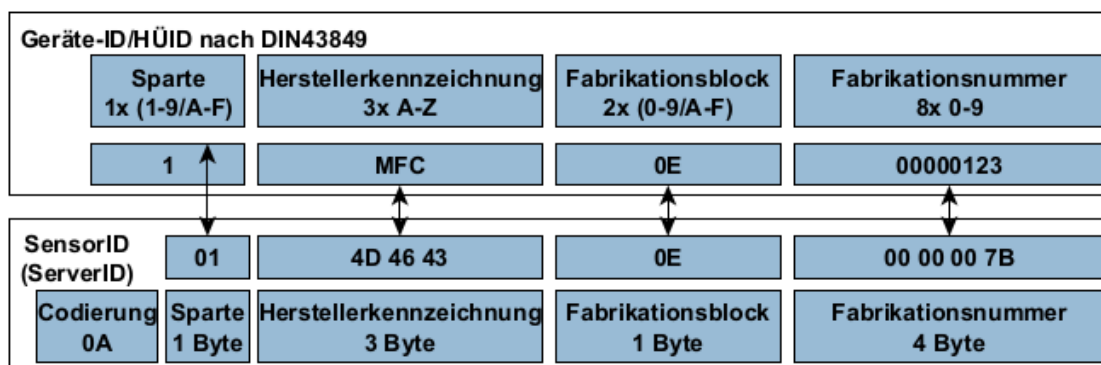


Abbildung 8.3. Abbildungsvorschrift von der Geräte-ID/HÜID auf die SensorID (ServerID)

9 SML - Smart Message Language

9.1 Einleitung

Dieses Kapitel beschreibt Anforderungen an die Implementierung eines SML-Clients zur Erzeugung und Verarbeitung von SML-Dateien und SML-Nachrichten, um Messwerte von einer Kommunikationseinheit (KE) der Messeinrichtung zu erhalten und die Konfiguration der KE der Messeinrichtung zu unterstützen.

Dieses Kapitel konkretisiert die Anforderungen der [DIN VDE 0418-63-9] zur Verwendung in intelligenten Messsystemen. Die Anforderungen werden dabei in Form von TA beschrieben.

Das SML-Protokoll beschreibt eine binärcodierte Transfer-Syntax für Nachrichten, ein Request-Response Verfahren für den Nachrichtenaustausch, eine Menge von Datenstrukturen z.B. für den Versand von Messwerttupeln oder Parametersätzen mit oder ohne COSEM-Klassensemantik.

9.2 Datenstrukturen

- Die Datentypen einer SML-Nachricht sind in [DIN VDE 0418-63-9] Kapitel 5 beschrieben.
- Die Binär-Codierung einer SML-Nachricht ist in [DIN VDE 0418-63-9] Kapitel 6 beschrieben.
- Das Integritätssicherungs-Rahmenformat (SML-Transport-Protokoll Version 1) ist in [DIN VDE 0418-63-9] Kapitel 8.1 beschrieben.
- Die Abfrage und Übermittlung von Messwerten und Parametern findet über COSEM-Klassen statt. Die Abbildung der Zugriffe auf COSEM-Objekte ist in [DIN VDE 0418-63-9] Kapitel 5.2 beschrieben.
- Es werden folgende COSEM-Interface-Klassen (Class-ID) verwendet:
 - Data (1), beschrieben in [EN62056-6-2].
 - Register (3), beschrieben in [EN62056-6-2].
 - Advanced Extended Register (32770) entspricht bis auf das Attribut "capture_time" dem Extended Register (4) (beschrieben in [EN62056-6-2]).

9.3 TA des SML-Clients

Die folgenden Abschnitte enthalten die TA des SML-Clients für die Erzeugung von SML-Requests (SML-Auftragsdateien mit SML-Request-Messages) und die Verarbeitung von SML-Responses (SML-Antwortdateien mit SML-Response-Messages).

Das SMGW implementiert an der LMN-Schnittstelle einen SML-Client.

Technischer Anwendungsfall	Beschreibung	Verwendung mit
▶TA.SML.CLI.UpdateMtrCertAndKey	Erneuerung des Schlüsselpaares und Zertifikats in der Messeinrichtung	Registrierende und Nicht-Registrierende Messeinrichtung (Basiszähler)
▶TA.SML.CLI.UpdateSmgwCert	Erneuerung des SMGW-Zertifikates in der Messeinrichtung	
▶TA.SML.CLI.RequestMeterData	Abruf von Messwerten aus der Messeinrichtung	
▶TA.SML.CLI.ProcessMeterData	Empfang der von der Messeinrichtung gelieferten Messwerte	
▶TA.SML.CLI.UpdateTime	Synchronisieren/Stellen der Zeit in der Messeinrichtung auslösen	Registrierende Messeinrichtung
▶TA.SML.CLI.TransferFileToMeter (Übermittlung einer Datei vom SMGW an die Messeinrichtung)	Dateiübertragung (Firmware, Parameter) an die Messeinrichtung	
▶TA.SML.CLI.GetClockState (Abfrage des Zustandes der Uhr)	Abruf des Zeitsynchronisationsstatus von der Messeinrichtung	

Technischer Anwendungsfall	Beschreibung	Verwendung mit
► TA.SML.CLI.ReadFileFromMeter (Übermittlung einer Datei von der Messeinrichtung an das SMGW)	Dateiübertragung (Logs) von der Messeinrichtung an das SMGW	
► TA.SML.CLI.ReadDeviceInfo (Abfrage des Gerätetyps und der SW-Version)	Abruf der Geräteinformationen von der Messeinrichtung (Firmware-Version)	
► TA.SML.CLI.ReadRegisteredValues (Übermittlung historischer, registrierter Werte an das SMGW)	Abruf historischer Messwerte aus der Messeinrichtung	

Tabelle 9.1 Übersicht der TA des SML-Clients

9.3.1 TA.SML.CLI.UpdateMtrCertAndKey

9.3.1.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Implementierung zur Erzeugung einer SML-Auftragsdatei zur Übermittlung eines neuen (vom SMGW erzeugten) Schlüsselpaar/Zertifikats-Tupels an die Messeinrichtung.

Die Auftragsdatei enthält eine Nachricht vom Typ SetProcParameter-Request mit der Konkatenation aus Schlüsselpaar und Zertifikat $MTR_LMN_TLS_CRT_{n+1}$.

9.3.1.2 Anforderungen

Das SMGW **MUSS** das künftig von der Messeinrichtung zur TLS-Authentifizierung gegenüber dem SMGW verwendete $MTR_LMN_TLS_CRT_{n+1}$ und den zugehörigen Privaten Schlüssel innerhalb einer TLS-Verbindung (authentisch und vertraulich) DER-codiert gemäß [X.690] als Octet-String in einer SML-Message "Set-ProcParameter.Req" an die Adresse (Class 1 (Data), 01005E310004, Attribut "value") übermitteln. [REQ.SML.CLI.UpdateMtrCertAndKey.10]¹

9.3.2 TA.SML.CLI.UpdateSmgwCert

9.3.2.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Implementierung zur Erzeugung einer SML-Auftragsdatei zur Aktualisierung des SMGW-TLS-Zertifikates der LMN-Schnittstelle in der Messeinrichtung.

Die Auftragsdatei enthält eine Nachricht vom Typ SetProcParameter-Request mit dem DER-codierten X.509-Zertifikat $GW_LMN_TLS_CRT_{n+1}$, das an eine Konfigurationsadresse der Messeinrichtung übermittelt wird.

9.3.2.2 Anforderungen

Das SMGW **MUSS** das künftige SMGW-Zertifikat $GW_LMN_TLS_CRT_{n+1}$, das von der Messeinrichtung zur TLS-Authentifizierung des SMGW verwendet wird, innerhalb einer TLS-Verbindung (authentisch) DER-codiert gemäß [X.690] als Octet-String in einer SML-Message "SetProcParameter.Req" an die Adresse (Class 1 (Data), 01005E310008, Attribut "value") übermitteln. [REQ.SML.CLI.UpdateSmgwCert.10]

9.3.3 TA.SML.CLI.RequestMeterData

9.3.3.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Implementierung zur Erzeugung einer SML-Auftragsdatei zur Abfrage von Messwerten von einer Messeinrichtung über COSEM-Objekt-Attribute. Die SML-Auftragsdatei enthält pro abzufragendem Messwert (identifiziert über OBIS-ID) eine GetProcParameter.Req-Nachricht, die die Attribute "value" und "status" der Messwertregister abfragen. Für jeden abzufragenden Messwert werden ein oder mehrere COSEM-Objekt-Attribute angefordert (z.B. Wert, Einheit, Messwertstatus).

¹ Der Octet-String enthält gemäß [DIN VDE 0418-63-7] die Konkatenation aus Zertifikat gemäß [RFC5280] mit SubjectPublicKey gemäß [RFC5480] und dem DER-codierten Privaten Schlüssel gemäß [RFC5915].

9.3.3.2 Anforderungen

Das SMGW **MUSS** die von der Messeinrichtung abzufragenden - über OBIS-IDs identifizierten - Register in einer SML-Message "GetProcParameter Req", zusammengefasst in einer SML-Auftragsdatei an die Messeinrichtung übermitteln. [REQ.SML.CLI.RequestMeterData.10]

Die Implementierung **MUSS** die optionalen Datenfelder "serverId", "username", "password" und "attribute" der Datenstruktur SML_GetProcParameter Req leer lassen (als NULL codieren). [REQ.SML.CLI.RequestMeterData.20]

Die Implementierung **MUSS** die von der Messeinrichtung abzufragenden Messwerte und Messgrößen für elektrische Energiezählerstände mit COSEM-Class-ID 32770 und für alle weiteren Zählerstände mit Class-ID Register (3) abfragen können und dabei mindestens die Attribute value (02), scaler_unit (03) und status (04) abfragen. [REQ.SML.CLI.RequestMeterData.30]

9.3.4 TA.SML.CLI.ProcessMeterData

9.3.4.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Implementierung zur Verarbeitung einer SML-Antwortdatei mit Messwerttupeln. Die SML-Antwortdaten enthält pro Messwert (identifiziert über OBIS-ID) eine GetProcParameter.Resp-Nachricht, die die Attribute "value" und "status" der Messwertregister enthält. Für jeden Messwert werden ein oder mehrere COSEM-Objekt-Attribute angegeben (z.B. Wert, Einheit, Messwertstatus).

9.3.4.2 Anforderungen

Identifikation der Messgrößen

Die Identifikation der Messgrößen entspricht denen der gemäß [TR-03109-1v2.0] im anwendungsfallspezifischen Konfigurationsprofil festgelegten Eingangsmessgrößen.

Das SMGW **MUSS** die SML-Tree: Attribut-Indizes value (02), scaler_unit (03), status (04) und capture_time (04) für die COSEM Class-ID Register (3) und 32770 auswerten in die Eingangsmesswerte umsetzen können. [REQ.TLSHDLC.SMGW.80]

Hinweis: Das Attribut capture_time ist vorhanden, sofern die (Kommunikationseinheit der) Messeinrichtung eine absolute Zeit führt und enthält bei Registrierenden Messeinrichtungen den Messzeitpunkt zum Sollregistrierzeitpunkt.

Hinweis: Ist das Status-Attribut nicht vorhanden, liegt keine Störung der Messeinrichtung oder des Messwertes vor.

Hinweis: Gas-Volumenzählerstände, Thermische Energiezählerstände und Wasserzählerstände werden an der LMN-Schnittstelle ohne Information über Bezugs-/Einspeiserichtung erfasst. Sofern es sich um einen Gas-Einspeise-Zähler handelt, wird die entsprechende OBIS-ID (7-0:6.D.0) in der Messwertverarbeitung entsprechend der Konfiguration ersetzt.

Wertebereiche und Konvertierung

Das SMGW **MUSS** in SML-Nachrichten empfangene Messwerte unter Berücksichtigung des Skalierungsfaktors ("scaler") und der Einheit ("unit") ohne Verlust signifikanter Stellen in die interne Repräsentation für originäre Messwerte umsetzen. Dabei soll die Implementierung mindestens mit 13 dezimalen Vorkomma- und 6 Nachkommastellen rechnen können. [REQ.TLSHDLC.SMGW.90]

Bestimmung des meterStatus und valueStatus der Eingangsmesswerte

Das SMGW **MUSS** in SML-Nachrichten empfangene Statusinformationen für die Eingangsmesswerte gemäß ▶ Kapitel 15 aus dem Messwert-Attribut "status" verarbeiten. [REQ.TLSHDLC.SMGW.100]



Anmerkung

Ist $\text{refTime}_n - \text{refTime}_{n-1} < 0$, soll ein Statusbit gesetzt werden, das eine zeitliche Anomalie in der Messwertübertragung kennzeichnet, da der empfangene Sekundenzeitstempel nicht monoton

steigend ist. $\text{"refTime"}_n - \text{"refTime"}_{n-1}$ bezeichnet die Zeitdifferenz der Referenzzeitstempel (Betriebs-Sekundenzeitstempel) zwischen der letzten und vorletzten empfangenen SML-Nachricht.

Zur Verarbeitung weiterer Statusbits wie z.B. Drehfeldrichtung werden keine Mindestanforderungen gestellt. Diese werden nicht als Warnung oder Fehler interpretiert (Status "ok").

Zeitliche Anforderungen

Die in den folgenden Anforderungen verwendeten Zeitbegriffe sind in der Datenstruktur "Eingangsmesswerte" (siehe TR-03109-1) beschrieben. "valTime" bezeichnet den in der SML-Nachricht übermittelten Messzeitpunkt. "refTime" bezeichnet den in der SML-Nachricht übermittelten Versandzeitpunkt.

Das SMGW **MUSS** gewährleisten, dass maximale Verarbeitungszeit der Nachricht zwischen physischem Empfang des ersten Zeichens des HDLC-Frames mit der Messwertnachricht und der Bildung des Empfangszeitstempels im SMGW nicht mehr als 500ms beträgt. [REQ.TLSHDLC.SMGW.50]

Sofern in der Messwertnachricht ein Sekundenzeitstempel (SML "refTime" zum Versandzeitpunkt) und ein Sekundenzeitstempel (SML "valTime", Messzeitpunkt) vorhanden sind, **MUSS** das SMGW den "Messzeitpunkt" der Eingangsmesswerte ("ts") aus dem Empfangszeitpunkt abzüglich der Differenz aus SML "refTime" und SML "valTime" bilden. [REQ.TLSHDLC.SMGW.60]

Sofern der Sekundenzeitstempel (SML "refTime") der Messwertnachricht nicht monoton steigend ist, d.h. $\text{"refTime"}_n - \text{"refTime"}_{n-1} < 0$, **MUSS** das SMGW den Messgerätestatus "meterStatus" gleich "decremented-Timestamp" setzen. $\text{"refTime"}_n - \text{"refTime"}_{n-1}$ bezeichnet die Zeitdifferenz der Referenzzeitstempel (Betriebs-Sekundenzeitstempel) zwischen der letzten und vorletzten empfangenen SML-Nachricht. [REQ.TLSHDLC.SMGW.70]

9.3.5 TA.SML.CLI.UpdateTime

9.3.5.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Implementierung zur Erzeugung einer SML-Auftragsdatei zur Aktualisierung der Uhrzeit der Messeinrichtung mit der Zeit des SMGW, die mit der gesetzlichen Zeit synchronisiert ist.

Das SMGW übermittelt die folgenden Informationen in einer SML-Nachricht:

- Systemzeit des SMGW als Ganzzahl in Sekunden seit Epochenbeginn mit einem Wertebereich, der gewährleistet dass kein Überlauf vor dem 1.1.2100 auftreten kann.
- Kennung des Epochenbeginns der SMGW Systemzeit(z.B. 00:00:00, 1.1.1900: NTP oder 00:00:00, 1.1.1970: Unix)
- Offset zwischen lokaler Zeitzone und UTC in Sekunden.
- DELTA/Offset zur Referenz-Clock (Zeitserver mit gesetzlicher Zeit) in Millisekundenauflösung.
- Vom SMGW gemessene RTT zwischen SMGW und Messeinrichtung in Millisekundenauflösung

Die Messeinrichtung kann die Zeit zur Synchronisation mit der Messgeräteuhr verwenden.

9.3.5.2 Anforderungen



Anmerkung

Entweder die SML-Datenstruktur oder die Struktur der Daten der COSEM-IC(Data:1) und deren OBIS-ID zur Übermittlung der in der Beschreibung genannten Informationen sind noch festzulegen.

9.3.6 TA.SML.CLI.GetClockState (Abfrage des Zustandes der Uhr)

9.3.6.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Implementierung zur Erzeugung einer SML-Auftragsdatei zur Abfrage des Zeitsynchronisationszustandes der Uhr der Messeinrichtung und zur Verarbeitung der SML-Antwortdatei.

Dieser TA übermittelt dem SMGW die folgenden Zustandsinformationen:

- Systemzeit der Messeinrichtung in Sekundenauflösung (Datum, Uhrzeit)
- Von der Messeinrichtung verwendeter Offset zwischen lokaler Zeitzone und UTC in Sekunden.
- Status der Uhr (Valid/Invalid, Noch nicht synchronisiert (RTC), Zeitabweichung > 3%)
- Datum/Uhrzeit der letzten Zeitsynchronisation (sofern von der Messeinrichtung unterstützt)
- Bei der letzten Synchronisation festgestellte Abweichung zwischen Zeit in der Messeinrichtung und Synchronisierter Zeit (sofern von der Messeinrichtung unterstützt)

9.3.6.2 Anforderungen



Anmerkung

Entweder die SML-Datenstruktur oder die Struktur der Daten der COSEM-IC(Clock:1) und deren OBIS-ID zur Übermittlung des Zeitsynchronisationszustandes sind noch festzulegen.

9.3.7 TA.SML.CLI.TransferFileToMeter (Übermittlung einer Datei vom SMGW an die Messeinrichtung)

9.3.7.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Implementierung zur Erzeugung einer SML-Auftragsdatei zur Übermittlung einer (Firmware-)Datei an die Kommunikationseinheit der Messeinrichtung.

Die erste Nachricht vom SMGW an die Messeinrichtung enthält die Dateibezeichnung, den Dateityp, die Dateigröße und ggf. weitere Metainformationen zur Datei.

Die weiteren Nachrichten vom SMGW an die Messeinrichtung enthalten die fortlaufend nummerierten Teile der Datei.

Die Messeinrichtung bestätigt nach jeder Nachricht den Empfang und übermittelt ggf. einen Fehler (SML-Attention).

9.3.7.1.1 Anforderungen



Anmerkung

Die SML-Datenstruktur der ersten und weiteren Nachrichten, sowie die Attention-ID sind noch festzulegen. Der Ablauf ist zu beschreiben.

9.3.8 TA.SML.CLI.ReadFileFromMeter (Übermittlung einer Datei von der Messeinrichtung an das SMGW)

9.3.8.1 Beschreibung

Dieser TA übermittelt eine Datei an das SMGW.

Die erste Nachricht an die Messeinrichtung enthält die angeforderte Dateibezeichnung bzw. den Dateityp. Die Messeinrichtung bestätigt den Transfer oder lehnt ihn mit einer Fehlermeldung ab.

Die weiteren Nachrichten fordern fortlaufend steigend nummerierten Teile der Datei an, die das SMGW an die empfangene Datei anhängt.

Anwendungskontext: Sofern die (KE der) Messeinrichtung eine Auslesung von aufgezeichneten Datensätzen (z.B. Power Quality, Gas-Beschaffenheit, eichtechnisches Logbuch zur Protokollierung von Störungen und Updates) unterstützt, können diese über OBIS-IDs identifizierten Speicherpartitionen/Logs durch das SMGW (ohne Interpretation des Inhaltes) abgerufen und im SMGW zwischengespeichert werden. Gemäß Auswertungsprofil werden die von der Messeinrichtung empfangenen und im SMGW temporär zwischengespeicherten Datensätze mittels INFO-REPORT an den berechtigten EMT übermittelt.

9.3.8.2 Anforderungen



Anmerkung

Die Anfrage wird gemäß [DIN VDE 0418-63-9] Kapitel SML_GetProfilePack.Req und die Antwort gemäß Kapitel SML_GetProfilePack.Resp verarbeitet.

Die SML-Datenstruktur der ersten und weiteren Nachrichten sind noch festzulegen. Der Ablauf ist zu beschreiben.

9.3.9 TA.SML.CLI.ReadDeviceInfo (Abfrage des Gerätetyps und der SW-Version)

9.3.9.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Implementierung zur Erzeugung einer SML-Auftragsdatei zur Abfrage der Geräteinformationen an die (Kommunikationseinheit der) Messeinrichtung und der Verarbeitung der SML-Antwortdatei.

Dieser TA übermittelt Geräteinformationen an das SMGW:

- Firmware-Version der Kommunikationseinheit der Messeinrichtung (Empfohlen: Semantic Versioning gemäß [SemVer])
- Hardware-Typ der Kommunikationseinheit der Messeinrichtung
- Software-Identifikation der Metrologisch relevanten Teile der Software (optional)

Anwendungskontext: Die Informationen können vom SMGW und GWA/MSB insbesondere für den korrekten Firmware-Update verwendet werden.

9.3.9.2 Anforderungen

Die Implementierung fordert die Geräteinformationen der Messeinrichtung gemäß ►Tabelle 9.2 an. Nicht in der Messeinrichtung implementierte Register oder Attention-Fehler führen zu einem leeren Datenfeld.

Bedeutung	Register (OBIS)	Datenfeld
Typ der Messeinrichtung (Gerätekategorie)	01005e310105	IC Class Data (1) "value". Kennzahl 6 Bytes gemäß ►Tabelle 9.3
Geräte-Identifikation	0100600100ff	IC Class Data (1) "value" (Server-ID aus [DIN43849], siehe ►Abbildung 8.3)
Geräte-Firmwareversion (metrologisch, eichrechtlich relevant)	010000020000	IC Class Data (1) "value" UTF-8
Geräte-Firmwareversion (z.B. Kommunikationsmodul)	0100000200NN (NN=01..0F)	IC Class Data (1) "value" UTF-8
Geräte-Hardwareversion (metrologisch, eichrechtlich relevant)	010060320104	IC Class Data (1) "value" UTF-8
Geräte-Hardwareversion (z.B. Kommunikationsmodul)	01006032NN04 (NN=02..0F)	IC Class Data (1) "value" UTF-8

Tabelle 9.2 Übersicht der Register mit Geräteinformationen der Messeinrichtung

Kennzahl	Gerätetyp
8181c7 8260ff .. 8181c7 826fff	FNN-Basiszähler (Varianten SLP/RLM/RLM+Grid, 1/3-Phasig, Steck/3P-Montage)

Tabelle 9.3 Geräteklassen

9.3.10 TA.SML.CLI.ReadRegisteredValues (Übermittlung historischer, registrierter Werte an das SMGW)

9.3.10.1 Beschreibung

Dieser TA übermittelt für einen vorgegebenen historischen Zeitraum (von-bis) und Menge von OBIS-IDs die in der Messeinrichtung gespeicherten Messwertdatensätze an das SMGW.

Die Anfrage an die Messeinrichtung enthält die angeforderte Dateibezeichnung bzw. den Dateityp. Die Messeinrichtung bestätigt den Transfer oder lehnt ihn mit einer Fehlermeldung ab.

Die weiteren Nachrichten fordern fortlaufend steigend nummerierten Teile der Datei an, die das SMGW an die empfangene Datei anhängt.

Anwendungskontext: Von der Messeinrichtung empfangene registrierte Messwerte werden in der - der Messeinrichtung zugeordneten - Registrierten Messwertliste (RWL) des SMGW ergänzt, sofern sie nicht mit gleichen Messzeitpunkt und Identifikation der Messgröße bereits in der RWL vorliegen.

9.3.10.2 Anforderungen



Anmerkung

Die Anfrage wird gemäß [DIN VDE 0418-63-9] Kapitel SML_GetProfilePack.Reg und die Antwort gemäß Kapitel SML_GetProfilePack.Resp verarbeitet.

Die Zeitstempel sind im SML-Format SML_TimestampLocal anzugeben. "timestamp" wird auf UTC bezogen. Das SMGW fasst "localOffset" + "seasonTimeOffset" in einem Local Offset gemäß [RFC3339] zusammen.

9.3.11 TA.SML.CLI.ResetTamperState

9.3.11.1 Beschreibung

Dieser TA beschreibt die Anforderungen an das Zurücksetzen des Manipulationszustandes einer Messeinrichtung.

Es werden alle Manipulationszustände zurückgesetzt, sofern die Messeinrichtung das Schreiben auf die Register unterstützt.

Die Bedeutung der Bits des Zählerstatuswortes ist in ►Kapitel 15 beschrieben.

9.3.11.2 Anforderungen

Das SMGW **MUSS** boolean "true" in einer SML-Message "SetProcParameter.Reg" an die Adresse (Class 1 (Data), 01005E310104, Attribut "value") übermitteln, sofern der Zustand "Magnetische Manipulation" im Zählerstatuswort gesetzt ist. [REQ.SML.CLI.ResetTamperState.10] Damit wird der Zustand der erkannten magnetischen Manipulation zurückgesetzt und die magnetische Manipulationserkennung aktiviert.

Das SMGW **MUSS** boolean "true" in einer SML-Message "SetProcParameter.Reg" an die Adresse (Class 1 (Data), 01005E310109, Attribut "value") übermitteln, sofern der Zustand "Mechanische Manipulation" im Zählerstatuswort gesetzt ist. [REQ.SML.CLI.ResetTamperState.20] Damit wird der Zustand der erkannten mechanischen Manipulation zurückgesetzt und die mechanische Manipulationserkennung aktiviert.

10 Wireless M-Bus Protokoll

10.1 Einleitung

Das Wireless M-Bus-Protokoll beschreibt Kommunikationsverfahren von energie- und speicherbegrenzten Messeinrichtungen mit einem Gateway über eine Funk-Kommunikationsstrecke mit geringer Bandbreite. Die Rollen der Kommunikationspartner werden in der englischen Literatur mit "Meter" bzw. "End-Device" und "Other" bzw. "Gateway" bezeichnet.

Dieses Kapitel beschreibt die Anforderungen an die Implementierung der Protokolle und Nachrichtenformate eines unidirektionalen Empfängers. Es enthält Anforderungen an die Kommunikation zwischen den Kommunikationspartnern "Meter" und "Gateway" und konkretisiert die Anforderungen der DIN EN 13757-Spezifikationen zur Verwendung in intelligenten Messsystemen.

Die Messwerte werden in Nachrichten der Anwendungsschicht übertragen. Diese Nachrichten bestehen aus Datagrammen der Verbindungsschicht.

Ein oder mehrere Messwerte werden in Datensätzen aus Datentyp (Dateninformationsblock, DIB), Messwertidentifikation, Einheit, Skalierungsfaktor (Wertinformationsblock, VIB) und Wert im M-Bus Applikationsformat übermittelt.

Der Absender verschlüsselt jede Nachricht authentisch mit einem gemeinsamen nachrichten- und messeinrichtungsindividuellen, symmetrischen Schlüssel. Der Empfänger prüft die Authentizität der Nachricht und entschlüsselt diese mit dem gemeinsamen Schlüssel.

Die Messgrößen und Messarten der in den Datensätzen codierten Messwerte werden zur Weiterverarbeitung im SMGW über OBIS-IDs identifiziert.

Betriebsart "Compact Profile"

Damit eine registrierperiodensynchrone Erfassung von Messwerten stattfinden kann, muss der Messwert in einem Akzeptanzfenster ($\pm 1\%$ der Registrierperiode) im SMGW vorliegen. Aufgrund der beschränkten Bandbreite des Funkkanals und des notwendigen Duty-Cycles des sendenden Zählers ist bei Registrierperioden im Minutenbereich eine Aussendung pro Messwert nicht mehr möglich und mehrere Messwerte müssen in einer Nachricht zusammengefasst werden. Die für diese Betriebsart verwendete Datenstruktur ist das "Inverse Compact Profile" gemäß [EN13757-3] Annex F. Die Verwendung des "Compact Profile" zur Registrierung von Messwerten im SMGW ist in [OMS-TR07] Kapitel 6 beschrieben. Die Anforderungen an die Messeinrichtung sind in [OMS-S2] Annex R beschrieben.

Sicherheitsprofile

Wireless M-Bus Nachrichten werden authentisch und vertraulich mit den symmetrischen Verfahren gemäß [TR-03109-3] übermittelt.

Das in [OMS-S2] beschriebene "Security Profile B" nutzt den Security Mode 7 gemäß [EN13757-7] (AES-CBC-128 und AES-CMAC-128).

Da künftige Wireless M-Bus Zähler gemäß [OMS-S2] das für batteriebetriebene Zähler energieeffizientere "Security Profile D" verwenden sollen, soll das SMGW als Empfänger dieses Verschlüsselungsverfahrens basierend auf [EN13757-7] Security Mode 10 ergänzend zu Security Mode 7 unterstützen.

Betriebsart und Rahmenformat

Da künftige Wireless M-Bus Zähler gemäß [OMS-S2] die für batteriebetriebene Zähler energieeffizientere Betriebsart "Mode C" gemäß [EN13757-4] verwenden sollen, wird das SMGW als Empfänger sowohl "Mode C" als auch "Mode T" unterstützen. Da ein Funkempfänger (Frequenzband und Modulation) ohne Änderung der physischen Parameter sowohl Mode C als auch Mode T Nachrichten empfangen kann, muss der Empfänger Nachrichten in Mode C und Mode T verarbeiten können.

Die Unterstützung weiterer Betriebsarten durch das SMGW entfällt, da ein SMGW mit einem Empfänger dazu das Frequenzband und die Modulation wechseln müsste und während der Zeit keine Mode T/C Nachrichten empfangen könnte.

Die Annexe I und L der [OMS-S2] sind nicht für SMGWs anwendbar, die nur unidirektionalen Empfang implementieren.

Mitgeltende Standards

Folgende Kapitel der [EN13757-3] sind für die Interoperabilität relevant:

- Abschnitt 6 (M-Bus Protokoll) mit den Einschränkungen dieser Detailspezifikation.
- Anhang E.1 (Fehler-Flag).
- Anhang F (Übertragung von Profilen) ergänzt um [OMS-S2] Annex G. Inverse Compact Profile für Messwertbündel. Die zur Abrechnung verwendeten Zeitstempel der Messwerte müssen im Gateway gebildet werden.
- Anhang H.2 (Übersetzung vordefinierter Datensatztypen), ergänzt um [OMS-S2] Annex A.

Folgende Anforderungen der [EN13757-3] werden nicht gefordert:

- Abschnitt 7 (Anwendungsrücksetzung und Anwendungsauswahl) ist für unidirektional kommunizierende Messeinrichtungen nicht umsetzbar.
- Abschnitt 8 (Synchronisierung der Uhren) ist für unidirektional kommunizierende Messeinrichtungen nicht umsetzbar.
- Abschnitt 9/Anhang D (Alarmprotokoll) wird nicht für die Interoperabilität gefordert, da der Status über das Datenfeld "STS" des TPL-Headers übermittelt wird.
- Unterabschnitt 10.3 (Allgemeine Fehler der Anwendungsschicht) ist für unidirektional kommunizierende Messeinrichtungen nicht umsetzbar.
- Abschnitt 11 (Setzen der Baudrate) ist für Wireless M-Bus Messeinrichtungen nicht umsetzbar.
- Abschnitt 12 (Aktion synchronisieren) ist für unidirektional kommunizierende Messeinrichtungen nicht umsetzbar.
- Abschnitt 13 (Herstellerspezifische Protokolle) werden nicht verarbeitet.
- Abschnitt 14 (Andere Anwendungsprotokolle) werden nicht verarbeitet.
- Abschnitt 15 (Image-Übertragung) ist für unidirektional kommunizierende Messeinrichtungen nicht umsetzbar.
- Anhang C (VIF-Codierung für Sondereinheiten) wird nicht gefordert, bzw. ist für unidirektional kommunizierende Messeinrichtungen nicht umsetzbar.
- Anhang E.2 (Übergabe der Fernsteuerung) ist für unidirektional kommunizierende Messeinrichtungen nicht umsetzbar.
- Anhang E.3 (Synchronisierung der Uhren) ist für unidirektional kommunizierende Messeinrichtungen nicht umsetzbar.
- Anhang G (Kompakter M-Bus-Datenrahmen)
- Anhang H.3 (Online-Ergänzung eines Eintrags in die Umwandlungstabelle zur Übersetzung von M-Bus nach OBIS).
- Anhang I (Image-Übertragung) ist für unidirektional kommunizierende Messeinrichtungen nicht umsetzbar.

10.2 Datenstrukturen

10.2.1 Struktur der Wireless M-Bus Nachrichten

►Abbildung 10.1 stellt die Datenfelder einer wM-Bus Nachricht mit Security Profile B dar.

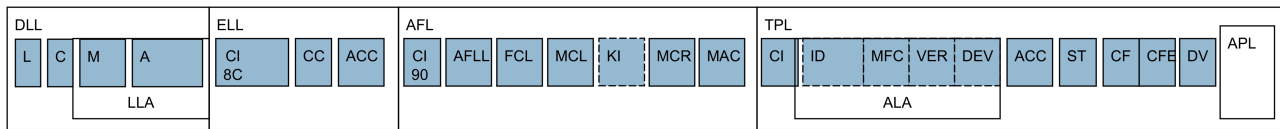


Abbildung 10.1. Übersicht über die Struktur einer Wireless M-Bus Nachricht mit Security Profile B (AFL und Security Mode 7)

Felder in durchgezogenen Linien sind verpflichtend. Felder in gestrichelten Linien sind optional.

►Abbildung 10.2 stellt die Datenfelder einer wM-Bus Nachricht mit Security Profile D dar. Durch den Entfall des AFL und Verzicht auf Block-Cipher im APL ist der Nachrichtenkopf deutlich kürzer.

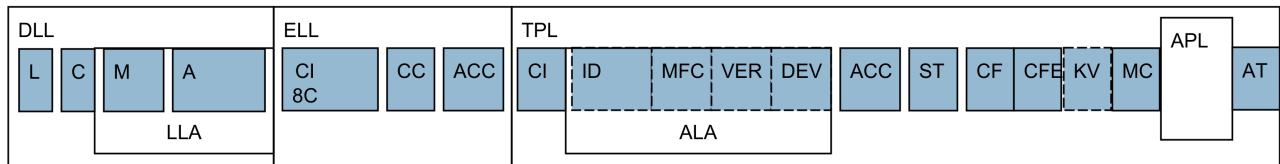


Abbildung 10.2. Übersicht über die Struktur einer Wireless M-Bus Nachricht mit Security Profile D (Security Mode 10)

Erläuterung der Data-Link-Layer (DLL) -Datenfelder:

- L - Längensfeld der Verbindungsschicht.
- C - Nachrichtentyp (Content-Type, z.B. SND-NR).
- M/A - Hersteller/Adresse des Sendegerätes, bilden die Link-Layer-Adresse (LLA) des Absenders.

Erläuterung der Extended-Link-Layer (ELL) -Datenfelder:

- CI - Inhaltstyp des folgenden Headers (Content Information).
- CC - Kommunikationssteuerungsfeld (Communication Configuration).
- ACC - Sequenzzähler für die Verbindungsschicht.
- M2/A2 - Hersteller/Adresse des Empfangsgerätes, bilden die Extended-Link-Layer-Adresse (ELLA) des Empfängers.

Erläuterung der Authentication- and Fragmentation-Layer(AFL) -Datenfelder:

- CI - Inhaltstyp des folgenden Headers (Content Information).
- AFLL - Länge des AFL-Headers.
- FCL - Frame Control.
- MCL - Message Control.
- KI - Schlüsselindex (Key Index).
- MCR - Message Counter.
- MAC - Authentication Tag/Message Authentication Code.
- ML - Message Length.

Erläuterung der Transport-Layer (TPL) -Datenfelder:

- CI - Inhaltstyp des folgenden Headers (Content Information).
- ID, MFC, VER, DEV - Identifikation, Hersteller, Version und Gerätetyp der Applikationsadresse (ALA) des Senders.
- ACC - Sequenznummer der Anwendungsschicht.
- STS - Gerätestatus.
- CF/CFE - Configuration Field (Extension) Verschlüsselungsmodus, Schlüsselauswahl.
- KV - Key Version (optional).

- AT - Authentication Tag (Security Mode 10).
- MC - Message Counter (Security Mode 10).
- DV - Decryption Verification (Security Mode 7)

Der Application-Layer (APL) wird im nächsten Abschnitt näher erläutert.

10.2.2 Struktur der Wireless M-Bus Application-Layer-Nachricht

►Abbildung 10.3 stellt die in dieser Detailspezifikation referenzierten Datenfelder der M-Bus Applikationsdaten dar. Die Applikationsdaten enthalten die Messwerte.

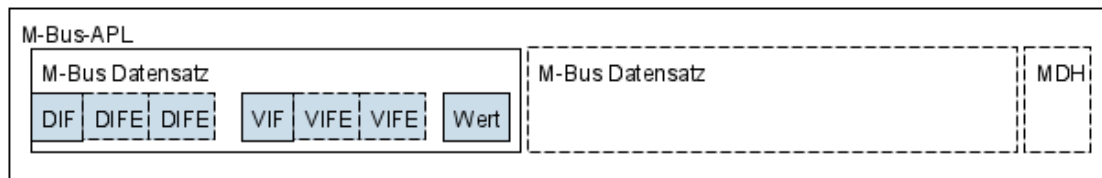


Abbildung 10.3. Übersicht über die M-Bus APL Struktur

Felder in durchgezogenen Linien sind verpflichtend, Felder in gestrichelten Linien sind optional.

Erläuterung der Datenfelder:

- DIF/DIFE - Data Information Field(-Extension) enthält Angaben zu Datentyp und Länge des Datentyps aber auch Informationen zu Tarifstufe oder Speicherindex.
- VIF/VIFE - Value Information Field(-Extension) enthält Angaben zu Messwerttyp, Messart, Einheit und Skalierungsfaktor.
- MDH - optionale herstellerspezifische Daten am Ende des normativ strukturierten Bereichs

10.3 Anforderungen an den Funkempfänger

Das SMGW **MUSS** die physische Medienzugangsschicht für drahtlose Kommunikation nach [EN13757-4] als Short Range Device Funkempfänger mit Mode T umsetzen. [REQ.WMBUS.Unidirektional.30]

Das SMGW **MUSS** die physische Medienzugangsschicht für drahtlose Kommunikation nach [EN13757-4] mit Mode C mit Rahmenformat A als Funkempfänger unterstützen. [REQ.WMBUS.Unidirektional.36]

Das SMGW **MUSS** den Data Link Layer und Extended Link Layer nach den Anforderungen der Spezifikation nach [EN13757-4] und nach [OMS-S2] Abschnitte 5.2 und 5.3 in der Rolle des Gateway umsetzen, um den Transport der Nachrichten und die Integrität der Verbindungen zu gewährleisten. [REQ.WMBUS.Unidirektional.40]

Das SMGW **MUSS** die Authentifizierung und Entschlüsselung von Nachrichten nach den Anforderungen der Spezifikation [EN13757-7] Security Mode 7 für das SMGW in der Rolle des Recipients ("Gateway") umsetzen ([OMS-S2] Security Profile "B"). [REQ.WMBUS.Unidirektional.50] Für die Authentifizierung und Entschlüsselung wird der Symmetrische Kommunikationsschlüssel aus dem LMN-Kommunikationsprofil verwendet, der der in der Nachricht identifizierten Messeinrichtung zugeordnet ist.

Das SMGW **SOLL** die Authentifizierung und Entschlüsselung von Nachrichten nach den Anforderungen der Spezifikation [EN13757-7] Security Mode 10 für das SMGW in der Rolle des Recipients ("Gateway") umsetzen ([OMS-S2] Security Profile "D"). [REQ.WMBUS.Unidirektional.51] Für die Authentifizierung und Entschlüsselung wird der Symmetrische Kommunikationsschlüssel aus dem LMN-Kommunikationsprofil verwendet, der der in der Nachricht identifizierten Messeinrichtung zugeordnet ist.(►ICS.WMBUS.Mode10.10)

Das SMGW **MUSS** empfangene Nachrichten verwerfen, deren Message Counter (Datenfeld AFL.MCR oder Datenfeld TPL.MC) gegenüber der letzten empfangenen Nachricht nicht monoton steigend ist. [REQ.WMBUS.Unidirektional.60]

Das SMGW **SOLL** für Security Mode 10 die CCM-Nonce und das Authentication Tag (AT) gemäß [EN13757-7] berechnen (►ICS.WMBUS.Mode10.10). [REQ.WMBUS.Unidirektional.71]

Das SMGW **SOLL** für Security Mode 10 die Authentication Tag Length 8 Byte, 12 Byte und 16 Byte unterstützen (►ICS.WMBUS.Mode10.10). [REQ.WMBUS.Unidirektional.72]

Der AES-CCM-Parameter Associated Data A ist gemäß [EN13757-7] die Konkatenation aus (TPL.CI || TPL.ACC || TPL.ST || TPL.CF || { TPL.CFE } || { TPL.KV } || Unencrypted APL), wobei nicht vorhandene, optionale Bytes in {} entfallen.

Die 13 Byte AES-CCM-Nonce wird gemäß [EN13757-7] wie in ►Tabelle 10.1 berechnet. Sofern ALA im TPL vorhanden ist, stammen Manufacturer (MFC), Identification Number (ID), Version, DeviceType aus den entsprechenden Feldern im TPL, anderenfalls aus den Feldern Manufacturer (M) und Address (A) im DLL.

12	11	10	9	8	7	6	5	4	3	2	1	0
TPL.MFC (LSB First)		TPL.ID (LSB First)				Version	Device Type	00h	TPL.MC (MSB First)			
DLL.M (LSB First)		DLL.A (LSB First)						00h	TPL.MC (MSB First)			

Tabelle 10.1 Berechnung der CCM Nonce



ICS.WMBUS.Mode10.10

Sofern Security Mode 10 unterstützt wird, **MUSS** der Hersteller im ICS angeben, welche AES-CCM Authentication tag length das SMGW verarbeiten kann.

10.4 TA.WMBUS.ProcessMeterData

10.4.1 Anforderungen an die Verarbeitung

Das SMGW **MUSS** die empfangene Anwendungsnachricht gemäß [EN13757-3] Abschnitt 6 interpretieren. [REQ.WMBUS.apl.20]

Falls dem M-Bus-Datenblock herstellerspezifische Daten folgen, **DARF** die Implementierung diese Daten **NICHT** für die abrechnungsrelevante Messwertverarbeitung heranziehen. [REQ.WMBUS.apl.30]

10.4.2 Zeitliche Anforderungen

Die in den folgenden Anforderungen verwendeten Zeitbegriffe sind in der Datenstruktur "Eingangsmesswerte" (siehe TR-03109-1) beschrieben. Die "Actuality Duration" gibt die Differenz zwischen Sendezeitpunkt und Messzeitpunkt an.

Das SMGW **MUSS** beim Empfang von Nachrichten im Format "Inverse Compact Profile" nach [EN13757-3] Anhang F den für die Abrechnung verwendbaren Messwert und Zeitstempel zum Sollregistrierzeitpunkt gemäß [OMS-TR07] Kapitel 6 unter Berücksichtigung von "Spacing Control Byte" und "Spacing Value Byte" berechnen (Sollregistrierzeitpunkt mit UTC-Bezug). [REQ.WMBUS.apl.10]

Das SMGW **MUSS** beim Empfang von Nachrichten im Format "Inverse Compact Profile" nach [EN13757-3] Anhang F den - nicht für die Abrechnung verwendbaren - aktuellen Messwert und den Zeitstempel "Messzeitpunkt" (measurementTs) aus dem "Base-Eintrag" des Compact Profiles abzüglich der in der Nachricht übermittelten "Actuality Duration" ([OMS-S2] Annex B "DP1!") ermitteln (Messzeitpunkt mit UTC-Bezug). [REQ.WMBUS.apl.11]

Das SMGW **MUSS** beim Empfang von Nachrichten ohne "Compact Profile", mit MB-Tag "DP1!" und MB-Tag "DT1!" den "Messzeitpunkt" (measurementTs) als Empfangszeitpunkt der Nachricht im SMGW abzüglich der in der Nachricht übermittelten "Actuality Duration" ([OMS-S2] Annex B "DP1!") ermitteln (Messzeitpunkt mit UTC-Bezug). [REQ.WMBUS.apl.12]

Das SMGW **KANN** beim Empfang von Nachrichten ohne "Compact Profile", mit MB-Tag "DT1!" den "Messzeitpunkt" (measurementTs) aus Datum/Zeit des MB-Tags([OMS-S2] Annex B "DT1!") ermitteln (Messzeitpunkt in "Local Time" der Messeinrichtung). [REQ.WMBUS.apl.13] Dieser Zeitstempel wird ergänzend zum Empfangszeitpunkt erfasst.

Das SMGW **MUSS** gewährleisten, dass maximale Verarbeitungszeit der Nachricht zwischen physischem Empfang des ersten Zeichens der Nachricht und Bildung des Empfangs-Zeitstempels im SMGW nicht mehr als 500ms beträgt. [REQ.WMBUS.apl.40]

10.4.3 Identifikation der Messgrößen

Das SMGW **MUSS** die MB-Tags (VIF/VIFE/DIF/DIFE) für Elektrizität, Gas, Wasser, Wärme/Kälte, die mit "M" oder "An" gekennzeichnet sind, gemäß [OMS-S2] Annex A in OBIS-IDs für die Eingangsmesswerte umsetzen. [REQ.WMBUS.apl.50]

Das SMGW **SOLL** die MB-Tags (VIF/VIFE/DIF/DIFE) für Elektrizität, Gas, Wasser, Wärme/Kälte, die mit "O" gekennzeichnet sind gemäß [OMS-S2] Annex A in OBIS-IDs für die Eingangsmesswerte umsetzen (Bedingung: ▶ICS.WMBUS.apl.10). [REQ.WMBUS.apl.60]

Implementation Conformance Statemets



ICS.WMBUS.apl.10

Der Hersteller **MUSS** im ICS erklären, ob die Implementierung über die mit "M", "An" gekennzeichneten MB-Tags hinaus weitere MB-Tags in OBIS-IDs umsetzt und beschreiben, welche dieser MB-Tags in welche OBIS-IDs umgesetzt werden.

10.4.4 Wertebereiche und Konvertierung

Das SMGW **MUSS** die Werte der unterstützten MB-Tags gemäß [OMS-S2] Annex B ohne Verlust signifikanter Stellen in die interne Repräsentation für originäre Messwerte umsetzen. Dabei soll die Implementierung mindestens mit 13 dezimalen Vorkomma- und 6 Nachkommastellen rechnen können. [REQ.WMBUS.apl.70]

10.4.5 Status Eingangsmesswerte

Das SMGW **MUSS** das 32-Bit Statuswort für die Eingangsmesswerte an das SMGW gemäß ▶Kapitel 15 aus STS und EF setzen. [REQ.WMBUS.apl.80]

EF1-EF3: (Extended) Fehler Flags gemäß [EN13757-3] Annex A Datapoints MM02 und MM03.

STS: Gerätestatusbyte des TPL

11 SOCKS-TLS Protokoll

11.1 Einleitung

Diese Detailspezifikation beschreibt die Verwendung von SOCKSv5 und TLS gemäß [RFC1928] und [DRAFT-IETF-AFT-SOCKS-SSL-00] als Steuerungsprotokoll zum Verbindungsaufbau eines TLS-Proxy-Kanals vom SOCKS-Client zum TLS-Proxy als SOCKS-Server, um zu signalisieren, zu welchem Kommunikationspartner im WAN der TLS-Proxy eine TLS-Verbindung aufbauen soll.

Beim Aufbau der TLS-Verbindung zwischen Kommunikationsadapter(CLS) und SMGW wird im SOCKS-TLS-Handshake mittels der TLS-Client- und TLS-Server-Zertifikate und der zugehörigen Schlüssel eine Client-Server Authentifizierung durchgeführt. Im SOCKS-Protokoll wird als Zieladresse ein eindeutiger Bezeichner für die TLS-Proxy-Funktion im SMGW übermittelt, der die Zieladresse und Identität des Kommunikationspartners im WAN über die Konfiguration der TLS-Proxy-Funktion im SMGW eindeutig identifiziert. Die TLS-Proxy-Funktion im SMGW initiiert bei berechtigter Proxy-Verbindung eine TLS-Verbindung zur konfigurierten Zieladresse auf.

Für das SMGW ist das Kapitel ▶Abschnitt 11.3 relevant.

Für CLS-Kommunikationsadapter ist das Kapitel ▶Abschnitt 11.2 relevant.

11.2 Technische Anwendungsfälle des SOCKS-TLS Clients

11.2.1 TA.SOCKSTLS.InitiateAuthNegotiation

11.2.1.1 Beschreibung

Dieser TA beschreibt die Anforderungen für die Initiierung eines SOCKS-Verbindungsaufbaus zu einem SOCKS-Server gemäß [RFC1928] für TLS-Authentifizierung nach [DRAFT-IETF-AFT-SOCKS-SSL-00].

11.2.1.2 Anforderungen

- Die Implementierung **MUSS** den Verbindungsaufbau zum SOCKS-Server für TCP-basierte Clients gemäß [RFC1928] Kap. 3 für SOCKSv5 (Ver = 0x05) durchführen. [REQ.TA.SOCKSTLS.InitiateAuthNegotiation.10]
- Die Implementierung **MUSS** dem SOCKS-Server die Authentifizierungsmethode "TLS" (0x06) anbieten und akzeptieren und den Ablauf mit Fehler abbrechen, falls der SOCKS-Server mit einer anderen Methode antwortet. [REQ.TA.SOCKSTLS.InitiateAuthNegotiation.20]

Method-Id=0x06 gemäß IANA Assignments Socks-Methods ([DRAFT-IETF-AFT-SOCKS-SSL-00] verwendet Id=0x86).

- Die Implementierung **DARF KEINE** Subauthentication für die Authentifizierungsmethode "TLS" durchführen. [REQ.TA.SOCKSTLS.InitiateAuthNegotiation.30]

11.2.2 TA.SOCKSTLS.ConnectRequest

11.2.2.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Initiierung eines Verbindungsaufbaues mit einem Kommunikationspartner im WAN über eine bestehende Verbindung zu einem SOCKS-Server.

Die Identifikation des Proxy-Kommunikationsprofiles im SMGW geschieht über eine IP-Adresse oder einen Server-Namen in der CONNECT-Nachricht.

11.2.2.2 Anforderungen

- Die Implementierung **MUSS** für jede weitere Proxy-Verbindung zu einem Kommunikationspartner im WAN eine neue TLS-Session zum SOCKS-Server aufbauen. [REQ.TA.SOCKSTLS.ConnectRequest.10]
- Die Implementierung **MUSS** SOCKSv5 und das Kommando CONNECT (0x01) gemäß [RFC1928] Kap. 4 verwenden. [REQ.TA.SOCKSTLS.ConnectRequest.20]

- Die Implementierung **MUSS** das CONNECT Kommando mit den Adresstyp "DomainName" (0x03) verwenden können, wobei der Parameter DomainName nur die Zeichen 0-9, a-z, - und Punkt enthalten darf. [REQ.TA.SOCKSTLS.ConnectRequest.30]
- Die Implementierung **SOLL** das CONNECT Kommando mit den Adresstypen IPv4 (ATYP 0x01) und IPv6 (ATYP 0x04) verwenden können. (►ICS.TA.SOCKSTLS.ConnectRequest.10) [REQ.TA.SOCKSTLS.ConnectRequest.35]
- Die Implementierung **MUSS** das CONNECT Kommando innerhalb der bestehenden SOCKS-TLS-Verbindung und mit dem SOCKS-Framing State "Data Flow" (0x03) übermitteln. [REQ.TA.SOCKSTLS.ConnectRequest.40]
- Die Implementierung **MUSS** die Replies gemäß [RFC1928] Kap. 6 auswerten, Werte des Datenfelds "REP" ungleich 0 als Fehler interpretieren und den Ablauf mit Fehler abbrechen. [REQ.TA.SOCKSTLS.ConnectRequest.50]
- Die Implementierung **MUSS** die Datenfelder BND.ADDR und BND.PORT der CONNECT-Reply-Nachricht ignorieren, da diese undefiniert sind. [REQ.TA.SOCKSTLS.ConnectRequest.60]
- Die Implementierung **MUSS** nach einem Verbindungsfehler (REP > 0) innerhalb von 10s die TLS-Verbindung trennen. [REQ.TA.SOCKSTLS.ConnectRequest.70]
- Die Implementierung **MUSS** einen neuen Verbindungsaufbau nach einem Verbindungsfehler (REP > 0) mindestens 5 Sekunden bis maximal 60 Sekunden randomisiert verzögern, um eine Überlastung der TLS-Proxy-Funktion zu vermeiden. [REQ.TA.SOCKSTLS.ConnectRequest.80]



ICS.TA.SOCKSTLS.ConnectRequest.10

Der Hersteller **MUSS** im ICS deklarieren, welche Adresstypen die Implementierung im CONNECT Kommando verwendet.

11.3 Technische Anwendungsfälle des SOCKS-TLS Servers

11.3.1 TA.SOCKSTLS.NegotiateAuth

11.3.1.1 Beschreibung

Dieser TA beschreibt die Anforderungen für die Vereinbarung der Authentifizierung bei einem SOCKS-Verbindungsaufbau von einem SOCKS-Client zu einem SOCKS-Server gemäß [RFC1928] für TLS-Authentifizierung nach [DRAFT-IETF-AFT-SOCKS-SSL-00].

11.3.1.2 Anforderungen

- Die Implementierung **MUSS** die Vereinbarung der Authentifizierung für TCP-basierte Server gemäß [RFC1928] Kap. 3 für SOCKSv5 (Ver = 0x05) durchführen. [REQ.TA.SOCKSTLS.NegotiateAuth.10]
- Die Implementierung **MUSS** die Authentifizierungsmethode "TLS" (0x06) akzeptieren und den Ablauf mit Fehler abbrechen, falls eine andere Methode vorgeschlagen wird. [REQ.TA.SOCKSTLS.NegotiateAuth.20]
- Die Implementierung **DARF KEINE** Subauthentication für die Authentifizierungsmethode "TLS" durchführen. [REQ.TA.SOCKSTLS.NegotiateAuth.30]

11.3.2 TA.SOCKSTLS.Connect

11.3.2.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Annahme einer SOCKS-Verbindungsaufbauanfrage.

Die Identifikation des Proxy-Kommunikationsprofiles im SMGW geschieht über eine Identifikation gemäß ATYP in der CONNECT-Nachricht.

11.3.2.2 Anforderungen

- Die Implementierung **MUSS** SOCKSv5 und das Kommando CONNECT (0x01) gemäß [RFC1928] Kap. 4 akzeptieren. [REQ.TA.SOCKSTLS.Connect.20]

- Die Implementierung **MUSS** das CONNECT Kommando mit dem Adresstyp "DomainName" (0x03) akzeptieren und zur Auswahl des Proxy-Kommunikationsprofils bei Übereinstimmung mit dem Datenfeld "Server-Name" ("localServerName") verwenden. [REQ.TA.SOCKSTLS.Connect.30]
- Die Implementierung **MUSS** die CONNECT-Anfrage ablehnen, wenn der DomainName nicht zwischen 1-63 Zeichen enthält oder Zeichen ungleich a-z, 0-9, Punkt, "-" enthält. [REQ.TA.SOCKSTLS.Connect.35]
- Die Implementierung **SOLL** die Adresstypen IPv4 (ATYP 0x01) und IPv6 (ATYP 0x04) zur Identifikation des WAN- und Proxy-Kommunikationsprofils anhand der WAN-Kommunikationsadresse unterstützen. (►ICS.TA.SOCKSTLS.Connect.10) [REQ.TA.SOCKSTLS.Connect.36]
- Die Implementierung **MUSS** ausschließlich das CONNECT Kommando innerhalb der bestehenden SOCKS-TLS-Verbindung und mit dem SOCKS-Framing State "Data Flow" (0x03) verarbeiten. [REQ.TA.SOCKSTLS.Connect.40]
- Die Implementierung **MUSS** ausschließlich Fehlercodes gemäß [RFC1928] Kap. 6 im Datenfeld "REP" übermitteln und den Ablauf mit Fehler abbrechen. Die Bedeutung der Fehlercodes im Kontext dieser Technischen Richtlinie wird in ►Tabelle 11.1 beschrieben. [REQ.TA.SOCKSTLS.Connect.50]
- Die Implementierung **SOLL** die Datenfelder BND.ADDR = 0 und BND.PORT = 0 im Reply senden. [REQ.TA.SOCKSTLS.Connect.60]
- Die Implementierung **MUSS** bei einem Fehler des TLS/SOCKS-Verbindungsaufbaues einen SOCKS-Reply mit REP > 0 übermitteln und die TLS-Verbindung innerhalb von 10s beenden. [REQ.TA.SOCKSTLS.Connect.70]
- Die Implementierung **MUSS** die CONNECT-Anfrage ablehnen, wenn kein Proxy-Kommunikationsprofil für den im CONNECT gemäß Adresstyp angefragten DomainNamen (localServerName) oder IP-Adresse und Port (wanPeerAddresses) konfiguriert ist. [REQ.TA.SOCKSTLS.Connect.80]

Reply (REP)	Bedeutung
0x00	Verbindungsaufbau zum SOCKS-Server erfolgreich (SMGW).
0x02	Proxy-Kommunikationsprofil für den gemäß ATYP angegebenen Destination-Domain-Name, IPv4- oder IPv6-Adresse ist nicht konfiguriert.
0x04	Nach Ablauf der Verbindungsaufbau-Wartezeiten war eine Verbindung zum WAN-Teilnehmer nicht möglich (Timeout).
0x05	Der Verbindungsaufbau zum WAN-Teilnehmer wurde abgelehnt.

Tabelle 11.1 Response-Codes für SOCKS-Verbindungen

**ICS.TA.SOCKSTLS.Connect.10**

Der Hersteller **MUSS** im ICS deklarieren, welche Adresstypen die Implementierung im CONNECT Kommando zur Auswahl eines Proxy-Kommunikationsprofils verwendet.

11.4 Technische Anwendungsfälle für SOCKS-TLS Client und Server

11.4.1 TA.SOCKSTLS.SendApplicationData

11.4.1.1 Beschreibung

Dieser TA enthält die Anforderungen an den Versand von Anwendungsdaten über eine bestehende SOCKS-TLS-Verbindung.

11.4.1.2 Anforderungen

**REQ.TA.SOCKSTLS.SendApplicationData.10**

Die Implementierung **MUSS** Anwendungsdaten innerhalb eines TLS-Application-Records senden, die in einem Rahmenformat gemäß [DRAFT-IETF-AFT-SOCKS-SSL-00] gekapselt sind und mit State="0x03" (Data Flow) übermittelt werden.

11.4.2 TA.SOCKSTLS.ReceiveApplicationData

11.4.2.1 Beschreibung

Dieser TA enthält die Anforderungen an den Empfang von Anwendungsdaten über eine bestehende SOCKS-TLS-Verbindung.

11.4.2.2 Anforderungen



REQ.TA.SOCKSTLS.ReceiveApplicationData.10

Die Implementierung **MUSS** die TLS-Application-Records, die in einem Rahmenformat gemäß [DRAFT-IETF-AFT-SOCKS-SSL-00] gekapselt sind und mit State="0x03" (Data Flow) übermittelt werden, verarbeiten.

11.4.3 TA.SOCKSTLS.SendSocksTlsRecord

11.4.3.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Implementierung zum Versand eines SOCKS-TLS-Records über ein oder mehrere TCP-Segmente. Die Implementierung bestimmt zuvor die Maximum Transmission Unit (MTU) zur Zerlegung von SOCKS-TLS-Records in TCP-Segmente.

11.4.3.2 Anforderungen



REQ.TA.SOCKSTLS.SendSocksTlsRecord.10

Die Implementierung **MUSS** den SOCKS-TLS-Frame gemäß [DRAFT-IETF-AFT-SOCKS-SSL-00] mit "VER" = 0x01 und "STATE" = (0x01-0x04) senden.

11.4.4 TA.SOCKSTLS.ReceiveSocksTlsRecord

11.4.4.1 Beschreibung

Dieser TA beschreibt die Anforderungen an die Implementierung zum Empfang und Assemblieren eines SOCKS-TLS-Records aus ein oder mehreren TCP-Segmenten. Der TA wartet bis ein vollständiger SOCKS-TLS-Record eingetroffen ist.

11.4.4.2 Anforderungen

- Die Implementierung **MUSS** den SOCKS-TLS-Frame gemäß [DRAFT-IETF-AFT-SOCKS-SSL-00] mit "VER" = 0x01 und "STATE" = (0x01-0x04) verarbeiten können. [REQ.TA.SOCKSTLS.ReceiveSocksTlsRecord.10]

11.4.5 TA.SOCKSTLS.ClosingHandshake

11.4.5.1 Beschreibung

Dieser TA beschreibt die Anforderungen an das geordnete Beenden einer SOCKS-TLS-Verbindung durch den SOCKS-TLS-Client oder SOCKS-TLS-Server, nachdem die SOCKS-TLS-Verbindung mit einem CONNECT erfolgreich hergestellt wurde. Sofern die Verbindung durch einen Fehler der TLS-Protokollschicht beendet wird, wird dieser im TLS-Alert übermittelt. Sofern der SOCKS-CONNECT durch einen Fehler beendet wird, wird dieser im REP Datenfeld der CONNECT-Response übermittelt und die TLS-Verbindung beendet. Die Übermittlung des Fehlergrunds nach dem Verbindungsaufbau ist im SOCKS-Protokoll nicht vorgesehen. Der Verbindungsabbau wird durch einen TLS-Alert im SOCKS-TLS-Framing durchgeführt. Anschließend beenden Client und Server die TCP-Verbindung.

Der Ablauf wird initiiert durch den SOCKS-TLS-Client oder SOCKS-TLS-Server, der die Verbindung beenden möchte. Sofern die Verbindung durch einen Fehler der TLS-Protokollschicht beendet wird, wird dieser im TLS-Alert übermittelt. Die Übermittlung des Fehlergrunds im SOCKS-Protokoll ist nicht vorgesehen.

11.4.5.2 Anforderungen

- Die Implementierung **MUSS** die TLS-Alert Records mit SOCKS-TLS-Framing gemäß ▶TA.SOCKSTLS.ReceiveSocksTlsRecord mit "expectedState" = "Closing Handshake" (0x04) empfangen und ▶TA.SOCKSTLS.ReceiveSocksTlsRecord mit "STATE" = "Closing Handshake" (0x04) senden. [REQ.TA.SOCKSTLS.ClosingHandshake.10]

Ein TLS "close notify" Alert zum Schließen der TLS-Verbindung wird eingebettet in ein SOCKS-Framing im State "Closing Handshake" übermittelt.

- Die Implementierung **MUSS** auf einen empfangenen SOCKS-TLS-"Closing Handshake" mit einem SOCKS-TLS-"Closing Handshake" antworten und die zugehörige Transportverbindung innerhalb von 5 Sekunden beenden. [REQ.TA.SOCKSTLS.ClosingHandshake.20]

12 mDNS - Device/Service Discovery Protokoll

12.1 Einleitung

Dieses Kapitel beschreibt Anforderungen an die automatische, dynamische Adresskonfiguration, Device- und Service Discovery mit dem Multicast Domain Name Service Protokoll (mDNS) und Domain Name Service Service Discovery (SD) Protokoll. Multicast DNS und DNS-SD werden verwendet, um eine automatische Konfiguration von Geräten und Diensten in einem Netzwerk zu ermöglichen. Zusammen mit einer automatischen IP-Adressvergabe wird dieses Konzept "Zeroconf" genannt. Diese Spezifikation enthält Anforderungen an die Implementierung des mDNS/DNS-SD Responders und Queriers (siehe [RFC6762] Kapitel 5 und 6 und [RFC6763]).

Der mDNS Responder beantwortet DNS-Adressanfragen nach <SMGW-ID>.local." (unique resource) und "smgw.local." (shared resource) mit der link-lokalen Netzwerkadresse des SMGW.¹

mDNS verwendet die Datenstrukturen des DNS-Protokolls und kommuniziert über Port 5353 mittels UDP-Protokoll und Multicast-IPv4 und/oder IPv6.

mDNS verwendet DNS-A-Records zum Mitteilen von IPv4-Adressen und DNS-AAAA-Records zum Mitteilen von IPv6-Adressen der Geräteschnittstelle im (lokalen) Netzwerk und kann IP-Adressen über DNS-PTR-Records zu Hostnamen auflösen.

DNS-SD verwendet DNS-SRV-Records zur Beschreibung der Dienstzugriffsadressen (z.B. Port-Nummer) und DNS-TXT-Records zur Übermittlung von detaillierteren, kontextabhängigen Dienstzugangsinformationen eines Gerätes im Netzwerk.

mDNS/DNS-SD bietet keinen Schutz vor manipulierten, verzögerten oder ausgespähten mDNS/DNS-SD-Nachrichten. Um einen gewissen Schutz zu erreichen, darf das Protokoll deshalb nur link-lokal und in physisch oder kryptografisch geschützten Netzwerken innerhalb von Liegenschaften verwendet werden.² Eine Betrachtung von Angreifer-Modellen zur Privacy von DNS-SD findet sich in [RFC8882].

12.2 Technische Anwendungsfälle des mDNS-Responders

Die folgenden Abschnitte enthalten die TA für die Implementierung eines mDNS-Responders.

12.2.1 TA.mDNS.Responder

12.2.1.1 Beschreibung

Dieser TA veranlasst die Aussendung von Multicast-Benachrichtigungen an die Netzwerkteilnehmer um den bzw. die eindeutigen ("Unique") und generischen ("Shared") Hostnamen in einem lokalen Netzwerk bekannt zu machen. Sofern der eindeutige Hostname im Netzwerk bereits vergeben wurde, wird dieser TA mit Fehler beendet.

12.2.1.2 Anforderungen



REQ.TA.mDNS.Responder.10

Die Implementierung **MUSS** die Anforderungen (MUST, SHOULD, MUST NOT, SHOULD NOT) an einen mDNS Responder nach [RFC6762] erfüllen. Dazu gehören insbesondere die Anforderungen in Kapitel 6 (Responding) und Kapitel 8 (Probe and Announce).

¹ Der abschließende Punkt im Fully-Qualified Domain-Namen identifiziert die DNS-Root. Im DNS-Leitungsprotokoll wird dazu ein abschließendes 0x00 codiert. In der Regel wird dieser Punkt in der Darstellung für Nutzerschnittstellen weggelassen.

² Die Authentizität und Vertraulichkeit der Anwendungsdaten der mittels mDNS/DNS-SD bekanntgegebenen Dienste wird durch das TLS-Protokoll gewährleistet.

**REQ.TA.mDNS.Responder.20**

Die Implementierung **MUSS** die Anforderungen an das Erzeugen von mDNS-Responses nach ▶Abschnitt 12.2.2 erfüllen.

**REQ.TA.mDNS.Responder.30**

Die Implementierung **MUSS** die Anforderungen an das Verarbeiten von mDNS-Queries nach ▶Abschnitt 12.2.3 erfüllen.

**REQ.TA.mDNS.Responder.40**

Die Implementierung **MUSS** die Queries nach *uniqueNames[]*, *sharedNames[]* mit den link-lokalen IP-Adressen der Schnittstelle beantworten, über die die Anfrage empfangen wurde.

**REQ.TA.mDNS.Responder.50**

Die Implementierung **SOLL** die Anforderungen (MUST, SHOULD, MUST NOT, SHOULD NOT) an DND-SD nach [RFC6763] erfüllen.

12.2.2 TA.mDNS.BuildResponse

12.2.2.1 Beschreibung

Dieser TA beschreibt die Anforderungen an das Erzeugen einer mDNS Responder Antwort.

12.2.2.2 Anforderungen

Die Implementierung **SOLL** gewährleisten, dass sich die SRV- und TXT-Records der Antworten auf über IPv4 bzw. IPv6 empfangene Anfragen nicht unterscheiden. [REQ.TA.mDNS.BuildResponse.20] Die A- und AAAA-Records unterscheiden sich.

Die Implementierung **MUSS** Multicast mDNS-Nachrichtenaussendungen auf 30 pro Minute begrenzen. [REQ.TA.mDNS.BuildResponse.30]

12.2.3 TA.mDNS.ProcessQuery

12.2.3.1 Beschreibung

Dieser TA beschreibt die Anforderungen an das Verarbeiten von mDNS-Anfragen.

12.2.3.2 Anforderungen

- Die Implementierung **MUSS** die Anfrage *queryRequest* nach den Anforderungen von [RFC6762] und [RFC6763] verarbeiten und im Fehlerfall den Ablauf mit Fehler beenden. Eine Antwort wird nach den Anforderungen von ▶Abschnitt 12.2.2 erzeugt. [REQ.TA.mDNS.ProcessQuery.10]
- Die Implementierung **SOLL** über IPv6 empfangene mDNS-Anfragen verarbeiten und beantworten können. [REQ.TA.mDNS.ProcessQuery.20]

12.3 Technische Anwendungsfälle für das mDNS Querying

Die folgenden Abschnitte enthalten die TA für die Implementierung des mDNS Querying zur Ermittlung von IP-Adressen und Service-Namen eines IP-Hosts im lokalen Netzwerk.

12.3.1 TA.mDNS.Querying

12.3.1.1 Beschreibung

Dieser TA veranlasst die Aussendung einer Multicast Abfrage an einen Host, der über einen eindeutigen ("Unique") oder generischen ("Shared") Hostnamen in einem lokalen Netzwerk identifiziert wird.

12.3.1.2 Anforderungen



REQ.TA.mDNS.Querying.10

Die Implementierung **MUSS** die Anforderungen (MUST, SHOULD, MUST NOT, SHOULD NOT) an das mDNS Querying gemäß [RFC6762] erfüllen. Dazu gehören insbesondere die Anforderungen in Kapitel 6 (Responding) und Kapitel 8 (Probe and Announce).



REQ.TA.mDNS.Querying.20

Die Implementierung **MUSS** die Anforderungen an das Erzeugen von mDNS-Responses nach ▶ Abschnitt 12.2.2 erfüllen.



REQ.TA.mDNS.Querying.30

Die Implementierung **MUSS** die Anforderungen an das Verarbeiten von mDNS-Queries nach ▶ Abschnitt 12.2.3 erfüllen.



REQ.TA.mDNS.Querying.40

Die Implementierung **MUSS** die Queries nach *uniqueNames[]*, *sharedNames[]* mit den link-lokalen IP-Adressen der Schnittstelle beantworten, über die die Anfrage empfangen wurde.



REQ.TA.mDNS.Querying.50

Die Implementierung **SOLL** die Anforderungen (MUST, SHOULD, MUST NOT, SHOULD NOT) an DND-SD nach [RFC6763] erfüllen.

12.3.2 TA.mDNS.BuildQuery

12.3.2.1 Beschreibung

Dieser TA beschreibt die Anforderungen an das Erzeugen einer mDNS Query.

12.3.2.2 Anforderungen



REQ.TA.mDNS.BuildQuery.20

Die Implementierung **SOLL** gewährleisten, dass sich die Antworten auf über IPv4 bzw. IPv6 empfangene Anfragen nicht unterscheiden.



REQ.TA.mDNS.BuildQuery.30

Die Implementierung **MUSS** Multicast mDNS-Nachrichtenaussendungen auf 30 pro Minute begrenzen.

12.3.3 TA.mDNS.ProcessResponse

12.3.3.1 Beschreibung

Dieser TA beschreibt die Anforderungen an das Verarbeiten von mDNS-Antwort-Nachrichten zur Ermittlung von IP-Adressen von selektierten mDNS-Hosts und Port-Nummern zur Service-Namen.

12.3.3.2 Anforderungen

- Die Implementierung **MUSS** die Nachricht *queryRequest* nach den Anforderungen von [RFC6762] und [RFC6763] verarbeiten und im Fehlerfall den Ablauf mit Fehler beenden. [REQ.TA.mDNS.ProcessResponse.10]

- Sofern die Implementierung an der Schnittstelle IPv6 unterstützt, **MUSS** die Implementierung über IPv6 empfangene mDNS-Nachrichten verarbeiten können. [REQ.TA.mDNS.ProcessResponse.20]

13 TLS - Transport Layer Security Protokoll

13.1 Einleitung

TLS ermöglicht einen vertraulichen, integritätsgeschützten und gegenseitig authentifizierten, bidirektionalen Transportkanal zwischen zwei Anwendungen. TLS ist ein hybrides Verschlüsselungsprotokoll, dessen Aufgabe die Sicherung auf der Ebene der Transportschicht (Schicht 4 im OSI-Referenzmodell [X.200]) ist und dessen Funktionen bis in die Anwendungsschicht (Schicht 7) hineinreichen.

Diese Detailspezifikation zu TLS enthält Interoperabilitäts-Anforderungen an die Implementierung eines TLS-Clients und eines TLS-Servers an den Kommunikationsschnittstellen des SMGW.

[RFC8446] präzisiert Interoperabilitäts-Anforderungen an Implementierungen, die sowohl das Protokoll TLS 1.3 als auch ältere Protokollversionen unterstützen. Da die Kommunikationspartner des iMSys keine TLS-Versionen vor 1.2 unterstützen dürfen, enthält diese Detailspezifikation Anforderungen an Implementierungen die ausschließlich TLS 1.2 gemäß [RFC5246] unterstützen und solche Implementierungen die sowohl TLS 1.2 gemäß [RFC5246] als auch TLS 1.3 gemäß [RFC8446] unterstützen. [RFC8446] stellt Anforderungen an TLS 1.2 Implementierungen, die der Verbesserung der Sicherheit und Interoperabilität dienen, und umgesetzt werden sollen. Die Interoperabilitätsanforderungen dieser Detailspezifikation folgen den Sicherheitsvorgaben der [TR-03109-3].

Die Anforderungen an die TLS-Implementierungen werden dabei in unterteilt in

- Allgemeine Anforderungen an die Interoperabilität beim Handshake, der Anwendungsdatenübertragung und dem Verbindungsende
- Ergänzende Anforderungen an den TLS-Handshake im WAN (Client)
- Ergänzende Anforderungen an den TLS-Handshake im HAN (Client, Server)
- Ergänzende Anforderungen an den TLS-Handshake im LMN (Client, Server)

Übersicht über die in dieser Detailspezifikation referenzierten Spezifikationen:

- TLS 1.3 gemäß [RFC8446] und TLS 1.2-Handshake gemäß [RFC5246] mit den TLS 1.2 spezifischen Anforderungen aus [RFC8446]. [RFC8446] beschreibt u.a. das Handshake-Protokoll mit TLS 1.3-Hello-Extensions und Handshake-Nachrichten, [RFC5246] das Handshake-Protokoll mit TLS 1.2-Hello-Extensions- und Handshake-Nachrichten.
- Cipher Suites, kryptografische Algorithmen und Parameter und weitere Anforderungen gemäß der in [TR-03109-3] referenzierten Spezifikationen.
 - Elliptische Kurven für TLS 1.2 gemäß [RFC8422], [RFC7027], [RFC5289]
 - encrypt_then_mac Extension gemäß [RFC7366] für AES-CBC Cipher Suites in TLS 1.2
 - extended_master_secret gemäß [RFC7627] für den TLS 1.2-Handshake
 - Keine gmt_unix_time im client_hello des TLS 1.2-Handshake
- server_name Extension gemäß [RFC6066]
- application_layer_protocol_negotiation Extension gemäß [RFC7301]
- Die Anforderungen an TLS-Zertifikate basierend auf [RFC5280] sind in Zertifikatsprofilen in [TR-03109-4] und [DS] beschrieben.

13.2 Datenstrukturen

Folgende Daten werden von der Implementierung für das TLS-Protokoll gemäß dieser Detailspezifikation verwendet:

- Zur TLS-Authentifizierung der Implementierung gegenüber dem Kommunikationspartner: Privater ECC-Schlüssel, X.509 TLS-Zertifikat mit zugehörigem öffentlichem ECC-Schlüssel, die die kryptografische Identität der Implementierung darstellen.

- Zur TLS-Authentifizierung des Kommunikationspartners über PKI-Vertrauensanker: Ein oder zwei Vertrauensanker des Ausstellers (CA) zur Validierung der Zertifikatskette (*trustedCAs*). Für Zertifikats-Wechselprozesse werden zur Verbesserung der Verfügbarkeit zwei Vertrauensanker empfohlen. Die Implementierung und der Kommunikationspartner müssen sich im Überlappszeitraum der Gültigkeit über das zu verwendende Zertifikat abstimmen.
- Zur TLS-Authentifizierung des Kommunikationspartners über "Direct-Trust": Ein TLS-X.509-Zertifikat zur Berechtigungsprüfung auf direkte Übereinstimmung, das für die Authentifizierung und Verschlüsselung der TLS- und Anwendungsdaten verwendet wird.
- Anwendungsparameter für den Sitzungsaufbau (Handshake-Mode (Client/Server/Full/Resumed) und Parameter für Hello-Extensions wie ALPN, SNI), die sich u.a. aus dem zugrundeliegenden Kommunikationsszenario ergeben.
- Flüchtige (interne) Zustandsdaten während der Lebensdauer einer TLS-Session gemäß [RFC8446] an die keine Interoperabilitätsanforderungen gestellt werden.
- Anwendungsdaten, die über TLS transportiert werden.

Die verwendeten TLS-Zertifikate sind im TLS 1.2-Handshake an den Schnittstellen beobachtbar, im TLS 1.3 Handshake werden sie verschlüsselt übertragen.

13.3 Technische Anwendungsfälle

Dieser Abschnitt umfasst die technischen Anwendungsfälle für TLS. Zu Beginn werden Anforderungen an alle Implementierungen definiert. Die folgenden Abschnitte ergänzen die Allgemeinen Anforderungen nach rollenspezifischen Anforderungen für TLS-Server und TLS-Client und Schnittstelle.

13.3.1 TA.TLS.Handshake

13.3.1.1 Beschreibung

Das TLS-Handshake-Protokoll hat die Aufgabe, die sichere Authentifizierung der Kommunikationspartner durchzuführen und die Rahmenbedingungen für einen vertraulichen Kommunikationskanal zu etablieren. Es gibt unterschiedliche Varianten des TLS 1.2-Handshakes (Mit oder ohne Serverside/Clientside State Resumption) und des TLS 1.3-Handshakes (bspw. mit oder ohne Kenntnis eines vorher vereinbarten Pre-Shared-Key) um den TLS-Verbindungsaufbau zu beschleunigen. Im Folgenden werden die Anforderungen definiert, die für alle Schnittstellen ungeachtet der Rolle gelten.

13.3.1.2 Ablaufbeschreibung

►Abbildung 13.1 und ►Abbildung 13.2 zeigt den Ablauf eines TLS 1.2-Handshakes zwischen TLS-Client und TLS-Server.

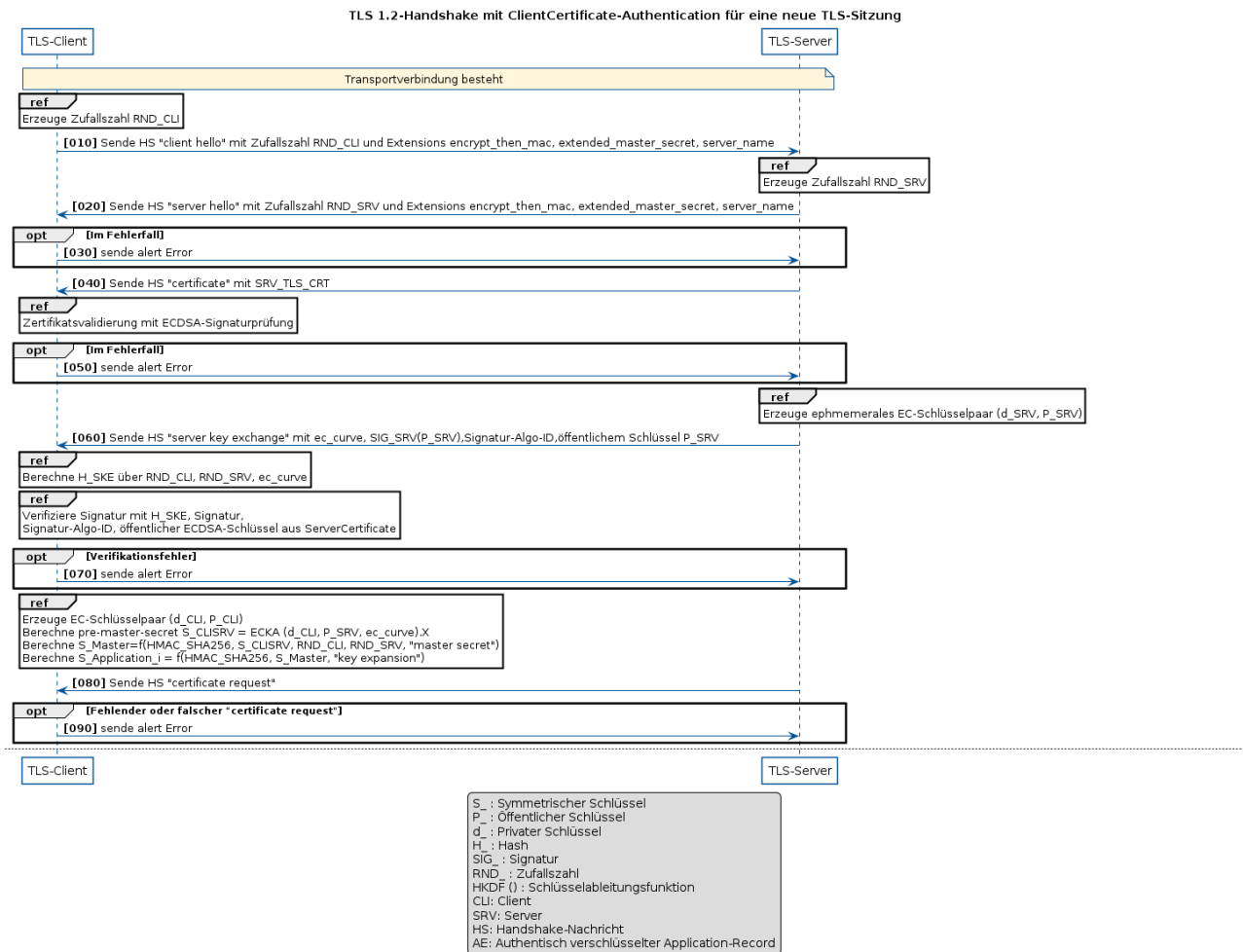


Abbildung 13.1. Ablauf eines TLS 1.2 Handshakes für eine neue TLS-Sitzung (1/2)

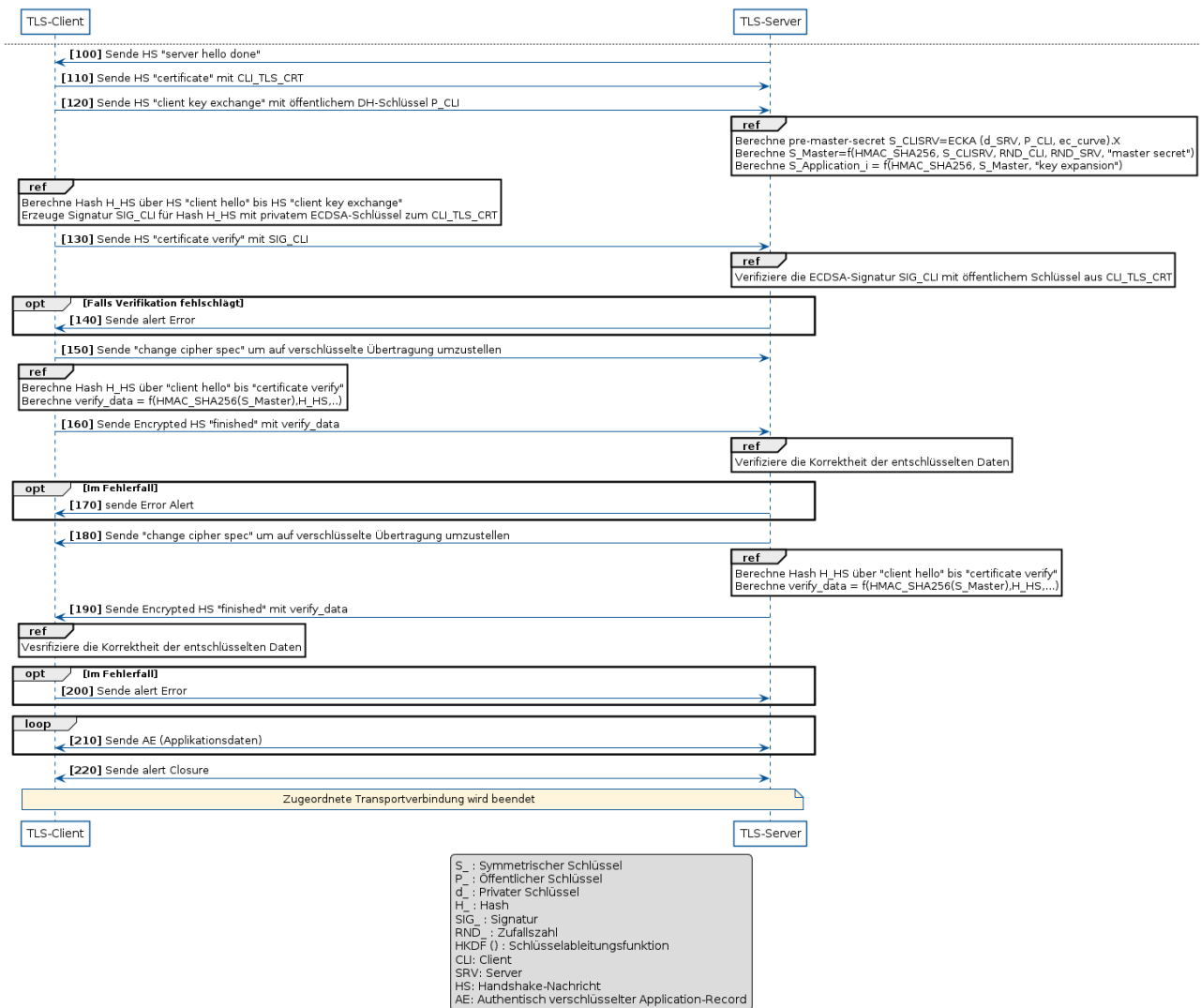


Abbildung 13.2. Ablauf eines TLS 1.2 Handshakes für eine neue TLS-Sitzung (2/2)

►Abbildung 13.3 und ►Abbildung 13.4 zeigt den Ablauf eines TLS 1.3-Handshakes zwischen TLS-Client und TLS-Server.

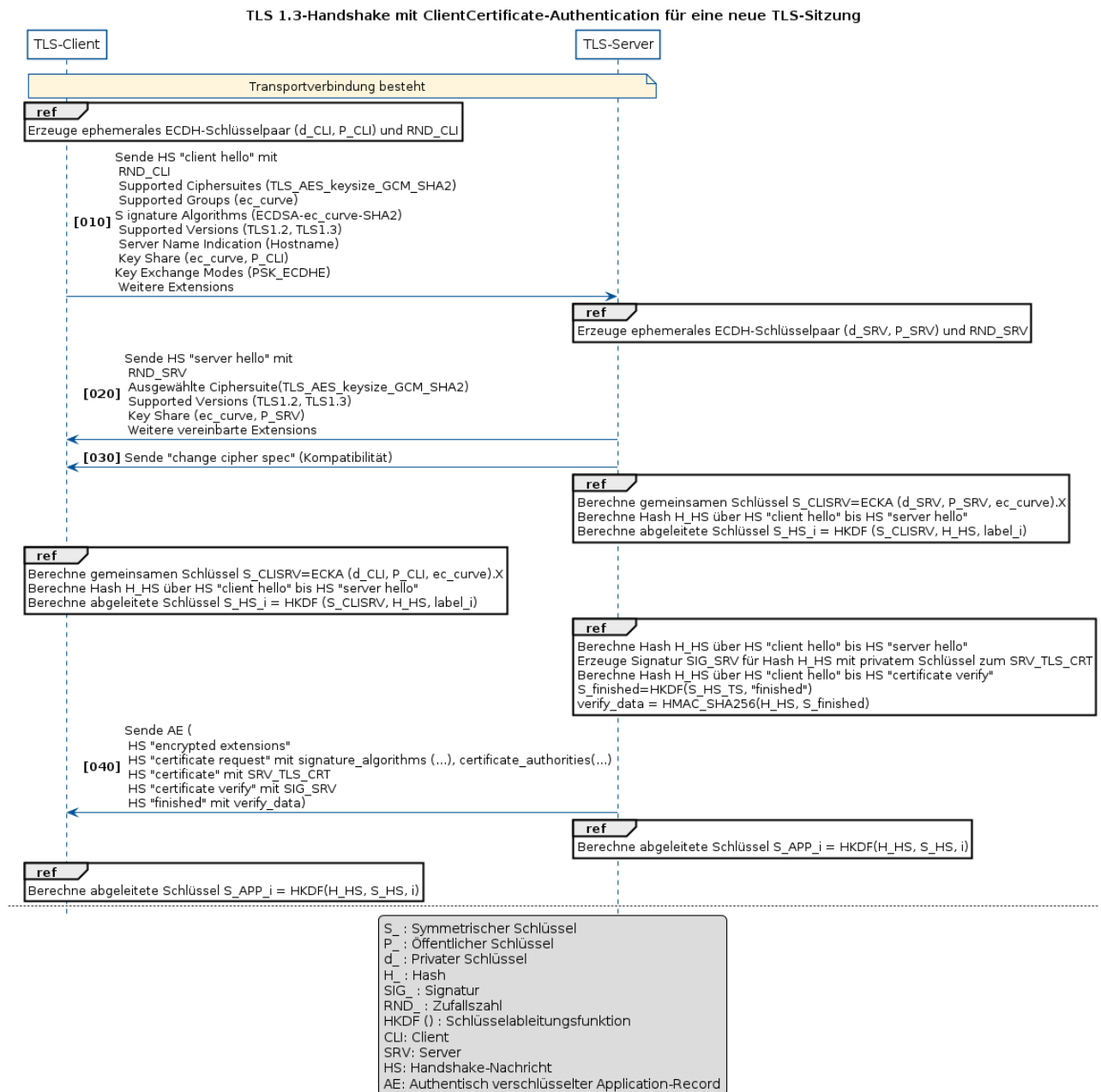


Abbildung 13.3. Ablauf eines TLS 1.3-Handshakes für eine neue TLS-Sitzung (1/2)

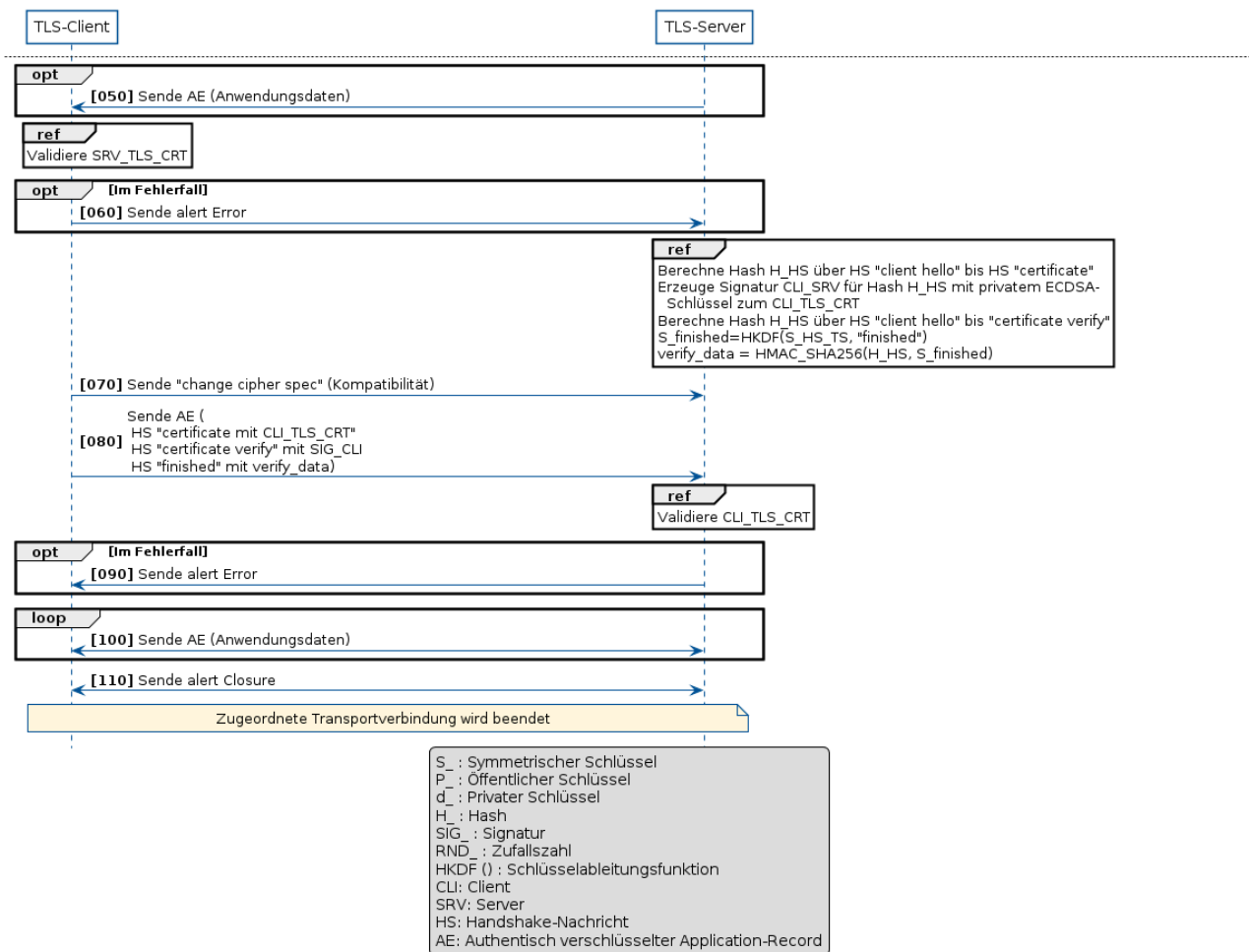


Abbildung 13.4. Ablauf eines TLS 1.3-Handshakes für eine neue TLS-Sitzung (2/2)

13.3.1.3 Anforderungen



REQ.TA.TLS.Handshake.10

Die Implementierung **MUSS** den TLS-Handshake der Version 1.2 nach [RFC5246] Kapitel 7.3 und 7.4 unterstützen.



REQ.TA.TLS.Handshake.20

Die Implementierung **MUSS** die verwendeten Zertifikate gemäß ihres Zertifikatsprofils auf Integrität, zeitliche Gültigkeit und Schlüsseignung im Kontext des Zugriffs prüfen.



REQ.TA.TLS.Handshake.30

Die Implementierung **MUSS** die nach [TR-03109-3] für die jeweiligen Schnittstellen genannten, mindestens zu unterstützenden Cipher Suiten und elliptischen Kurven akzeptieren.



REQ.TA.TLS.Handshake.50

Die Implementierung **MUSS** im TLS 1.2-Handshake die von der Implementierung unterstützten NamedCurveIDs (alias SupportedGroupsID) gemäß [RFC8422] Kapitel 5.1 senden bzw. akzeptieren. (Siehe ▶ICS.TA.TLS.WanHandshake.30, ▶ICS.TA.TLS.HanHandshake.30, ▶ICS.TA.TLS.LmnHandshake.30)

**REQ.TA.TLS.Handshake.60**

Die Implementierung **SOLL** NamedCurveIds für höhere Ordnungen bzw. Schlüssellängen an den Anfang der Liste der supported_groups-Extension stellen.

**REQ.TA.TLS.Handshake.70**

Falls die Implementierung den TLS 1.3-Handshake unterstützt, **MUSS** die Implementierung die unterstützten NamedCurveIDs (alias SupportedGroupsID) der in [TR-03109-3] erlaubten Kurvenparameter gemäß [RFC8446] Kap 4.2.7 und [RFC8734] senden bzw. akzeptieren.

**REQ.TA.TLS.Handshake.80**

Die Implementierung **MUSS** für den TLS 1.2-Handshake die encrypt_then_mac Extension gemäß [RFC7366] unterstützen.

**REQ.TA.TLS.Handshake.90**

Die Implementierung **SOLL** für den TLS 1.2-Handshake die extended_master_secret Extension gemäß [RFC7627] unterstützen.

**REQ.TA.TLS.Handshake.110**

Die TLS-Client Implementierung **MUSS** den TLS-Handshake mit Alert "fatal" beenden, falls der Server Hello-Extensions aus [RFC6066] in einem "resumed" Handshake sendet.

**REQ.TA.TLS.Handshake.120**

Die Implementierung **SOLL** im ClientHello des TLS 1.2-Handshake anstelle der eigenen Zeit eine Zufallszahl als gmt_unix_time verwenden.

**REQ.TA.TLS.Handshake.130**

Die Implementierung **MUSS** den TLS-Handshake nach Zertifikatsvalidierungs- oder Authentifizierungsfehlern mit Alert "fatal" beenden.

**REQ.TA.TLS.Handshake.140**

Die Implementierung als TLS-Client **SOLL** im TLS1.2-Handshake in der Nachricht "ClientHello" die Extension "trusted_ca_keys" gemäß [RFC6066] mit einer Liste von X.501-DistinguishedNames der in der Implementierung administrativ konfigurierten (CA-)Zertifikate zur Authentifizierung des TLS-Server übermitteln, sofern mehr als ein CA-Zertifikat zur Authentifizierung konfiguriert ist (▶ ICS.TA.TLS.Handshake.20).

**REQ.TA.TLS.Handshake.145**

Die Implementierung als TLS-Server **SOLL** im TLS1.2-Handshake in der Nachricht "Certificate-Request" in "certificate_authorities" eine Liste von X.501-DistinguishedNames der in der Implementierung administrativ konfigurierten (CA-)Zertifikate zur Authentifizierung des TLS-Clients übermitteln, sofern mehr als ein CA-Zertifikat zur Authentifizierung konfiguriert ist (▶ ICS.TA.TLS.Handshake.30).

**REQ.TA.TLS.Handshake.150**

Falls die Implementierung den TLS 1.3-Handshake unterstützt, **MUSS** die Implementierung als TLS-Client im TLS1.3-Handshake in der Nachricht "ClientHello" die Extension "certificate_authorities" gemäß [RFC8446] mit einer Liste von X.501-DistinguishedNames der in der Implementie-

ung administrativ konfigurierten (CA-)Zertifikate zur Authentifizierung des TLS-Servers übermitteln, sofern mehr als ein CA-Zertifikat zur Authentifizierung konfiguriert ist.



REQ.TA.TLS.Handshake.155

Falls die Implementierung den TLS 1.3-Handshake unterstützt, **MUSS** die Implementierung als TLS-Server im TLS1.3-Handshake in der Nachricht "CertificateRequest" die Extension "certificate_authorities" gemäß [RFC8446] mit einer Liste von X.501-DistinguishedNames der in der Implementierung administrativ konfigurierten (CA-)Zertifikate zur Authentifizierung des TLS-Clients übermitteln, sofern mehr als ein CA-Zertifikat zur Authentifizierung konfiguriert ist.



REQ.TA.TLS.Handshake.160

Die Implementierung des TLS-Clients **MUSS** ein empfangenes TLS-Server-Zertifikat gemäß [RFC5280] Kapitel 6 validieren, indem sie auf Übereinstimmung mit einem für den Verwendungszweck geeigneten administrativ konfigurierten SM-PKI-Endnutzer-Zertifikat geprüft werden. Für den TLS-Verbindungsaufbau zur Zeitsynchronisation kann auf die zeitliche Gültigkeitsprüfung verzichtet werden, solange keine vertrauenswürdige Zeit in der Implementierung verfügbar ist.



REQ.TA.TLS.Handshake.170

Sofern gemäß Kommunikationsszenario die Implementierung des TLS-Clients selbstsignierte TLS-Server Zertifikate akzeptieren kann, **MUSS** die Implementierung ein empfangenes, selbstsigniertes Server-Zertifikat vor der Verwendung auf Übereinstimmung mit einem für die Berechtigungsprüfung administrativ konfigurierten TLS-Zertifikat prüfen. Auf die zeitliche Gültigkeitsprüfung kann verzichtet werden.



REQ.TA.TLS.Handshake.180

Die Implementierung eines TLS-1.3-Servers **MUSS** im TLS 1.3-Handshake die Rückwärtskompatibilität mit älteren Clients gemäß [RFC8446] Annex D.2 unterstützen (supported_versions Extension)



REQ.TA.TLS.Handshake.190

Die Implementierung eines TLS-1.3-Clients **MUSS** im TLS 1.3-Handshake die Rückwärtskompatibilität mit älteren Servern gemäß [RFC8446] Annex D.1 unterstützen (supported_versions Extension)

13.3.1.4 Implementation Conformance Statements



ICS.TA.TLS.Handshake.10

Der Hersteller **MUSS** erklären, ob die Implementierung in der supported_groups extension die NamedCurves für höhere Schlüssellängen an den Anfang stellt.



ICS.TA.TLS.Handshake.20

Der Hersteller **MUSS** erklären, ob die Implementierung als TLS 1.2-Client die CA-Zertifikate in der trusted_ca_keys Extension der ClientHello-Nachricht sendet.



ICS.TA.TLS.Handshake.30

Der Hersteller **MUSS** erklären, ob die Implementierung als TLS 1.2-Server die CA-Zertifikate in certificate_authorities der CertificateRequest-Nachricht sendet.

13.3.1.5 Implementierungshinweise

1. Zur Vereinbarung eines gemeinsamen Vertrauensankers und zur Vereinfachung von Wechselprozessen des Vertrauensankers für die Zertifikatsvalidierung wird für alle Implementierungen im HAN empfohlen, die von der Implementierung verwendeten Vertrauensanker (Direct-Trust oder RootCA-Zertifikat) dem Kommunikationspartner im TLS-Handshake zu präsentieren. Für TLS 1.3 wird die Unterstützung der Certificate-Authorities Extension, für TLS 1.2 die Unterstützung der certificate_authorities im Certificate Request empfohlen.
2. Da in den folgenden Jahren noch Implementierungen genutzt werden, die ausschließlich TLS 1.2 unterstützen, ist eine Unterstützung sowohl des TLS 1.3-Handshake gemäß [RFC8446] als auch die Rückwärtskompatibilität mit einer reinen TLS1.2 Implementierung gemäß [RFC8446] Kap. D.1 (als TLS-Client) bzw. D.2 (als TLS-Server) notwendig.
3. Aufgrund der erforderlichen Rückwärtskompatibilität ist es erforderlich gemäß [TR-03109-3] sowohl AES-GCM- als auch AES-CBC-Ciphersuites im TLS 1.2-Handshake zu unterstützen. Dies erfordert die Unterstützung der TLS-Hello Extension encrypt_then_mac ([RFC7366]).
4. Kryptografische Agilität unterstützt die längerfristige Verwendung einer Implementierung. Deshalb wird empfohlen, die Implementierung (Hardware und Software) bereits für die künftig empfohlenen Verfahren gemäß [TR-03109-3] auszulegen. Eine Implementierung sollte insbesondere im TLS-Zertifikate mit NIST-P256/brainpoolP256r1/brainpoolP384r1/brainpoolP512r1 Kurvenparametern verarbeiten und speichern können (sowohl eigene als auch die des Kommunikationspartners).
5. Zur Verringerung der Datenmenge im Kommunikationsnetz, zur Beschleunigung des Verbindungsaufbaus und der Reduktion der Berechnungszeit für asymmetrische Kryptografie wird empfohlen innerhalb der maximalen TLS-Session-Lebensdauer (siehe [TR-03109-3] die PSK-basierte TLS-Session-Resumption unterstützen.

13.3.2 TA.TLS.Close

Die folgenden Anforderungen an das Verhalten beim Beenden der TLS-Verbindung ergänzen oder präzisieren die Anforderungen der TLS Spezifikationen.



REQ.TA.TLS.Close.10

Die Implementierung **MUSS** nach empfangenen TLS-Alerts [RFC8446] Kapitel 6 mit Level "fatal" die TLS-Verbindung schließen, so dass keine weiteren Anwendungsdaten über die TLS-Verbindung ausgetauscht werden.



REQ.TA.TLS.Close.20

Die Implementierung **SOLL** nach empfangenen TLS-Alerts mit Level "fatal" für den Betreiber/Administrator ein Ereignis protokollieren.



REQ.TA.TLS.Close.30

Die Implementierung **MUSS** Alerts gemäß [RFC8446] Kapitel 6 erzeugen können.



REQ.TA.TLS.Close.40

Die Implementierung **MUSS** Alerts gemäß [RFC8446] Kapitel 6 verarbeiten können.



REQ.TA.TLS.Close.60

Die TLS 1.2-Implementierung **MUSS** die Verbindung schließen, wenn application_data TLS-Records vor Abschluss des Handshakes eintreffen

**REQ.TA.TLS.Close.70**

Die TLS 1.2-Implementierung **MUSS** im AES-CBC- Mode TLS-Fragmente verarbeiten können, die ein AES-CBC-TLS-Block-Padding mit einer Länge von insgesamt 16-256 Bytes enthalten. Wird ein längerer Record empfangen, wird ein TLS-Alert "record_overflow" gesendet und die TLS-Verbindung beendet.

**REQ.TA.TLS.Close.80**

Die Implementierung **MUSS** innerhalb von 20s nach dem Senden eines TLS-close_notify-Alert nach [RFC8446] Kapitel 6.1 die damit verknüpfte Transportverbindung beenden.

**REQ.TA.TLS.Close.90**

Wird die Transportverbindung nicht innerhalb von 20 Sekunden nach Empfang eines close_notify geschlossen, **MUSS** die Implementierung die Verbindung einseitig beenden.

13.3.3 TA.TLS.OpenWanSessionAsClient

13.3.3.1 Beschreibung

Dieser Abschnitt beschreibt die Anforderungen für den TLS-Handshake, der durch den TLS-Client im WAN initiiert wird.

13.3.3.2 Anforderungen

**REQ.TA.TLS.OpenWanSessionAsClient.10**

Die Implementierung **SOLL** an der WAN-Schnittstelle den TLS-Handshake der Version 1.3 nach [RFC8446] Kapitel 4 unterstützen.

**REQ.TA.TLS.OpenWanSessionAsClient.20**

Die Implementierung **SOLL** die nach [TR-03109-3] für die WAN-Schnittstelle genannten, empfohlenen Cipher Suites und elliptischen Kurven unterstützen .

**REQ.TA.TLS.OpenWanSessionAsClient.30**

Die Implementierung des TLS-Clients **MUSS** in der ClientHello-Nachricht im TLS 1.3-Handshake eine ClientHello-Extension "server_name" vom Typ host_name(0) nach [RFC6066] Kapitel 3 übermitteln. Der server_name MUSS einen Bezeichner im DNS-Format nach [RFC1035] Kapitel 2.3.1 enthalten.

Der Bezeichner ist gemäß WAN- oder Proxy-Kommunikationsprofil "tlsSni" bzw. "wanTlsSni" zu wählen, sofern dieses Datenfeld im Kommunikationsprofil angegeben ist.

**REQ.TA.TLS.OpenWanSessionAsClient.40**

Die Implementierung **KANN** im ClientHello weitere Extensions anbieten, sofern diese nicht durch [TR-03109-3] unzulässig sind und für die Interoperabilität die Antwort vom Server unerheblich ist.

**REQ.TA.TLS.OpenWanSessionAsClient.50**

Die Implementierung **SOLL** im TLS 1.2-Handshake die stateful Session Resumption mit session_ids gemäß [RFC5246] unterstützen.

**REQ.TA.TLS.OpenWanSessionAsClient.60**

Sofern die Implementierung den TLS 1.3-Handshake unterstützt, **SOLL** die Implementierung die TLS 1.3-Session Resumption mit PSKs gemäß [RFC8446] Kapitel 4.2.11 unterstützen.

13.3.3.3 ICS**ICS.TA.TLS.WanHandshake.10**

Der Hersteller **MUSS** für die WAN-Schnittstelle deklarieren, ob die Implementierung den TLS 1.3-Handshake gemäß [RFC8446] Kapitel 4 unterstützt.

**ICS.TA.TLS.WanHandshake.20**

Der Hersteller **MUSS** für die WAN-Schnittstelle deklarieren, welche Cipher Suites die Implementierung anbietet bzw. akzeptiert.

**ICS.TA.TLS.WanHandshake.30**

Der Hersteller **MUSS** für die WAN-Schnittstelle deklarieren, welche namedCurveIDs die Implementierung für das Signaturverfahren ECDSA und das Schlüsselaustauschprotokoll ECDHE unterstützt (d.h. in der supported_groups-Extension bzw. elliptic_curves-Extension anbietet).

**ICS.TA.TLS.WanHandshake.40**

Der Hersteller **MUSS** für die WAN-Schnittstelle die von der Implementierung unterstützten Signatur-Algorithmen (signature_algorithms) deklarieren.

**ICS.TA.TLS.WanHandshake.41**

Der Hersteller **MUSS** für die WAN-Schnittstelle die von der Implementierung unterstützten Zertifikats-Signatur-Algorithmen (signature_algorithms_cert) deklarieren.

**ICS.TA.TLS.WanHandshake.50**

Der Hersteller **MUSS** für die WAN-Schnittstelle die zusätzlich zu den in [TR-03109-3] und [RFC8446] Kapitel 9.2 verpflichtend zu unterstützenden Client/Server-Hello-Extensions implementierten Extensions deklarieren.

**ICS.TA.TLS.WanHandshake.60**

Der Hersteller **MUSS** für die WAN-Schnittstelle deklarieren, ob die Implementierung die TLS 1.2-Serverside-Stateful Session-Resumption mit Session-ID nach [RFC5246] Kapitel 7.3 unterstützt.

**ICS.TA.TLS.WanHandshake.70**

Der Hersteller **MUSS** für die WAN-Schnittstelle deklarieren, ob die Implementierung den verkürzten TLS 1.3-Handshake basierend auf einem in einer vorherigen TLS-Session erzeugten PSK gemäß [RFC8446] Kapitel 2.2 unterstützt.

**ICS.TA.TLS.WanHandshake.80**

Der Hersteller **MUSS** für die WAN-Schnittstelle deklarieren, ob die Implementierung für den TLS 1.2-Handshake die Extension encrypt_then_mac gemäß [RFC7366] unterstützt.

**ICS.TA.TLS.WanHandshake.90**

Der Hersteller **MUSS** für die WAN-Schnittstelle deklarieren, ob die Implementierung für den TLS 1.2-Handshake die Extension `extended_master_secret` gemäß [RFC7627] unterstützt.

**ICS.TA.TLS.WanHandshake.100**

Der Hersteller **MUSS** für die WAN-Schnittstelle deklarieren, ob die `gmt_unix_time` in der von der Implementierung gesendeten ClientHello-Nachricht des TLS1.2-Handshakes durch eine Zufallszahl ersetzt wird.

13.3.4 TA.TLS.OpenHanSessionAsClient

13.3.4.1 Beschreibung

Dieser Abschnitt beschreibt die Anforderungen für den TLS-Handshake, der durch den TLS-Client im HAN initiiert wird.

13.3.4.2 Anforderungen

**REQ.TA.TLS.OpenHanSessionAsClient.10**

Die Implementierung **SOLL** den TLS-Handshake der Version 1.3 nach [RFC8446] Kapitel 4 unterstützen.

**REQ.TA.TLS.OpenHanSessionAsClient.20**

Die Implementierung **SOLL** die nach [TR-03109-3] für die HAN-Schnittstelle genannten, **empfohlenen** Cipher Suiten und elliptischen Kurven unterstützen.

**REQ.TA.TLS.OpenHanSessionAsClient.30**

Die Implementierung des TLS-Clients **MUSS** in der ClientHello-Nachricht im TLS 1.3-Handshake eine ClientHello-Extension "server_name" vom Typ `host_name(0)` nach [RFC6066] Kapitel 3 übermitteln. Der `server_name` MUSS einen Bezeichner im DNS-Format nach [RFC1035] Kapitel 2.3.1 enthalten.

Der Bezeichner ist gemäß Proxy-Kommunikationsprofil "localServerName" für HKS.TLSPROXY.CLI zu wählen, sofern dieses Datenfeld im Kommunikationsprofil angegeben ist.

**REQ.TA.TLS.OpenHanSessionAsClient.40**

Die Implementierung **KANN** im ClientHello weitere Extensions anbieten, sofern diese nicht durch [TR-03109-3] unzulässig sind und für die Interoperabilität die Antwort vom Server unerheblich ist.

13.3.4.3 ICS

**ICS.TA.TLS.HanHandshake.10**

Der Hersteller **MUSS** für die HAN-Schnittstelle deklarieren, ob die Implementierung den TLS 1.3-Handshake gemäß [RFC8446] Kapitel 4 unterstützt.

**ICS.TA.TLS.HanHandshake.20**

Der Hersteller **MUSS** für die HAN-Schnittstelle deklarieren, welche Cipher Suiten die Implementierung anbietet bzw. akzeptiert.

**ICS.TA.TLS.HanHandshake.30**

Der Hersteller **MUSS** für die HAN-Schnittstelle deklarieren, welche `namedCurveIDs` die Implementierung für das Signaturverfahren ECDSA und das Schlüsselaustauschprotokoll ECDHE unterstützt (d.h. in der `supported_groups`-Extension bzw. `elliptic_curves`-Extension anbietet).

**ICS.TA.TLS.HanHandshake.40**

Der Hersteller **MUSS** für die HAN-Schnittstelle die von der Implementierung unterstützten Zertifikats-Signatur-Algorithmen (`signature_algorithms`) deklarieren.

**ICS.TA.TLS.HanHandshake.50**

Der Hersteller **MUSS** für die HAN-Schnittstelle die zusätzlich zu den in [TR-03109-3] und [RFC8446] Kapitel 9.2 verpflichtend zu unterstützenden Client/Server-Hello-Extensions implementierten, Extensions deklarieren.

**ICS.TA.TLS.HanHandshake.60**

Der Hersteller **MUSS** für die HAN-Schnittstelle deklarieren, ob die Implementierung die TLS 1.2-Serverside-Stateful Session-Resumption mit Abbreviated Handshake basierend auf Pre-Shared Key nach [RFC5246] Kapitel 7.3 unterstützt.

**ICS.TA.TLS.HanHandshake.70**

Der Hersteller **MUSS** für die HAN-Schnittstelle deklarieren, ob die Implementierung den verkürzten TLS 1.3-Handshake basierend auf einem in einer vorherigen TLS-Session erzeugten PSK gemäß [RFC8446] Kapitel 2.2 unterstützt.

**ICS.TA.TLS.HanHandshake.80**

Der Hersteller **MUSS** für die HAN-Schnittstelle deklarieren, ob die Implementierung für den TLS 1.2-Handshake die Extension `encrypt_then_mac` gemäß [RFC7366] unterstützt.

**ICS.TA.TLS.HanHandshake.90**

Der Hersteller **MUSS** für die HAN-Schnittstelle deklarieren, ob die Implementierung für den TLS 1.2-Handshake die Extension `extended_master_secret` gemäß [RFC7627] unterstützt.

**ICS.TA.TLS.HanHandshake.100**

Der Hersteller **MUSS** für die HAN-Schnittstelle deklarieren, ob die `gmt_unix_time` in der von der Implementierung gesendeten ClientHello-Nachricht durch eine Zufallszahl ersetzt wird.

13.3.5 TA.TLS.OpenHanSessionAsServer

13.3.5.1 Beschreibung

Dieser Abschnitt beschreibt die ergänzenden Anforderungen für den TLS-Handshake, eines TLS-Servers im HAN.

13.3.5.2 Anforderungen

**REQ.TA.TLS.OpenHanSessionAsServer.10**

Die Implementierung **SOLL** an der HAN-Schnittstelle als Server den TLS-Handshake der Version 1.3 nach [RFC8446] Kapitel 4 unterstützen.



REQ.TA.TLS.OpenHansSessionAsServer.20

Sofern die Implementierung ein SM-PKI-TLS-Zertifikat mit ExtendedKeyUsage "TLS-Client" verwendet, **SOLL** dies sowohl für TLS-Client- als auch für TLS-Server-Authentifizierung verwendet werden können.

13.3.5.3 ICS

Siehe ▶Abschnitt 13.3.4.3

13.3.6 TA.TLS.OpenLmnSessionAsClient

13.3.6.1 Beschreibung

Dieser Abschnitt beschreibt die ergänzenden Anforderungen für den TLS-Handshake, der durch den TLS-Client im LMN initiiert wird.

13.3.6.2 Anforderungen



REQ.TA.TLS.OpenLmnSessionAsClient.10

Die Implementierung **SOLL** an der LMN-Schnittstelle als Client den TLS-Handshake der Version 1.3 nach [RFC8446] Kapitel 4 unterstützen.



REQ.TA.TLS.OpenLmnSessionAsClient.30

Der Implementierung des TLS-Clients **SOLL** in der ClientHello-Nachricht eine ClientHello-Extension "alpn" mit dem Applikationslayer-Protokoll und Version senden.



REQ.TA.TLS.OpenLmnSessionAsClient.40

Die Implementierung **SOLL** die nach [TR-03109-3] für die LMN-Schnittstelle genannten, empfohlenen Cipher Suiten und elliptischen Kurven unterstützen.



REQ.TA.TLS.OpenLmnSessionAsClient.50

Die Implementierung **KANN** im ClientHello weitere Extensions anbieten, sofern diese nicht durch [TR-03109-3] unzulässig sind und für die Interoperabilität die Antwort vom Server unerheblich ist.



REQ.TA.TLS.OpenLmnSessionAsClient.60

Die Implementierung **SOLL** im TLS 1.2-Handshake die stateful Session Resumption mit session_ids gemäß [RFC5246] unterstützen.



REQ.TA.TLS.OpenLmnSessionAsClient.70

Sofern die Implementierung den TLS 1.3-Handshake unterstützt, **MUSS** die Implementierung die TLS 1.3-Session Resumption mit PSKs gemäß [RFC8446] Kapitel 4.2.11 unterstützen.

13.3.6.3 ICS



ICS.TA.TLS.LmnHandshake.10

Der Hersteller **MUSS** für die LMN-Schnittstelle deklarieren, ob die Implementierung als Client den TLS 1.3-Handshake gemäß [RFC8446] Kapitel 4 unterstützt.

**ICS.TA.TLS.LmnHandshake.15**

Der Hersteller **MUSS** für die LMN-Schnittstelle deklarieren, ob die Implementierung als Server den TLS 1.3-Handshake gemäß [RFC8446] Kapitel 4 unterstützt.

**ICS.TA.TLS.LmnHandshake.20**

Der Hersteller **MUSS** für die LMN-Schnittstelle deklarieren, welche Cipher Suites die Implementierung anbietet bzw. akzeptiert.

**ICS.TA.TLS.LmnHandshake.30**

Der Hersteller **MUSS** für die LMN-Schnittstelle deklarieren, welche namedCurveIDs die Implementierung für das Signaturverfahren ECDSA und das Schlüsselaustauschprotokoll ECDHE unterstützt (d.h. in der supported_groups-Extension bzw. elliptic_curves-Extension anbietet).

**ICS.TA.TLS.LmnHandshake.40**

Der Hersteller **MUSS** für die LMN-Schnittstelle die von der Implementierung unterstützten Zertifikats-Signatur-Algorithmen (signature_algorithms) deklarieren.

**ICS.TA.TLS.LmnHandshake.50**

Der Hersteller **MUSS** für die LMN-Schnittstelle deklarieren, ob die Implementierung die Client-Hello-Extension "server_name" übermittelt.

**ICS.TA.TLS.LmnHandshake.60**

Der Hersteller **MUSS** für die LMN-Schnittstelle deklarieren, ob die Implementierung die Client-Hello-Extension "alpn" übermittelt.

**ICS.TA.TLS.LmnHandshake.70**

Der Hersteller **MUSS** für die LMN-Schnittstelle die zusätzlich zu den in [TR-03109-3] und [RFC8446] Kapitel 9.2 verpflichtend zu unterstützenden Client/Server-Hello-Extensions implementierten, Extensions deklarieren.

**ICS.TA.TLS.LmnHandshake.80**

Der Hersteller **MUSS** für die LMN-Schnittstelle deklarieren, ob die Implementierung für den TLS 1.2-Handshake die Extension encrypt_then_mac gemäß [RFC7366] unterstützt.

**ICS.TA.TLS.LmnHandshake.90**

Der Hersteller **MUSS** für die LMN-Schnittstelle deklarieren, ob die Implementierung den verkürzten TLS 1.3-Handshake basierend auf einem in einer vorherigen TLS-Session erzeugten PSK gemäß [RFC8446] Kapitel 2.2 unterstützt.

**ICS.TA.TLS.LmnHandshake.100**

Der Hersteller **MUSS** für die LMN-Schnittstelle deklarieren, ob die Implementierung für den TLS 1.2-Handshake die Extension extended_master_secret gemäß [RFC7627] unterstützt.

**ICS.TA.TLS.LmnHandshake.110**

Der Hersteller **MUSS** für die LMN-Schnittstelle deklarieren, ob die gmt_unix_time in der von der Implementierung gesendeten ClientHello-Nachricht durch eine Zufallszahl ersetzt wird.

13.3.7 TA.TLS.OpenLmnSessionAsServer

13.3.7.1 Beschreibung

Dieser Abschnitt beschreibt die ergänzenden Anforderungen an den TLS-Handshake eines TLS-Servers im LMN.

13.3.7.2 Anforderungen



REQ.TA.TLS.OpenLmnSessionAsServer.10

Die Implementierung **SOLL** an der LMN-Schnittstelle als TLS-Server den TLS-Handshake der Version 1.3 nach [RFC8446] Kapitel 4 unterstützen.



REQ.TA.TLS.OpenLmnSessionAsServer.20

Die Implementierung **SOLL** die nach [TR-03109-3] für die LMN-Schnittstelle genannten, empfohlenen Cipher Suites und elliptischen Kurven unterstützen.



REQ.TA.TLS.OpenLmnSessionAsServer.30

Die Implementierung **SOLL** im TLS 1.2-Handshake die stateful Session Resumption mit session_ids gemäß [RFC5246] unterstützen.



REQ.TA.TLS.OpenLmnSessionAsServer.40

Sofern die Implementierung den TLS 1.3-Handshake unterstützt, **SOLL** die Implementierung die TLS 1.3-Session Resumption mit PSKs gemäß [RFC8446] Kapitel 4.2.11 unterstützen.

13.3.7.3 ICS

Siehe ▶Abschnitt 13.3.6.3

13.4 Nutzung des Sicherheitsmoduls beim TLS-Handshake des SMGW

Das SMGW nutzt beim TLS-Handshake ein zertifiziertes Sicherheitsmodul gemäß [TR-03109-2]:

- zur Erzeugung der Zufallszahlen der ClientHello Nachricht,
- zur Schlüsselaushandlung des TLS primären Sitzungsschlüssels gemäß Elliptic Curve Diffie-Hellman,
- zur Signaturerzeugung und -prüfung für die Authentifizierung gemäß ECDSA,
- zur Erzeugung der asymmetrischen Schlüsselpaare und Speicherung der statischen Schlüsselpaare.

Das SMGW führt die Verschlüsselung, Entschlüsselung, Schlüsselableitung, Hash-Erzeugung mit symmetrischen Verfahren ohne die Dienste des Sicherheitsmoduls durch. Dies dient einer höheren Performance und der Verbesserung kryptografischer Agilität.

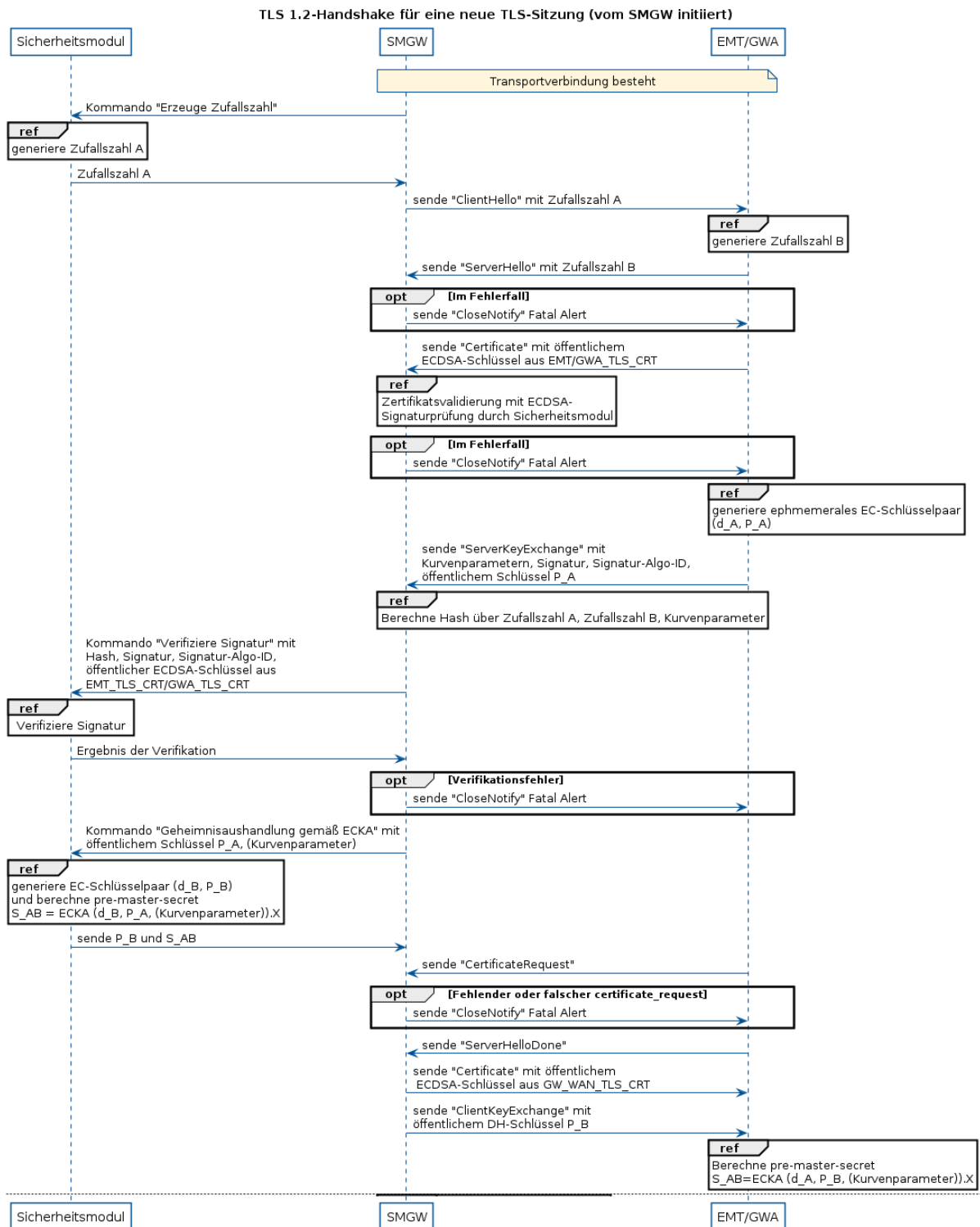


Abbildung 13.5. Interaktion zwischen Gateway und Sicherheitsmodul beim TLS 1.2-Handshake 1/2

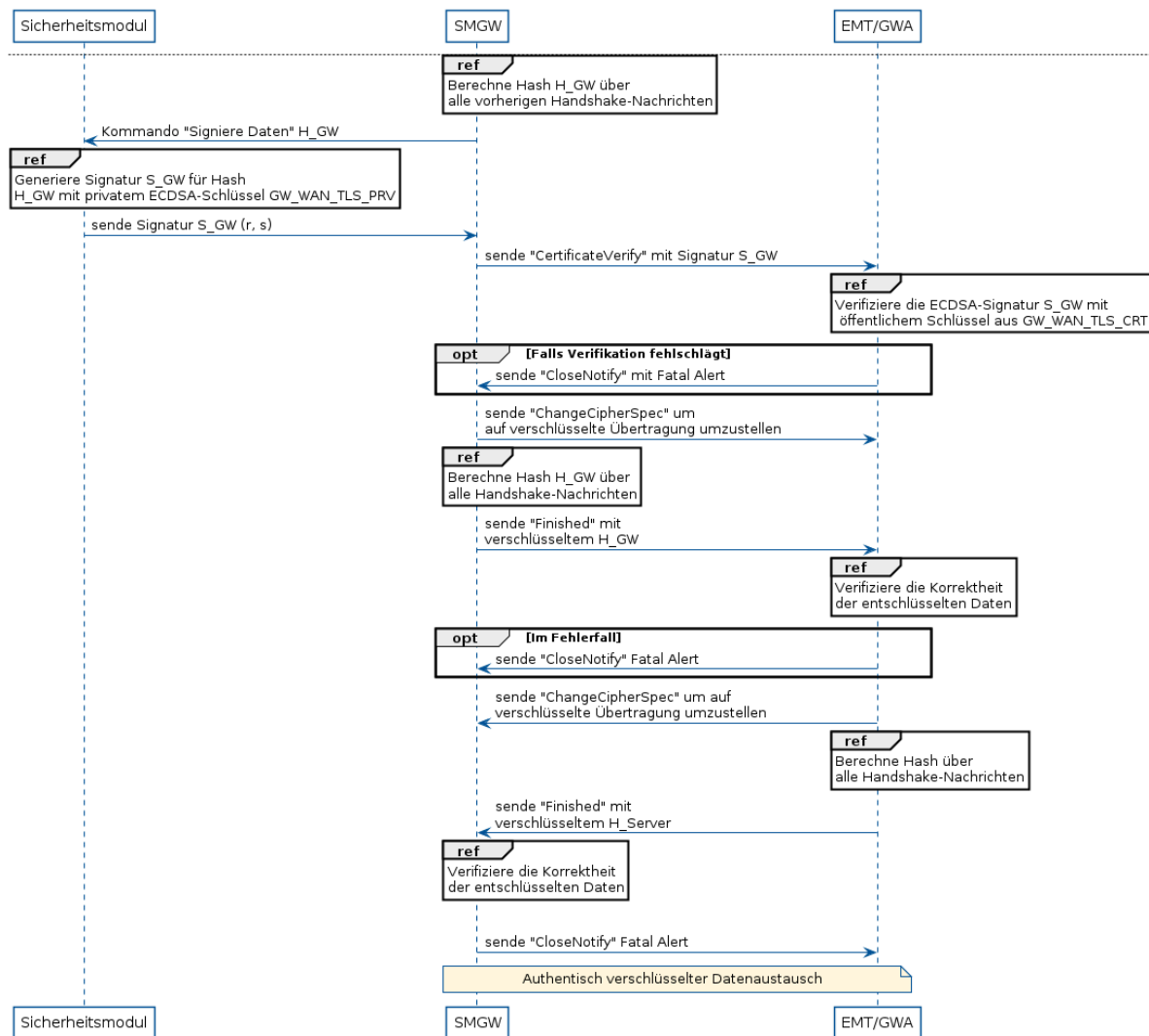


Abbildung 13.6. Interaktion zwischen Gateway und Sicherheitsmodul beim TLS 1.2-Handshake 2/2

Die Reihenfolge der TLS-Kommandos beim TLS-Handshake kann von der in ►Abbildung 13.5 und ►Abbildung 13.6 gezeigten Reihenfolge eventuell abweichen. Die Abbildungen sind diesbezüglich lediglich exemplarisch und informativ zu verstehen. Ausschlaggebend ist die Anforderung in [TR-03109-2], aus der sich Abhängigkeiten bezüglich der Reihenfolge der skizzierten Kommandos ergeben.

14 NTP - Network Time Protokoll

14.1 Einleitung

Dieses Kapitel beschreibt Anforderungen an die Nachrichten zur Zeitsynchronisation mit dem Network-Time-Protokoll (NTP) bzw. Simple Network Time Protocol (SNTP). Es konkretisiert die Anforderungen der Spezifikation [RFC5905] zur Implementierung von NTP. Die Anforderungen werden dabei zur Referenzierung aus Kommunikationsszenarien in die Kommunikationsrollen (Client, Server) und die Dienstprimitive (Request, Response) unterschieden.

Die Beschreibung der Anforderungen an die Sicherung und den Aufbau der Kommunikationsverbindung für NTP sind nicht Bestandteil dieses Kapitels.

14.1.1 Abweichungen von RFC5905

- Die NTP-Authentication wird nicht verwendet, da die Authentifizierung zertifikatsbasiert durch TLS realisiert wird.
- Die NTP-Extension-Felder werden nicht verwendet.
- Der UDP-Header ist nicht Bestandteil der NTP-Nachricht.
- Die Port-Nummer des Transport-Layers weicht aufgrund des TLS-Transports von [RFC5905] ab.
- Der Transport findet über ein verbindungsorientiertes und nicht über verbindungsloses Transportprotokoll statt.
- Es wird nur die Association-Betriebsart Client/Server Mode (ephemeral) verwendet.
- Eine Anforderung an die Implementierung zur Verarbeitung von sogenannten Kiss-Codes¹ besteht nicht.
- Falls die Systemzeit in der Implementierung nicht valide oder unbekannt ist, wird der erste vom Zeitserver authentisch empfangene Zeitstempel zum Setzen der Systemzeit verwendet, um keinen manuellen Benutzereingriff zu erfordern.
- Zur Verbesserung des Datenschutzes empfiehlt diese Spezifikation gemäß [RFC8633] für NTP-Clients die im Request ausgesandten Informationen zu minimieren.

14.2 Datenstrukturen

14.2.1 Einleitung

Diese Detailspezifikation verwendet folgende Datenstrukturen:

- Die Anfrage an den Zeitserver und die Antwort vom Zeitserver im NTP Packet-Header-Format (ohne UDP Header) siehe ▶NTPMessage.

14.2.2 NTPMessage

Die Datenstruktur NTPMessage entspricht dem NTP-Packet-Header Format nach [RFC5905] Figure 8 und wird hier zur Förderung des Verständnisses mit den in dieser Detailspezifikation referenzieren Bezeichnungen wiederholt. Bytefolgen, die Decimal- oder Integer-Datentypen darstellen, werden in Network Byte-Order (MSB first) übertragen.

Datenfeld	Datentyp (Wertebereich)	Beschreibung	Datenminimierung
leap	Bitstring 2 Bit (0-3)	Information des Servers zu Schaltsekunden nach [RFC5905] Figure 9.	0

¹ Kiss-of-Death Packets enthalten vier Zeichen SteuerCodes im Datenfeld "refId", die vom NTP-Server zum Steuern des Clients gesendet werden können. Eine nach RFC5905 vorgesehene Verwendung ist das Verlangsamen der Abfragerate des NTP-Clients.

Datenfeld	Datentyp (Wertebereich)	Beschreibung	Datenminimierung
version	Bitstring 3 Bit (4)	Version des NTP-Protokolls.	4
mode	Bitstring 3 Bit (3-4)	Assoziations-Modus nach [RFC5905] Figure 10.	3
stratum	Integer 1 Byte (0-16)	Stratum des Servers nach [RFC5905] Figure 11.	0
poll	Integer 1 Byte (4-17)	log2 (Poll-Intervall in Sekunden)	0 oder Poll-Intervall
precision	Integer 1 Byte (-18..0)	log2 (Präzision der Systemuhr in Sekundenbruchteilen)	32
rootDelay	Decimal (2 Byte Seconds, 2 Byte Fraction)	Gesamte Round-Trip-Zeit vom Client zum Root-Zeitserver und zurück zum Client in Sekunden/Sekundenbruchteilen (vom Server).	0
rootDispersion	Decimal (2 Byte Seconds, 2 Byte Fraction)	Streuung der Zeit bezüglich der Zeit des Root-Zeitserver in Sekunden/Sekundenbruchteilen (vom Server).	0
refId	Octet-String 4 Byte	- Identifikation des Zeitserver zur Vermeidung von Synchronisationsschleifen falls "stratum" ungleich 0 oder "leap" ungleich 3. - Kiss-Code nach [RFC5905] Figure 13 vom Server, falls "stratum" gleich 0 und "leap" gleich 3.	0
referenceTimestamp	Decimal (4 Byte Seconds, 4 Byte Fraction)	Zeitstempel des Clients in Sekunden/Sekundenbruchteilen, der angibt, wann die Systemzeit zuletzt gesetzt oder synchronisiert wurde.	0
originTimestamp	Decimal (4 Byte Seconds, 4 Byte Fraction)	Zeitstempel des Clients in Sekunden/Sekundenbruchteilen, der angibt, wann der NTP-Request zum Server erzeugt wurde.	0
receiveTimestamp	Decimal (4 Byte Seconds, 4 Byte Fraction)	Zeitstempel in Sekunden/Sekundenbruchteilen, der angibt, wann der NTP-Request beim Server empfangen wurde.	0
transmitTimestamp	Decimal (4 Byte Seconds, 4 Byte Fraction)	Zeitstempel in Sekunden/Sekundenbruchteilen, der angibt, wann die NTP-Response vom Server versandt wurde.	0 oder random

Tabelle 14.1 NTPMessage

Hinweise:

- Der "destinationTimestamp" wird als Empfangszeitstempel der NTP-Response beim Client gebildet.
- Extension-Felder sind nicht vorhanden.
- Da NTP durch TLS geschützt ist, werden die Datenfelder "dgst" und "Key identifier" nicht verwendet.
- Die in der Spalte "Datenminimierung" genannten Werte werden für die Aussendung von NTP-Client Requests empfohlen (siehe [RFC8633]). RFC5905-konforme NTP-Server akzeptieren diese NTP-Requests.

14.3 Technische Anwendungsfälle des NTP-Clients

Die folgenden Abschnitte enthalten die TA für die Implementierung eines NTP-Clients.

14.3.1 TA.NTP.BuildRequest

14.3.1.1 Beschreibung

In diesem TA werden die Anforderungen an die Implementierung des NTP-Clients zur Erzeugung eines NTP-Requests beschrieben, um eine Zeitsynchronisation anzufordern.

14.3.1.2 Anforderungen



REQ.TA.NTP.BuildRequest.10

Die Implementierung **MUSS** einen NTP-Request als NTPMessage der Version 4 für einen Client (Datenfeld "mode" gleich 3) gemäß [RFC5905] bilden.



REQ.TA.NTP.BuildRequest.20

Die Implementierung **MUSS** den NTP-Request ohne UDP-Header erzeugen und senden. ²



REQ.TA.NTP.BuildRequest.30

Die Implementierung **MUSS** den NTP-Request ohne NTP-Extensions senden. Der NTP-Request hat eine Länge von 48 Bytes. ³



REQ.TA.NTP.BuildRequest.40

Die Implementierung **SOLL** die Werte der Datenfelder des NTP-Request aus Gründen der Datensparsamkeit gemäß Spalte "Datenminimierung" aus ►Tabelle 14.1 setzen. (►ICS.TA.NTP.BuildRequest.10).

ICS



ICS.TA.NTP.BuildRequest.10

Der Hersteller **MUSS** im ICS deklarieren, ob die Datenminimierung von ausgesandten NTP-Client-Requests umgesetzt wird.

14.3.2 TA.NTP.ProcessResponse

14.3.2.1 Beschreibung

In diesem TA werden die Anforderungen an die Implementierung zur Verarbeitung einer NTP-Response vom Zeitserver beschrieben, um mit der NTP-Response eine Zeitsynchronisation durchzuführen. Die Systemzeit (UTC), die Information über die letzte erfolgreiche Synchronisation, der maximale Fehler (Abweichung der Systemzeit von der Root-Zeit = $\text{rootdelay}/2 + \text{rootdisp}$) und die geschätzte Genauigkeit (dispersion/jitter) werden der Fachanwendung bereitgestellt.

14.3.2.2 Hinweise für die Implementierung

Hinweis: Da die höchste Poll-Frequenz der Zeitsynchronisation durch andere Vorgaben dieser Spezifikation auf einmal pro Minute limitiert wird, ist eine Verarbeitung des Kiss-Codes "RATE" nicht vorgesehen.

Hinweis: Am 8. Februar 2036 beginnt die NTP Ära "1". Hier ist Aufmerksamkeit bei der Konvertierung in die Systemzeit und umgekehrt notwendig.

Hinweis: Am 19. Januar 2038 um 3:14:08 Uhr UTC sind Implementierungen, die vorzeichenbehaftete 32bit UTC-Counter verwenden, von einem Überlauf betroffen.

² Hinweis: Diese Anforderung stellt eine Änderung zu [RFC5905] Kapitel 7.3 dar.

³ Hinweis: Diese Anforderung schränkt [RFC5905] ein.

Hinweis: Falls Schaltsekunden-Ankündigungen bei der Konvertierung in die Systemzeit ausgewertet werden, sollte der Synchronisation mit der Systemzeit besondere Aufmerksamkeit geschenkt werden.

Hinweis: [RFC5905] Annex A.5.1 und A.5.1.1 geben Hinweise zur Plausibilisierung empfangener Nachrichten, bevor diese zur Zeitsynchronisation verwendet werden.

14.3.2.3 Anforderungen



REQ.TA.NTP.ProcessResponse.10

Die Implementierung **MUSS** Nachrichten ignorieren, die auf dem Transportweg unzulässig verzögert wurden, d.h. die Differenz aus Versandzeitpunkt (*ntpRequests* "originTimestamp") und Empfangszeitpunkt (*ntpResponse* "destinationTimestamp") sind größer als 9 Sekunden.



REQ.TA.NTP.ProcessResponse.30

Die Implementierung **MUSS** Nachrichten ignorieren, deren Zeitstempel "originTimestamp" nicht identisch mit dem zuletzt gesendeten Zeitstempel "transmitTimestamp" des NTP-Requests ist.



REQ.TA.NTP.ProcessResponse.40

Die Implementierung **MUSS** Nachrichten ignorieren, deren Zeitstempel des Datenfeldes "transmitTimestamp" nicht in dem vom Hersteller im ▶ICS.TA.NTP.ProcessResponse.10 deklarierten Zeitraum zur korrekten Konvertierung in die UTC-Systemzeit liegt.



REQ.TA.NTP.ProcessResponse.50

Die Implementierung **MUSS** Nachrichten mit Datenfeld "leap"=3 (Nicht synchronisierter NTP-Server) ignorieren.



REQ.TA.NTP.ProcessResponse.51

Die Implementierung **MUSS** Nachrichten mit Datenfeld "version" < "4" (Nicht RFC5905-konform) ignorieren.



REQ.TA.NTP.ProcessResponse.52

Die Implementierung **MUSS** Nachrichten mit Datenfeld "mode" ungleich "4" (Server) ignorieren.



REQ.TA.NTP.ProcessResponse.110

Falls die Systemzeit im synchronisierten Zustand ist, **SOLL** die Implementierung Nachrichten ignorieren, bei denen die Differenz zwischen "SystemTime" und empfangenem "transmitTimestamp" größer als 1000s ist.⁴

ICS



ICS.TA.NTP.ProcessResponse.10

Der Hersteller deklariert im ICS, bis zu welchem Datum die Implementierung empfangene Zeitstempel korrekt in die Zeitskala UTC konvertieren kann, um damit die Systemuhr zu synchronisieren.

⁴ Hinweis: Dadurch sollten im synchronisierten Zustand Angriffe durch starke Änderungen an der NTP-Zeit des Zeitserver keine Auswirkungen auf den Betrieb der Implementierung haben. Nachdem die Implementierung in einen unsynchronisierten Zustand übergeht, ist eine Neusynchronisierung mit dem Zeitserver möglich.

14.3.3 Technische Anwendungsfälle des NTP-Servers

Die folgenden Abschnitte enthalten die TA für die Implementierung eines NTP-Servers.

14.3.3.1 TA.NTP.ProcessRequest

14.3.3.1.1 Beschreibung

In diesem TA werden die Anforderungen an die Implementierung des NTP-Servers zur Verarbeitung eines empfangenen NTP-Requests beschrieben.

14.3.3.1.2 Anforderungen



REQ.TA.NTP.ProcessRequest.10

Die Implementierung **MUSS** einen NTP-Request mit einer Länge ungleich 48 Bytes, einem Datenfeld "mode" ungleich 3 (Client) oder einer "version" ungleich 4 ignorieren.

14.3.3.2 TA.NTP.BuildResponse

14.3.3.2.1 Beschreibung

In diesem TA werden die Anforderungen an die Implementierung zur Erzeugung einer NTP-Response beschrieben.

14.3.3.2.2 Hinweise für die Implementierung

Hinweis: Am 8. Februar 2036 beginnt die NTP Ära "1". Hier ist Aufmerksamkeit bei der Konvertierung der Systemzeit in den NTP-Zeitstempel notwendig.

Hinweis: Am 19. Januar 2038 um 3:14:08 Uhr UTC sind Implementierungen, die vorzeichenbehaftete 32bit UTC-Counter verwenden, von einem Überlauf betroffen.

Hinweis: An die Weitergabe von Schaltsekunden-Ankündigungen werden keine Anforderungen gestellt.

14.3.3.2.3 Anforderungen



REQ.TA.NTP.BuildResponse.10

Die Implementierung **MUSS** eine Nachricht mit "version" gleich 4, einem Datenfeld "mode" gleich 4 (Server) und einer Länge von 48 Bytes erzeugen.



REQ.TA.NTP.BuildResponse.20

Sofern die Systemzeit (z.B. nach Überschreiten der Gangreserverzeit) nicht mit dem Upstream-Zeitserver synchronisiert wurde **MUSS** die Implementierung das Datenfeld "leap" gleich 3 erzeugen.



REQ.TA.NTP.BuildResponse.30

Die Implementierung **MUSS** die Systemzeit mindestens mit einer Granularität von 0.5 Sekunden in das Datenfeld "transmitTimestamp" konvertieren.



REQ.TA.NTP.BuildResponse.40

Sofern die Systemzeit mit dem Upstream-Zeitserver synchronisiert wurde **MUSS** die Implementierung die Datenfelder "stratum", "leap", "refID", "poll", "precision", "rootDelay", "rootDispersion", "referenceTimestamp", "originTimestamp", "receiveTimestamp" aus den aktuellen NTP-Prozess-Variablen gemäß [RFC5905] Kapitel 9.2 erzeugen

**REQ.TA.NTP.BuildResponse.41**

Sofern die Systemzeit mit dem Upstream-Zeitserver synchronisiert wurde **MUSS** die Implementierung das Datenfeld "stratum" eins höher als das Stratum des Upstream-Servers in der Response übermitteln.

15 Messwertstatus

Zur Übermittlung von Messwert- und Gerätestatusinformationen wird eine Zusammenfassung von Datenbits als Statuswort verwendet. Die Bedeutung der Bits in den verschiedenen Statuswörtern der Messeinrichtungen (SML/HDLC, wMBUS) und das vom SMGW erzeugte kombinierte Statuswort werden in diesem Kapitel beschrieben.

In den folgenden Abschnitten wird das niederwertigste Bit mit LSB (Least-significant-bit) und das höchstwertige Bit mit MSB (Most-significant-bit) bezeichnet. Die Zuordnung von Bit 0 zu MSB und LSB ist vom verwendeten Protokoll und Schnittstelle abhängig.

15.1 Kombiniertes Statuswort - Codierung und Bedeutung

Tabelle ► Tabelle 15.1 beschreibt die Bedeutung und Zuordnung der Statusbits für das Statuswort, das aus dem 32-Bit Statuswort des SMGW und dem 32 Bit Statuswort der Messeinrichtung kombiniert wird.

Das Kombinierte Statuswort wird vom SMGW gebildet und an der WAN und HAN Schnittstelle übertragen.

Der Typ des kombinierten Statuswortes "drahtlos kommunizierende/drahtgebundene Messeinrichtung" lassen sich durch Bit 17 des SMGW-Statuswortes unterscheiden. Der Typ des Messgerätestatuswortes lässt sich durch Bit 2 und 3 des Messgeräte-Statuswortes unterscheiden.

Bit	Bedrahtet kommunizierende Messeinrichtung el. Energie (FNN LH BZ)	Bedrahtet kommunizierende Messeinrichtung Gas DVGW G697	Drahtlose kommunizierende Messeinrichtung (Gas DVGW G694, Wasser, Thermische Energie)
Status SMGW Wert und Bedeutung			
0	1	1	1
1	1: Messwerte nicht zur Abrechnung (SMGW_ValueNotValidated)		
2	1	1	1
3	0	0	0
4	0	0	0
5	0	0	0
6	0	0	0
7	0	0	0
8	1: SMGW hat fatalen Fehler erkannt (SMGW_Error)		
9	1: SMGW Systemzeit ungültig (SMGW_TimeNotValid)		
10	0	0	0
11	0	0	0
12	1: Warnung im SMGW liegt vor (SMGW_Warning)		
13	1: Kein Messwert im Empfangszeitfenster (SMGW_NoValue)		
14	1: Magnetische, mechanische, elektrische Manipulation in der Messeinrichtung erkannt (MTR_Tamper)		
15	0: FNN Basiszähler am SMGW	0	1: Fehler am Zähler nach OMS Vol 2 STS "Anomalie" oder "Permanent Error" erkannt (MTR_Error)
16	1: Messwertversand aufgrund Schwellwertüberschreitung (SMGW_ValueTriggered)		
17	0: Typ Statuswort des Zähler (FNN BZ, DVGW G697)		1: Typ Statuswort: Drahtloser Zähler OMS
18	0	0	0
19	0	0	0
20	0	0	0
21	0	0	0

Bit	Bedrahtet kommunizierende Messeinrichtung el. Energie (FNN LH BZ)	Bedrahtet kommunizierende Messeinrichtung Gas DVGW G697		Drahtlose kommunizierende Messeinrichtung (Gas DVGW G694, Wasser, Thermische Energie)	
22	0	0		0	
23	0	0		0	
24	0	0		0	
25	0	0		0	
26	0	0		0	
27	0	0		0	
28	0	0		0	
29	0	0		0	
30	0	0		0	
31	0	0		0	
Status Messeinrichtung Wert und Bedeutung (Status Eingangsmesswerte)					
0	0		Klasse nach G697	1: EF1 Fehlerbyte vorhanden	EF1
1	0	0		1: Magnetische Beeinflussung erkannt (MTR_Tamper)	
2	1: Typ Basiszähler, 0: Anderer Zähler	0		1: Verbindung zum Zähler/Ventil unterbrochen (MTR_Warning)	
3	0	1: Typ: Gas drahtgebunden, DVGW G697 (Klasse 1-3)	1-3	1: Allgemeiner Hardware/Softwarefehler (MTR_Error)	
4	0	0		1: Batterie getrennt/Abschaltung erkannt (MTR_Warning)	
5	0	0		1: Kritischer Fehler durch externe Eingabe erkannt (MTR_Fatal)	
6	0	0		1: Batterie schwach (MTR_Warning)	
7	0	0		1: Mechanische Manipulation erkannt (MTR_Tamper)	
8	1: Wirkleistung vorhanden, 0: Wirkleistungssumme zu gering	1: Alarm-Sammelmeldung (Temporär). (z.B. Neustart des Gerätes, Anlauf) (MTR_Warning)	1-3	1: EF2 Fehlerbyte vorhanden (Für Gas: 0)	EF2
9	1: Magnetische Manipulation (MTR_Tamper)	1: Umgehungsversuch. Magnetisch-Elektrische Manipulation (z.B. Kabelbruch) (MTR_Tamper)	1-3	1: Uhrensynchronisationsfehler erkannt (MTR_TimeNotValid)	
10	1: Mechanische Manipulation (Klemmdeckel oder Plombierstift offen) (MTR_Tamper)	1: Umgehungsversuch. Physisch - Mechanische Manipulation (z.B. Klemmdeckel geöffnet) (MTR_Tamper)	1-3	1: Zähler außerhalb des zulässigen Bereichs (MTR_Warning)	
11	Energierichtung Summe 1: -A, 0: +A	1: Störung Messwert Betriebsvolumen. Eichrechtlich relevant. Physisch - Mechanische Manipulation (MTR_Warning)	1-3	1: Verbrauch außerhalb des zulässigen Bereichs (MTR_Warning)	

Bit	Bedrahtet kommunizierende Messeinrichtung el. Energie (FNN LH BZ)	Bedrahtet kommunizierende Messeinrichtung Gas DVGW G697		Drahtlose kommunizierende Messeinrichtung (Gas DVGW G694, Wasser, Thermische Energie)	
12	Energierichtung L1 1: -A, 0: +A	1: Störung Messwert Druck oder Betriebsdichte (z.B. Störung Impulsgeber, Encoder, Grenzwert Durchfluss, Betriebsvolumen) (MTR_Warning)	3	1: Fehlender Durchfluss erkannt (MTR_Warning)	
13	Energierichtung L2 1: -A, 0: +A	1: Störung Messwert Temperatur oder Normdichte (MTR_Warning)	3	1: Leckage erkannt (MTR_Warning)	
14	Energierichtung L3 1: -A, 0: +A	1: Störung Berechnung K-Zahl (MTR_Warning)	3	1: Guthabengrenze überschritten	
15	Drehfeld: 0: L1->L2->L3, 1: Davon abweichend	1: Störung Berechnung Energie (MTR_Warning)	3	1: Nicht autorisierter Zugriffsversuch	
16	Rücklaufsperre: 1: Aktiviert	1: Messbereich unterschritten (MTR_Warning)	3	1: EF3 Fehlerbyte (Historische Fehler) vorhanden (Für Gas: 0)	EF3
17	1: Eichrechtl. Relevanter Fehler (MTR_Fatal)	1: Fehler eichrelevant (fatal) (eines der Bits 8, 11-16 gesetzt) (MTR_Fatal)	1-3	1: historischer Zähler außerhalb des zulässigen Bereichs	
18	1: L1 Spannung vorhanden	1: Messbereich überschritten (MTR_Warning)	3	1: historischer fehlender Durchfluss erkannt	
19	1: L2 Spannung vorhanden	1: Störung Quellgerät oder KE (MTR_Warning)	1-3	1: historische Leckage erkannt	
20	1: L3 Spannung vorhanden	1: Warnung Messbereich unterschritten (MTR_Warning)	3	1: historischer Externer Alarm erkannt	
21	0	1: Warnung Messbereich überschritten (MTR_Warning)	3	1: historische Verbindung unterbrochen erkannt	
22	0	1: Warnung Betriebsdauer Batterie (MTR_Warning)	1-3	1: historische magnetische Beeinflussung erkannt	
23	0	1: Füllstand metrologisches Logbuch > 90% (MTR_Warning)	3	1: historische mechanische Manipulation erkannt	
24	0	1: Warnung Füllstand metrologisches Logbuch (MTR_Warning)	3	herstellerspezifische Bedeutung	STS
25	0	1: Benutzereingriff: Abrechnungsrelevanter Wert (MTR_Warning)	1-3	herstellerspezifische Bedeutung	
26	0	1: Benutzereingriff: Parameter geändert (MTR_Warning)	2-3	herstellerspezifische Bedeutung	
27	0	1: Revisionsvermerk (MTR_Warning)	3	1: temporärer Fehler (MTR_Error)	
28	0	0: Zustand Eichschloss offen (MTR_Warning)	3	1: dauerhafter Fehler (MTR_Fatal)	
29	0	0		1: Energieversorgung niedrig (MTR_Warning)	
30	0	0		1: Weiterer Anwendungsfehler oder ungewöhnlicher Zustand (MTR_Error)	
31	0	0		1: Anwendung beschäftigt	

Tabelle 15.1 Bedeutung und Zuordnung der Bits im Kombinierten Statuswort

Zu beachten ist, dass die Bitnummerierung der EF1, EF2, EF3 und STS Statusbytes in der OMS Spezifikation für die LMN-Schnittstelle beschrieben ist und dabei jeweils das Bit mit der Nummer 7 das niederwertigste Bit ist. EF1-EF3 werden auch als SF1-SF3 bezeichnet.

Literaturverzeichnis

- [DIN VDE 0418-63-7] *DIN VDE V 0418-63-7 (VDE V 0418-63-7) Messeinrichtungen und -systeme - Teil 63-7: Protokollstapel der leitungsgelassenen LMN-Schnittstelle (Bisher E DIN 43863-7)*. 2023-08. VDE|DKE K461.
- [DIN VDE 0418-63-9] *DIN VDE 0418-63-9 (VDE 0418-63-9) Messeinrichtungen und -systeme - Teil 63-9: Intelligentes Kommunikationsprotokoll für Elektrizitätszähler (SML) (Bisher E DIN43863-9)*. 2022-02. VDE|DKE K461.
- [DIN43849] *DIN43849 - Messeinrichtungen und -systeme, sowie Zusatzeinrichtungen und Steuergeräte – Herstellerübergreifende Identifikationsnummer (Ersetzt DIN 43863-5:2021-04)*. 2024-05. VDE|DKE K461.
- [DRAFT-IETF-AFT-SOCKS-SSL-00] *Secure Sockets Layer for SOCKS Version 5*. IETF.
- [DS] *TR-03109-1 Detailspezifikationen*. Bundesamt für Sicherheit in der Informationstechnik. Aktuelle Fassung.
- [EN13757-3] *DIN EN 13757-3 - Kommunikationssysteme für Zähler – Teil 3: Anwendungsprotokolle; Deutsche Fassung EN 13757-3:2018*. 2018-06. DIN/CEN TC294.
- [EN13757-4] *DIN EN 13757-4 - Kommunikationssysteme für Zähler – Teil 4: Drahtlose M-Bus-Kommunikation; Englische Fassung EN 13757-4:2019*. 2019-09. DIN/CEN TC294.
- [EN13757-7] *DIN EN 13757-7 - Kommunikationssysteme für Zähler – Teil 7: Transport- und Sicherheitsdienste; Deutsche Fassung EN 13757-7:2018*. 2018-06. DIN/CEN TC294.
- [EN62056-6-2] *DIN EN IEC 62056-6-2: Datenkommunikation der elektrischen Energiemessung – DLMS/COSEM – Teil 6-2: Interface Klassen (IEC 62056-6-2:2017); Englische Fassung EN IEC 62056-6-2:2018 Electricity metering data exchange - The DLMS/COSEM suite - Part 6-2: COSEM Interface classes*. IEC TC13. 2021-06.
- [ISO13239] *ISO 13239 - Information technology -- Telecommunications and information exchange between systems -- High-level data link control (HDLC) procedures*. ISO/IEC JTC 1/SC 6. 2002.
- [OMS-S2] *Open Metering System Specification – Volume 2 Primary Communication – Issue 5.0.1*. Dezember 2023. OMS-Group.
- [OMS-TR07] *Open Metering System Technical Report 07 - Zählerstandsübertragung mittels M-Bus-Kompaktprofil – Version 1.0.1*. 2020. OMS-Group.
- [RFC1035] *Domain names - implementation and specification*. IETF und P. Mockapetris. 1987.
- [RFC1928] *SOCKS Protocol Version 5*. IETF.
- [RFC1950] *ZLIB Compressed Data Format Specification version 3.3*. IETF, L. Peter Deutsch und Jean-Loup Gailly. 1996.
- [RFC1951] *DEFLATE Compressed Data Format Specification version 1.3*. IETF und L. Peter Deutsch. 1996.
- [RFC1952] *GZIP Compressed Data Format Specification version 4.3*. IETF und L. Peter Deutsch. 1996.
- [RFC3274] *Compressed Data Content Type for Cryptographic Message Syntax (CMS)*. IETF und P. Gutmann. 2002.
- [RFC3339] *Date and Time on the Internet: Timestamps*. IETF. July 2002.
- [RFC3394] *Advanced Encryption Standard (AES) Key Wrap Algorithm*. IETF, J. Schaad und R. Housley. 2002.
- [RFC3565] *Use of the Advanced Encryption Standard (AES) Encryption Algorithm in Cryptographic Message Syntax (CMS)*. IETF und J. Schaad. 2003.
- [RFC3986] *Uniform Resource Identifier (URI): Generic Syntax*. IETF, Tim Berners-Lee, Roy T. Fielding und Larry Masinter. 2005.
- [RFC4493] *The AES-CMAC Algorithm*. IETF, TH Song, R. Poovendran, J. Lee und T. Iwata. 2006.
- [RFC5083] *Cryptographic Message Syntax (CMS) Authenticated-Enveloped-Data Content Type*. IETF und R. Housley. 2007.
- [RFC5084] *Using AES-CCM and AES-GCM Authenticated Encryption in the Cryptographic Message Syntax (CMS)*. IETF und R. Housley. 2007.

- [RFC5234] *Augmented BNF for Syntax Specifications: ABNF*. 2008.
- [RFC5246] *The Transport Layer Security (TLS) Protocol Version 1.2*. IETF, T. Dierks und E. Rescorla. 2008.
- [RFC5280] *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. IETF, D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley und W. Polk. 2008.
- [RFC5289] *TLS Elliptic Curve Cipher Suites with SHA-256/384 and AES Galois Counter Mode (GCM)*. IETF und E. Rescorla. 2008.
- [RFC5480] *Elliptic Curve Cryptography Subject Public Key Information*. IETF, Turner, Brown, Yiu, Housley und Polk. 2009.
- [RFC5652] *Cryptographic Message Syntax (CMS)*. IETF und R. Housley. 2009.
- [RFC5754] *Using SHA2 Algorithms with Cryptographic Message Syntax*. IETF und S. Turner. 2010.
- [RFC5905] *Network Time Protocol Version 4: Protocol and Algorithms Specification*. IETF, D. Mills, J. Martin, J. Burbank und W. Kasch. 2010.
- [RFC5915] *Elliptic Curve Private Key Structure*. IETF, S. Turner und D. Brown. 2010.
- [RFC6066] *Transport Layer Security (TLS) Extensions: Extension Definitions*. IETF und D. Eastlake. 2011.
- [RFC6762] *Multicast DNS*. IETF. Februar 2013.
- [RFC6763] *DNS-Based Service Discovery*. IETF. Februar 2013.
- [RFC7027] *Elliptic Curve Cryptography (ECC) Brainpool Curves for Transport Layer Security (TLS)*. IETF, J. Merkle, und M. Lochter. 2013.
- [RFC7193] *The application/cms Media Type*. IETF, S. Turner, R. Housley und J. Schaad. 2014.
- [RFC7301] *Transport Layer Security (TLS) - Application-Layer Protocol Negotiation Extension*. IETF, S. Friedl, A. Langley und E. Stephan. 2014.
- [RFC7366] *Encrypt-then-MAC for Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)*. IETF und P. Gutmann. 2014.
- [RFC7616] *HTTP Digest Access Authentication*. IETF. September 2015.
- [RFC7627] *Transport Layer Security (TLS) Session Hash and Extended Master Secret Extension*. IETF und K. Bhargavan. 2015.
- [RFC8259] *The JavaScript Object Notation (JSON) Data Interchange Format*. IETF und T. Bray. 2017.
- [RFC8422] *Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS) Versions 1.2 and Earlier*. IETF, Y. Nir, S. Josefsson und M. Pegourie-Gonnard. 2018.
- [RFC8446] *The Transport Layer Security (TLS) Protocol Version 1.3*. IETF. August 2018.
- [RFC8449] *Record Size Limit Extension for TLS*. IETF. August 2018.
- [RFC8633] *Network Time Protocol Best Current Practices*. IETF. July 2019.
- [RFC8734] *Elliptic Curve Cryptography (ECC) Brainpool Curves for Transport Layer Security (TLS) Version 1.3*. IETF. February 2020.
- [RFC8882] *DNS-Based Service Discovery (DNS-SD) Privacy and Security Requirements*. IETF. September 2020.
- [RFC9110] *HTTP Semantics*. IETF, R. Fielding, M. Nottingham und J. Reschke. 2022.
- [RFC9112] *HTTP/1.1*. IETF, R. Fielding, M. Nottingham und J. Reschke. 2022.
- [SemVer] *Semantic Versioning 2.0.0*.  semver.org [<https://semver.org>]
- [SM-PKI-CP] *SM-PKI-CP - Certificate Policy für die SM-PKI v1.1.1*. 2017. Bundesamt für Sicherheit in der Informationstechnik.
- [TR-03109-1v2.0] Bundesamt für Sicherheit in der Informationstechnik. *Technische Richtlinie TR-03109-1, Version 2.0 (RC1): Anforderungen an die Interoperabilität der Kommunikationseinheit eines intelligenten Messsystems*. 2024.

- [TR-03109-2] *Technische Richtlinie BSI-TR-03109-2: Anforderungen an die Funktionalität und Interoperabilität des Sicherheitsmoduls*. 2014. Bundesamt für Sicherheit in der Informationstechnik.
- [TR-03109-3] *Technische Richtlinie BSI-TR-03109-3: Kryptographische Vorgaben für die Infrastruktur von intelligenten Messsystemen*. 2014. Bundesamt für Sicherheit in der Informationstechnik.
- [TR-03109-4] *Technische Richtlinie BSI-TR-03109-4: Smart Metering PKI - Public Key Infrastruktur für Smart Meter Gateways*. 2014. Bundesamt für Sicherheit in der Informationstechnik.
- [TR-03109-5] Bundesamt für Sicherheit in der Informationstechnik. *Technische Richtlinie TR-03109-5 Version 1.0, Kommunikationsadapter*. 2023.
- [TR-03111] *Technische Richtlinie BSI-TR-03111 v2.10 Elliptic Curve Cryptography*. 2018. Bundesamt für Sicherheit in der Informationstechnik.
- [TR-03116-3] *BSI TR-03116-3: Kryptographische Vorgaben für Projekte der Bundesregierung Teil 3 - Intelligente Messsysteme*. Jährlich aktualisiert. Bundesamt für Sicherheit in der Informationstechnik.
- [X.200] *Information technology – Open Systems Interconnection – Basic Reference Model: The basic model*. 07/1994. ITU.
- [X.690] *ASN.1 encoding rules: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER)*. 07/2002. ITU.
- [XML1.0] World Wide Web Consortium. *Extensible Markup Language (XML) 1.0 (Fifth Edition)*. 2008.

Glossar

MTR_LMN_TLS_CERT	Das vom SMGW erzeugte LMN-Zertifikat für die TLS-Kommunikation des Zählers. Das Zertifikat hat die Eigenschaften eines X.509v3-Zertifikates basierend auf [RFC5280]. Das erste TLS-Zertifikat des Zählers wird auch als MTR_LMN_TLS_CERT_0 bezeichnet, während folgende dann mit MTR_LMN_TLS_CERT_1, MTR_LMN_TLS_CERT_2 usw. bezeichnet werden.
GW_LMN_TLS_CERT	Das vom SMGW erzeugte, selbstsignierte LMN-Zertifikat des SMGW zur TLS-Kommunikation mit dem Zähler. Das Zertifikat hat die Eigenschaften eines LMN-Zertifikates. Das erste im SMGW vorhandene LMN-Zertifikat des SMGW wird auch als GW_LMN_TLS_CERT_0 bezeichnet während folgende dann mit GW_LMN_TLS_CERT_1, GW_LMN_TLS_CERT_2 usw. bezeichnet werden.
GWA_WAN_SIG_CERT	Das Inhaltsdaten-Signatur-Zertifikat des GWA mit dem Zertifikatsprofil eines C_{Sign}(GWA) nach [TR-03109-4] Anhang A.
SubjectCN	Das "commonName" Attribut des X.520 "distinguishedName" Attributes des "Subject"-Datenfeldes eines X.509-Zertifikates. Enthält den Namen des Zertifikatsinhabers.
QueryParameter	Liste von Namen und Werten, die bei einer Abfrage zur Selektion von Eigenschaften zur Begrenzung der abgefragten Daten dienen.
GW_HAN_TLS_CERT	Das TLS-Authentifizierungszertifikat des SMGW an der HAN-Schnittstelle basierend auf [RFC5280] Kapitel 4.
SRV_HAN_TLS_CERT	Das TLS-Authentifizierungszertifikat des Servicetechnikers als HAN-Teilnehmer mit dem basierend auf [RFC5280] Kapitel 4
CON_HAN_TLS_CERT	Das TLS-Authentifizierungszertifikat des Anschlussnutzers (Consumer) als HAN-Teilnehmer basierend auf [RFC5280] Kapitel 4
CLS_HAN_TLS_CERT	Das TLS-Authentifizierungszertifikat eines CLS-Gerätes als HAN-Teilnehmer basierend auf [RFC5280] Kapitel 4
GWACA_SIG_CERT	Das Signaturzertifikat der CA des GWA, die die Zertifikate der Servicetechniker für Diagnose an der HAN-Schnittstelle des SMGW ausstellt. Das Zertifikat hat die Eigenschaften eines X.509v3-Zertifikates basierend auf [RFC5280] Kapitel 4
GWHCA_SIG_CERT	Das Signaturzertifikat der CA des GWH, die die Zertifikate der Servicetechniker für Diagnose an der HAN-Schnittstelle des SMGW ausstellt. Das Zertifikat hat die Eigenschaften eines X.509v3-Zertifikates basierend auf [RFC5280] Kapitel 4

Anhang A Abkürzungsverzeichnis

Abkürzung	Beschreibung
aEMT	Aktiver Externer Marktteilnehmer
AFL	Authentication and Fragmentation Layer, Wireless MBUS
API	Application Programming Interface
APL	Application Protocol Layer, Wireless MBUS
ARP	Address Resolution Protocol
ASN	Abstract Syntax Notation
BER	Basic Encoding Rules (ASN.1)
BSI	Bundesamt für Sicherheit in der Informationstechnik
CA	Certification Authority
CLS	Controllable Local System
CMS	Cryptographic Message Syntax, Inhaltsdatensicherung nach ASN.1
CON	Consumer bzw. Anschlussnutzer
COSEM	COmpanion Specification for Energy Metering
DER	Distinguished Encoding Rules (ASN.1)
DS	Detailspezifikation
EMT	Externer Marktteilnehmer
GWA	Smart-Meter-Gateway-Administrator
GWH	Smart-Meter-Gateway-Hersteller
HAN	Home Area Network
HDLC	High Level Data Link Control
HKS	HAN-Kommunikationsszenario
HTTP	HyperText Transfer Protocol
IC	Interface Class (für COSEM)
ICS	Implementation Conformance Statement
IETF	Internet Engineering Task Force
IP	Internet Protocol
KS	Kommunikationsszenario
LKS	LMN-Kommunikationsszenario
LMN	Local Meter Network
LSB	Least Significant Bit
wM-Bus	Wireless Meter Bus
MK	Main Key (ein PSK)
MSB	Most Significant Bit
MTR	Messeinrichtung
N/A	Nicht anwendbar
NTP	Network Time Protocol
OBIS	OBject Identification System (für COSEM)
PSK	Pre-Shared Key, zuvor vereinbarter symmetrischer Schlüssel
PTB	Physikalisch-Technische Bundesanstalt

Abkürzung	Beschreibung
RFC	Request For Comments
RTT	Round Trip Time
SM	Sicherheitsmodul
SM-PKI	Smart-Meter - Public Key Infrastructure
SMGW	Smart-Meter-Gateway
SML	Smart Message Language
SNI	Server Name Indication
SRV	Servicetechniker des SMGW
TA	Technischer Anwendungsfall
TCP	Transmission Control Protocol
TLS	Transport Layer Security, Transportsicherungsprotokoll
TPL	Transport Protocol Layer, Wireless MBUS
TR	Technische Richtlinie
UDP	User Datagram Protocol
UTC	Coordinated Universal Time, Zeitskala
WAN	Wide Area Network
WKS	WAN-Kommunikationsszenario
XML	Extendable Markup Language

Tabelle A.1 In der TR verwendete Abkürzungen

Anforderungsverzeichnis

REQ.BenutzerAuth.Allgemein.10	36
REQ.BenutzerAuth.Allgemein.20	36
REQ.BenutzerAuth.Allgemein.30	36
REQ.CMS.AesCbcCmac.10	23
REQ.CMS.AesCbcCmac.100	23
REQ.CMS.AesCbcCmac.110	23
REQ.CMS.AesCbcCmac.120	23
REQ.CMS.AesCbcCmac.20	23
REQ.CMS.AesCbcCmac.30	23
REQ.CMS.AesCbcCmac.40	23
REQ.CMS.AesCbcCmac.50	23
REQ.CMS.AesCbcCmac.60	23
REQ.CMS.AesCbcCmac.70	23
REQ.CMS.AesCbcCmac.80	23
REQ.CMS.AesCbcCmac.90	23
REQ.CMS.AuthEnvelopedData.10	16
REQ.CMS.AuthEnvelopedData.20	16
REQ.CMS.AuthEnvelopedData.25	19
REQ.CMS.AuthEnvelopedData.30	19
REQ.CMS.AuthEnvelopedData.40	20
REQ.CMS.AuthEnvelopedData.50	20
REQ.CMS.AuthEnvelopedData.60	20
REQ.CMS.AuthEnvelopedData.70	20
REQ.CMS.AuthEnvelopedData.80	20
REQ.CMS.AuthEnvelopedData.90	20
REQ.CMS.CompressedData.10	23
REQ.CMS.ContentInfo.10	16
REQ.CMS.ContentInfo.20	16
REQ.CMS.MessageCipherSuite.10	15
REQ.CMS.MessageCipherSuite.20	15
REQ.CMS.SignedData.10	20
REQ.CMS.SignedData.20	20
REQ.CMS.SignedData.30	20
REQ.CMS.SignedData.40	20
REQ.CMS.SignedData.50	22

REQ.CMS.SignedData.60	22
REQ.CMS.SignedData.70	22
REQ.CMS.Zertifikate.10	15
REQ.CMS.Zertifikate.20	15
REQ.HTTP.QueryParameter.10	33
REQ.SML.CLI.RequestMeterData.10	43
REQ.SML.CLI.RequestMeterData.20	43
REQ.SML.CLI.RequestMeterData.30	43
REQ.SML.CLI.ResetTamperState.10	47
REQ.SML.CLI.ResetTamperState.20	47
REQ.SML.CLI.UpdateMtrCertAndKey.10	42
REQ.SML.CLI.UpdateSmgwCert.10	42
REQ.SYMHDL.C.SMGW.InitialPairing.10	38
REQ.SYMHDL.C.SMGW.RemovePairingInMtr.10	39
REQ.TA.mDNS.BuildQuery.20	61
REQ.TA.mDNS.BuildQuery.30	61
REQ.TA.mDNS.BuildResponse.20	60
REQ.TA.mDNS.BuildResponse.30	60
REQ.TA.mDNS.ProcessQuery.10	60
REQ.TA.mDNS.ProcessQuery.20	60
REQ.TA.mDNS.ProcessResponse.10	61
REQ.TA.mDNS.ProcessResponse.20	62
REQ.TA.mDNS.Querying.10	61
REQ.TA.mDNS.Querying.20	61
REQ.TA.mDNS.Querying.30	61
REQ.TA.mDNS.Querying.40	61
REQ.TA.mDNS.Querying.50	61
REQ.TA.mDNS.Responder.10	59
REQ.TA.mDNS.Responder.20	60
REQ.TA.mDNS.Responder.30	60
REQ.TA.mDNS.Responder.40	60
REQ.TA.mDNS.Responder.50	60
REQ.TA.NTP.BuildRequest.10	83
REQ.TA.NTP.BuildRequest.20	83
REQ.TA.NTP.BuildRequest.30	83
REQ.TA.NTP.BuildRequest.40	83

REQ.TA.NTP.BuildResponse.10	85
REQ.TA.NTP.BuildResponse.20	85
REQ.TA.NTP.BuildResponse.30	85
REQ.TA.NTP.BuildResponse.40	85
REQ.TA.NTP.BuildResponse.41	86
REQ.TA.NTP.ProcessRequest.10	85
REQ.TA.NTP.ProcessResponse.10	84
REQ.TA.NTP.ProcessResponse.110	84
REQ.TA.NTP.ProcessResponse.30	84
REQ.TA.NTP.ProcessResponse.40	84
REQ.TA.NTP.ProcessResponse.50	84
REQ.TA.NTP.ProcessResponse.51	84
REQ.TA.NTP.ProcessResponse.52	84
REQ.TA.SOCKSTLS.ClosingHandshake.10	58
REQ.TA.SOCKSTLS.ClosingHandshake.20	58
REQ.TA.SOCKSTLS.Connect.20	55
REQ.TA.SOCKSTLS.Connect.30	56
REQ.TA.SOCKSTLS.Connect.35	56
REQ.TA.SOCKSTLS.Connect.36	56
REQ.TA.SOCKSTLS.Connect.40	56
REQ.TA.SOCKSTLS.Connect.50	56
REQ.TA.SOCKSTLS.Connect.60	56
REQ.TA.SOCKSTLS.Connect.70	56
REQ.TA.SOCKSTLS.Connect.80	56
REQ.TA.SOCKSTLS.ConnectRequest.10	54
REQ.TA.SOCKSTLS.ConnectRequest.20	54
REQ.TA.SOCKSTLS.ConnectRequest.30	55
REQ.TA.SOCKSTLS.ConnectRequest.35	55
REQ.TA.SOCKSTLS.ConnectRequest.40	55
REQ.TA.SOCKSTLS.ConnectRequest.50	55
REQ.TA.SOCKSTLS.ConnectRequest.60	55
REQ.TA.SOCKSTLS.ConnectRequest.70	55
REQ.TA.SOCKSTLS.ConnectRequest.80	55
REQ.TA.SOCKSTLS.InitiateAuthNegotiation.10	54
REQ.TA.SOCKSTLS.InitiateAuthNegotiation.20	54
REQ.TA.SOCKSTLS.InitiateAuthNegotiation.30	54

REQ.TA.SOCKSTLS.NegotiateAuth.10	55
REQ.TA.SOCKSTLS.NegotiateAuth.20	55
REQ.TA.SOCKSTLS.NegotiateAuth.30	55
REQ.TA.SOCKSTLS.ReceiveApplicationData.10	57
REQ.TA.SOCKSTLS.ReceiveSocksTlsRecord.10	57
REQ.TA.SOCKSTLS.SendApplicationData.10	56
REQ.TA.SOCKSTLS.SendSocksTlsRecord.10	57
REQ.TA.TLS.Close.10	71
REQ.TA.TLS.Close.20	71
REQ.TA.TLS.Close.30	71
REQ.TA.TLS.Close.40	71
REQ.TA.TLS.Close.60	71
REQ.TA.TLS.Close.70	72
REQ.TA.TLS.Close.80	72
REQ.TA.TLS.Close.90	72
REQ.TA.TLS.Handshake.10	68
REQ.TA.TLS.Handshake.110	69
REQ.TA.TLS.Handshake.120	69
REQ.TA.TLS.Handshake.130	69
REQ.TA.TLS.Handshake.140	69
REQ.TA.TLS.Handshake.145	69
REQ.TA.TLS.Handshake.150	69
REQ.TA.TLS.Handshake.155	70
REQ.TA.TLS.Handshake.160	70
REQ.TA.TLS.Handshake.170	70
REQ.TA.TLS.Handshake.180	70
REQ.TA.TLS.Handshake.190	70
REQ.TA.TLS.Handshake.20	68
REQ.TA.TLS.Handshake.30	68
REQ.TA.TLS.Handshake.50	68
REQ.TA.TLS.Handshake.60	69
REQ.TA.TLS.Handshake.70	69
REQ.TA.TLS.Handshake.80	69
REQ.TA.TLS.Handshake.90	69
REQ.TA.TLS.OpenHanSessionAsClient.10	74
REQ.TA.TLS.OpenHanSessionAsClient.20	74

REQ.TA.TLS.OpenHanSessionAsClient.30	74
REQ.TA.TLS.OpenHanSessionAsClient.40	74
REQ.TA.TLS.OpenHanSessionAsServer.10	75
REQ.TA.TLS.OpenHanSessionAsServer.20	76
REQ.TA.TLS.OpenLmnSessionAsClient.10	76
REQ.TA.TLS.OpenLmnSessionAsClient.30	76
REQ.TA.TLS.OpenLmnSessionAsClient.40	76
REQ.TA.TLS.OpenLmnSessionAsClient.50	76
REQ.TA.TLS.OpenLmnSessionAsClient.60	76
REQ.TA.TLS.OpenLmnSessionAsClient.70	76
REQ.TA.TLS.OpenLmnSessionAsServer.10	78
REQ.TA.TLS.OpenLmnSessionAsServer.20	78
REQ.TA.TLS.OpenLmnSessionAsServer.30	78
REQ.TA.TLS.OpenLmnSessionAsServer.40	78
REQ.TA.TLS.OpenWanSessionAsClient.10	72
REQ.TA.TLS.OpenWanSessionAsClient.20	72
REQ.TA.TLS.OpenWanSessionAsClient.30	72
REQ.TA.TLS.OpenWanSessionAsClient.40	72
REQ.TA.TLS.OpenWanSessionAsClient.50	72
REQ.TA.TLS.OpenWanSessionAsClient.60	73
REQ.TLSHDLC.SMGW.10	38
REQ.TLSHDLC.SMGW.100	43
REQ.TLSHDLC.SMGW.20	38
REQ.TLSHDLC.SMGW.30	38
REQ.TLSHDLC.SMGW.40	38
REQ.TLSHDLC.SMGW.50	44
REQ.TLSHDLC.SMGW.60	44
REQ.TLSHDLC.SMGW.70	44
REQ.TLSHDLC.SMGW.80	43
REQ.TLSHDLC.SMGW.90	43
REQ.WakeUp.Datenstruktur.10	2
REQ.WakeUp.Datenstruktur.20	3
REQ.WakeUp.Datenstruktur.30	3
REQ.Webservice.HttpClient.10	29
REQ.Webservice.HttpClient.100	29
REQ.Webservice.HttpClient.110	29

REQ.Webservice.HttpClient.120	29
REQ.Webservice.HttpClient.125	29
REQ.Webservice.HttpClient.130	29
REQ.Webservice.HttpClient.50	29
REQ.Webservice.HttpClient.60	29
REQ.Webservice.HttpClient.70	29
REQ.Webservice.HttpClient.80	29
REQ.Webservice.HttpClient.90	29
REQ.Webservice.HttpServer.10	26
REQ.Webservice.HttpServer.110	27
REQ.Webservice.HttpServer.120	27
REQ.Webservice.HttpServer.130	27
REQ.Webservice.HttpServer.140	27
REQ.Webservice.HttpServer.150	27
REQ.Webservice.HttpServer.160	27
REQ.Webservice.HttpServer.170	27
REQ.Webservice.HttpServer.180	27
REQ.Webservice.HttpServer.190	28
REQ.Webservice.HttpServer.20	26
REQ.Webservice.HttpServer.201	28
REQ.Webservice.HttpServer.202	28
REQ.Webservice.HttpServer.203	28
REQ.Webservice.HttpServer.210	28
REQ.Webservice.HttpServer.220	28
REQ.Webservice.HttpServer.230	28
REQ.Webservice.HttpServer.240	28
REQ.Webservice.HttpServer.250	28
REQ.Webservice.HttpServer.30	26
REQ.Webservice.HttpServer.60	26
REQ.Webservice.HttpServer.62	26
REQ.Webservice.HttpServer.70	26
REQ.Webservice.HttpServer.80	26
REQ.Webservice.HttpServer.90	27
REQ.Webservice.HttpServer.91	27
REQ.WMBUS.apl.10	52
REQ.WMBUS.apl.11	52

REQ.WMBUS.apl.12	52
REQ.WMBUS.apl.13	52
REQ.WMBUS.apl.20	52
REQ.WMBUS.apl.30	52
REQ.WMBUS.apl.40	52
REQ.WMBUS.apl.50	53
REQ.WMBUS.apl.60	53
REQ.WMBUS.apl.70	53
REQ.WMBUS.apl.80	53
REQ.WMBUS.Unidirektional.30	51
REQ.WMBUS.Unidirektional.36	51
REQ.WMBUS.Unidirektional.40	51
REQ.WMBUS.Unidirektional.50	51
REQ.WMBUS.Unidirektional.51	51
REQ.WMBUS.Unidirektional.60	51
REQ.WMBUS.Unidirektional.71	51
REQ.WMBUS.Unidirektional.72	52
REQ.ZertifikateHAN.Allgemein.10	9
REQ.ZertifikateLMN.Allgemein.10	4

ICS-Anforderungen

ICS.BenutzerAuth.Allgemein.10	36
ICS.BenutzerAuth.Allgemein.20	36
ICS.CMS.AuthEnvelopedData.10	19
ICS.CMS.AuthEnvelopedData.20	20
ICS.CMS.AuthEnvelopedData.30	20
ICS.CMS.SignedData.10	22
ICS.TA.NTP.BuildRequest.10	83
ICS.TA.NTP.ProcessResponse.10	84
ICS.TA.SOCKSTLS.Connect.10	56
ICS.TA.SOCKSTLS.ConnectRequest.10	55
ICS.TA.TLS.Handshake.10	70
ICS.TA.TLS.Handshake.20	70
ICS.TA.TLS.Handshake.30	70
ICS.TA.TLS.HanHandshake.10	74
ICS.TA.TLS.HanHandshake.100	75
ICS.TA.TLS.HanHandshake.20	74

ICS.TA.TLS.HanHandshake.30	75
ICS.TA.TLS.HanHandshake.40	75
ICS.TA.TLS.HanHandshake.50	75
ICS.TA.TLS.HanHandshake.60	75
ICS.TA.TLS.HanHandshake.70	75
ICS.TA.TLS.HanHandshake.80	75
ICS.TA.TLS.HanHandshake.90	75
ICS.TA.TLS.LmnHandshake.10	76
ICS.TA.TLS.LmnHandshake.100	77
ICS.TA.TLS.LmnHandshake.110	77
ICS.TA.TLS.LmnHandshake.15	77
ICS.TA.TLS.LmnHandshake.20	77
ICS.TA.TLS.LmnHandshake.30	77
ICS.TA.TLS.LmnHandshake.40	77
ICS.TA.TLS.LmnHandshake.50	77
ICS.TA.TLS.LmnHandshake.60	77
ICS.TA.TLS.LmnHandshake.70	77
ICS.TA.TLS.LmnHandshake.80	77
ICS.TA.TLS.LmnHandshake.90	77
ICS.TA.TLS.WanHandshake.10	73
ICS.TA.TLS.WanHandshake.100	74
ICS.TA.TLS.WanHandshake.20	73
ICS.TA.TLS.WanHandshake.30	73
ICS.TA.TLS.WanHandshake.40	73
ICS.TA.TLS.WanHandshake.41	73
ICS.TA.TLS.WanHandshake.50	73
ICS.TA.TLS.WanHandshake.60	73
ICS.TA.TLS.WanHandshake.70	73
ICS.TA.TLS.WanHandshake.80	73
ICS.TA.TLS.WanHandshake.90	74
ICS.WMBUS.apl.10	53
ICS.WMBUS.Mode10.10	52
ICS.ZertifikateHAN.Allgemein.10	9
ICS.ZertifikateHAN.Allgemein.11	9
ICS.ZertifikateHAN.Allgemein.12	9
ICS.ZertifikateHAN.Allgemein.13	9

ICS.ZertifikateHAN.Allgemein.20	9
ICS.ZertifikateHAN.Allgemein.21	9
ICS.ZertifikateHAN.Allgemein.30	10
ICS.ZertifikateLMN.Allgemein.10	4
ICS.ZertifikateLMN.Allgemein.11	4